

Red Hat Data Grid 8.6

Data Grid Server Guide

Deploy, secure, and manage Data Grid Server deployments

Last Updated: 2026-04-15

Red Hat Data Grid 8.6 Data Grid Server Guide

Deploy, secure, and manage Data Grid Server deployments

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Install and configure Data Grid Server deployments.

Table of Contents

RED HAT DATA GRID 8

DATA GRID DOCUMENTATION 9

DATA GRID DOWNLOADS 10

MAKING OPEN SOURCE MORE INCLUSIVE 11

CHAPTER 1. GETTING STARTED WITH DATA GRID SERVER 12
1.1. DATA GRID SERVER CONTAINER IMAGE 12
1.1.1. Container registries 12
1.1.2. Container execution 12
1.1.3. Hot Rod Clients 13
1.1.4. Identities Batch 13
1.1.5. Server Configuration 13
1.1.5.1. Kubernetes/OpenShift Clustering 14
1.1.5.2. Java Properties 14
1.1.5.3. Deploying artifacts to the server lib directory 15
1.1.6. Kubernetes 15
1.1.6.1. Liveness and Readiness Probes 15
1.2. DATA GRID SERVER DISTRIBUTION 16
1.2.1. Data Grid Server distribution requirements 16
1.2.2. Downloading Data Grid Server distributions 16
1.2.3. Installing Data Grid Server 16
1.2.4. Starting Data Grid Server 16
1.2.5. Passing Data Grid Server configuration at startup 17
1.3. CREATING DATA GRID USERS 18
1.3.1. Granting roles to users 19
1.3.1.1. Adding users to groups 19
1.3.2. Data Grid user roles and permissions 20
1.4. VERIFYING CLUSTER VIEWS 21
1.5. SHUTTING DOWN DATA GRID SERVER 22
1.5.1. Shutdown and restart of Data Grid clusters 23
Cluster shutdown 23
Server shutdown 23
1.6. DATA GRID SERVER INSTALLATION DIRECTORY STRUCTURE 23
1.6.1. Server root directory 24
server/conf 24
server/data 24
server/lib 25
server/log 25
Ansible collection 25

CHAPTER 2. NETWORK INTERFACES AND SOCKET BINDINGS 26
2.1. NETWORK INTERFACES 26
Internet Protocol (IP) address 26
Loopback address 26
Non-loopback address 27
Any address 28
Link local 28
Site local 29
2.1.1. Match and fallback strategies 29

Match host 30
Match interface 30
Match address 31
Fallback 32
2.2. SOCKET BINDINGS 32
Custom socket binding declarations 33
2.3. CHANGING THE BIND ADDRESS FOR DATA GRID SERVER 35
2.3.1. Listening on all addresses 35
2.4. DATA GRID SERVER PORTS AND PROTOCOLS 36
Single port 37
2.4.1. Configuring network firewalls for Data Grid traffic 37
2.5. SPECIFYING PORT OFFSETS 37

CHAPTER 3. DATA GRID SERVER ENDPOINTS 39
3.1. DATA GRID SERVER PROTOCOLS 39
3.1.1. Hot Rod 39
3.1.2. REST 39
3.1.3. RESP 39
3.1.4. Memcached 40
3.1.5. Comparison of protocols 40
3.1.6. Hot Rod client compatibility with Data Grid Server 40
3.2. CONFIGURING DATA GRID SERVER ENDPOINTS 41
Multiple endpoint configuration 42
3.3. ENDPOINT PROTOCOL CONNECTORS 43
3.3.1. Endpoint request limits 46
3.4. ENDPOINT IP ADDRESS FILTERING RULES 46
IP address filter configuration 47
3.5. INSPECTING AND MODIFYING RULES FOR FILTERING IP ADDRESSES 48

CHAPTER 4. ENDPOINT AUTHENTICATION MECHANISMS 49
4.1. DATA GRID SERVER AUTHENTICATION 49
4.2. CONFIGURING DATA GRID SERVER AUTHENTICATION MECHANISMS 49
Authentication mechanism configuration 50
4.2.1. Disabling authentication 51
4.3. DATA GRID SERVER AUTHENTICATION MECHANISMS 52
4.3.1. SASL authentication mechanisms 54
4.3.2. SASL quality of protection (QoP) 56
4.3.3. SASL policies 56
SASL policy configuration 57
4.3.4. HTTP authentication mechanisms 58

CHAPTER 5. SECURITY REALMS 60
5.1. CREATING SECURITY REALMS 60
Multiple property realms 61
5.2. SETTING UP KERBEROS IDENTITIES 63
Kerberos identity configuration 64
5.3. PROPERTY REALMS 66
Property realm configuration 66
5.3.1. Property realm file structure 67
5.4. LDAP REALMS 68
5.4.1. LDAP connection properties 68
5.4.2. LDAP realm user authentication methods 69
5.4.3. Mapping user entries to their associated groups 69
5.4.4. LDAP realm configuration reference 70

5.4.4.1. LDAP realm principal rewriting 71
5.4.4.2. LDAP principal rewriting configuration reference 72
Case principal transformer 72
Common name principal transformer 73
Regex principal transformer 74
5.4.4.3. LDAP user and group mapping process with Data Grid 75
5.5. TOKEN REALMS 76
Token realm configuration 76
Token realm configuration with TLS 78
5.6. TRUST STORE REALMS 80
Trust store realm configuration 80
5.7. DISTRIBUTED SECURITY REALMS 82
Distributed realm configuration 82
5.8. AGGREGATE SECURITY REALMS 84
Aggregate realm configuration 84
5.8.1. Name rewriters 86
5.8.1.1. Case Principal Transformer 86
5.8.1.2. Common Name Principal Transformer 87
5.8.1.3. Regex Principal Transformer 88
5.9. SECURITY REALM CACHING 88
Realm caching realm configuration 88
5.9.1. Flushing realm caches 89

CHAPTER 6. CONFIGURING TLS/SSL ENCRYPTION 90
6.1. CONFIGURING DATA GRID SERVER KEYSTORES 90
Keystore configuration 91
6.1.1. SSL/TLS Certificate rotation 92
6.1.2. Generating Data Grid Server keystores 92
Generated keystore configuration 92
6.1.3. Configuring TLS versions and cipher suites 94
SSL engine configuration 94
6.2. CONFIGURING DATA GRID SERVER ON A SYSTEM WITH FIPS 140-2 COMPLIANT CRYPTOGRAPHY 96
6.2.1. Configuring the PKCS11 cryptographic provider 96
Keystore configuration 96
6.2.2. Configuring the Bouncy Castle FIPS cryptographic provider 97
Keystore configuration 98
6.3. CONFIGURING CLIENT CERTIFICATE AUTHENTICATION 99
Client certificate authentication configuration 100
6.4. CONFIGURING AUTHORIZATION WITH CLIENT CERTIFICATES 102
Client certificate authorization configuration 103

CHAPTER 7. STORING DATA GRID SERVER CREDENTIALS IN KEYSTORES 105
7.1. SETTING UP CREDENTIAL KEYSTORES 105
7.2. SECURING PASSWORDS FOR CREDENTIAL KEYSTORES 106
7.3. CREDENTIAL KEYSTORE CONFIGURATION 107
Credential keystore with a clear text password 107
Credential keystore with a masked password 108
External command passwords 109
7.4. CREDENTIAL KEYSTORE REFERENCES 109
Datasource connections 109
LDAP connections 111

CHAPTER 8. SECURITY AUTHORIZATION WITH ROLE-BASED ACCESS CONTROL 113
8.1. DATA GRID USER ROLES AND PERMISSIONS 113

8.1.1. Permissions 113
8.1.2. Role and permission mappers 115
8.1.2.1. Mapping users to roles and permissions in Data Grid 116
8.1.3. Configuring role mappers 116
Role mapper configuration 117
8.1.4. Configuring the cluster role and permission mappers 117
8.1.4.1. Creating new roles 118
8.1.4.2. Granting roles to users 118
8.1.4.3. Cluster role mapper name rewriters 119
8.1.4.3.1. Case Principal Transformer 119
8.1.4.3.2. Regex Principal Transformer 120
8.2. CONFIGURING CACHES WITH SECURITY AUTHORIZATION 121
Implicit role configuration 121
Explicit role configuration 122

CHAPTER 9. ENABLING AND CONFIGURING DATA GRID STATISTICS AND JMX MONITORING 123
9.1. ENABLING STATISTICS IN REMOTE CACHES 123
Remote cache statistics 123
9.2. ENABLING HOT ROD CLIENT STATISTICS 123
Hot Rod Java client statistics 124
9.3. CONFIGURING DATA GRID METRICS 124
Metrics configuration 125
Legacy format 125
9.4. REGISTERING JMX MBEANS 126
JMX configuration 127
9.4.1. Enabling JMX remote ports 128
9.4.2. Data Grid MBeans 129
9.4.3. Registering MBeans in custom MBean servers 129
JMX MBean server lookup configuration 130
9.5. EXPORTING METRICS DURING A STATE TRANSFER OPERATION 130
9.6. MONITORING THE STATUS OF CROSS-SITE REPLICATION 131
Monitoring cross-site replication with the REST API 131
Monitoring cross-site replication with the Prometheus metrics 134

CHAPTER 10. ENABLING AND CONFIGURING DATA GRID OPENTELEMETRY TRACING 136
10.1. CONFIGURING DATA GRID TRACING 136
Tracing configuration 136
10.1.1. Further Tracing Options 137
Tracing data format 137
10.2. CONFIGURE TRACING AT CACHE LEVEL 138
Tracing categories 138
Enable/disable tracing for a given cache 138
Enable/disable tracing at runtime 139

CHAPTER 11. ADDING MANAGED DATASOURCES TO DATA GRID SERVER 140
11.1. CONFIGURING MANAGED DATASOURCES 140
Managed datasource configuration 141
11.2. CONFIGURING CACHES WITH JNDI NAMES 142
JNDI name in cache configuration 143
11.3. CONNECTION POOL TUNING PROPERTIES 144

CHAPTER 12. SETTING UP DATA GRID CLUSTER TRANSPORT 146
12.1. DEFAULT JGROUPS STACKS 146
12.2. CLUSTER DISCOVERY PROTOCOLS 147

12.2.1. PING 147
12.2.2. TCPPING 148
12.2.3. MPING 148
12.2.4. TCPGOSSIP 148
12.2.5. JDBC_PING2 149
12.2.5.1. Using a server datasource for JDBC_PING2 discovery 149
12.2.6. DNS_PING 150
12.2.7. Cloud discovery protocols 151
Providing dependencies for cloud discovery protocols 151
12.3. USING THE DEFAULT JGROUPS STACKS 151
12.4. CUSTOMIZING JGROUPS STACKS 152
12.4.1. Inheritance attributes 153
12.5. USING JGROUPS SYSTEM PROPERTIES 154
12.5.1. Cluster transport properties 154
12.5.2. System properties for cloud discovery protocols 155
12.5.2.1. Amazon EC2 156
12.5.2.2. Google Cloud Platform 156
12.5.2.3. Azure 156
12.5.2.4. OpenShift 157
12.6. USING INLINE JGROUPS STACKS 157
12.7. USING EXTERNAL JGROUPS STACKS 158
12.8. ENCRYPTING CLUSTER TRANSPORT 158
12.8.1. Securing cluster transport with TLS identities 158
12.8.2. JGroups encryption protocols 160
12.8.3. Securing cluster transport with asymmetric encryption 161
12.8.4. Securing cluster transport with symmetric encryption 162
12.9. TCP AND UDP PORTS FOR CLUSTER TRAFFIC 163
Cross-site replication 163
12.10. VIRTUAL THREADS SUPPORT 164
12.10.1. Disable Virtual Threads 164

CHAPTER 13. CREATING REMOTE CACHES 165
13.1. DEFAULT CACHE MANAGER 165
Default Cache Manager configuration 165
13.2. CREATING CACHES WITH DATA GRID CONSOLE 166
13.3. CREATING REMOTE CACHES WITH THE DATA GRID CLI 167
13.4. CREATING REMOTE CACHES FROM HOT ROD CLIENTS 168
13.5. CREATING REMOTE CACHES WITH THE REST API 168

CHAPTER 14. RUNNING SCRIPTS AND TASKS ON DATA GRID SERVER 170
14.1. ADDING TASKS TO DATA GRID SERVER DEPLOYMENTS 170
14.1.1. Data Grid Server tasks 170
14.2. ADDING SCRIPTS TO DATA GRID SERVER DEPLOYMENTS 172
14.2.1. Data Grid Server scripts 172
14.2.1.1. Script metadata 173
14.2.1.2. Script bindings 174
14.2.1.3. Script parameters 174
14.2.2. Programmatically Creating Scripts 174
14.3. RUNNING SCRIPTS AND TASKS 174
Programmatic execution 175

CHAPTER 15. CONFIGURING DATA GRID SERVER LOGGING 177
15.1. DATA GRID SERVER LOG FILES 177
15.1.1. Configuring Data Grid Server logs 177

15.1.2. Log levels 177
15.1.3. Data Grid logging categories 178
15.1.4. Log appenders 178
15.1.5. Log pattern formatters 179
15.1.6. Enabling the JSON log handler 179
15.2. ACCESS LOGS 180
15.2.1. Enabling access logs 180
15.2.2. Access log properties 180
15.3. AUDIT LOGS 181
15.3.1. Enabling audit logging 181
15.3.2. Configuring audit logging appenders 182
15.3.3. Using custom audit logging implementations 182

CHAPTER 16. PERFORMING ROLLING UPGRADES FOR DATA GRID SERVER CLUSTERS 184
16.1. SETTING UP TARGET DATA GRID CLUSTERS 184
16.2. SYNCHRONIZING DATA TO TARGET CLUSTERS 185

CHAPTER 17. TROUBLESHOOTING DATA GRID SERVER DEPLOYMENTS 187
17.1. GETTING DIAGNOSTIC REPORTS FROM DATA GRID SERVER 187
17.1.1. Understanding the report 187
17.1.1.1. Environment 187
17.1.1.2. Data Grid server 188
17.2. CHANGING DATA GRID SERVER LOGGING CONFIGURATION AT RUNTIME 188
17.2.1. Access logging 190
Categories 190
17.3. GATHERING RESOURCE STATISTICS FROM THE CLI 190
17.4. JVM SETTINGS FOR DATA GRID 191
Editing the configuration file 192
Setting an environment variable 192
17.4.1. Garbage Collection Logging 192
17.4.2. Heap Dump on OutOfMemoryError 193
17.5. VIRTUAL THREADS SUPPORT 193
17.5.1. Disable Virtual Threads 194
17.6. ACCESSING CLUSTER HEALTH VIA REST 194
17.7. ACCESSING CLUSTER HEALTH VIA JMX 195
17.8. BENCHMARK THE DATA GRID SERVER 195

CHAPTER 18. REFERENCE 196
18.1. DATA GRID SERVER 8.6.0 README 196
18.1.1. Requirements 196
18.1.2. Starting servers 196
18.1.3. Stopping servers 196
18.1.4. Configuration 196
18.1.5. Bind address 197
18.1.6. Bind port 198
18.1.7. Clustering address 198
18.1.8. Cluster stacks 198
18.1.9. Authentication 198
18.1.10. Server home directory 199
18.1.11. Server root directory 199
18.1.12. Logging 200

RED HAT DATA GRID

Data Grid is a high-performance, distributed in-memory data store.

Schemaless data structure

Flexibility to store different objects as key-value pairs.

Grid-based data storage

Designed to distribute and replicate data across clusters.

Elastic scaling

Dynamically adjust the number of nodes to meet demand without service disruption.

Data interoperability

Store, retrieve, and query data in the grid from different endpoints.

DATA GRID DOCUMENTATION

Documentation for Data Grid is available on the Red Hat customer portal.

- Data Grid 8.6 Documentation
- Data Grid 8.6 Component Details
- Supported Configurations for Data Grid 8.6
- Data Grid 8 Feature Support
- Data Grid Deprecated Features and Functionality

DATA GRID DOWNLOADS

Access the Data Grid Software Downloads on the Red Hat customer portal.



NOTE

You must have a Red Hat account to access and download Data Grid software.

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see our CTO Chris Wright's message.

CHAPTER 1. GETTING STARTED WITH DATA GRID SERVER

Install the server, create a user, and start your first Data Grid cluster. Data Grid Server can run either as a containerized image or as a standalone Java process.

1.1. DATA GRID SERVER CONTAINER IMAGE

Data Grid Server as a container image requires a container manager, such as Docker or Podman.

1.1.1. Container registries

The Data Grid Server container image is available at the following registries:

Registry	URL
Docker Hub	https://hub.docker.com/r/infinispan/server
Quay.io	https://quay.io/repository/infinispan/server

1.1.2. Container execution

Start an instance of Infinispan Server by executing the following command:

Docker

```
docker run -p 11222:11222 --name infinispan infinispan/server
```

Podman

```
podman run -p 11222:11222 --net=host --name infinispan infinispan/server
```

> When utilising podman it is necessary for the **--net=host** to be passed when not executing as **sudo**.

By default, the image has authentication enabled on all exposed endpoints. When executing the above command the image automatically generates a username/password pair with the **admin** role, prints the values to stdout and then starts the Infinispan server with the authenticated endpoints exposed on port **11222**. Therefore, it's necessary to utilise the printed credentials when accessing the exposed endpoints via clients.

It is also possible to provide an administrator username/password combination via environment variables:

Docker

```
docker run -p 11222:11222 -e USER="admin" -e PASS="changeme" --name infinispan
infinispan/server
```

Podman

```
podman run -p 11222:11222 -e USER="admin" -e PASS="changeme" --net=host --name infinispan
infinispan/server
```

> We recommend utilising the auto-generated credentials or USER & PASS env variables for initial development only. Providing authentication and authorization configuration via an [Identities Batch file](#identities-batch) allows for much greater control.

1.1.3. Hot Rod Clients

When connecting a Hot Rod client to the image, the following SASL properties must be configured on your client (with the username and password properties changed as required):

```
infinispan.client.hotrod.auth_username=admin
infinispan.client.hotrod.auth_password=changme
infinispan.client.hotrod.sasl_mechanism=DIGEST-MD5
```

1.1.4. Identities Batch

User identities and roles can be defined by providing a cli batch file via the **IDENTITIES_BATCH** env variable. All the cli commands defined in this file are executed before the server is started, therefore it iss only possible to execute offline commands otherwise the container will fail. For example, including **create cache …** in the batch would fail as it requires a connection to an Infinispan server.

Data Grid provides implicit roles for some users.

TIP

Check Infinispan documentation to know more about implicit roles and authorization

Below is an example Identities batch CLI file **identities.batch**, that defines four users and their role:

```
user create "Alan Shearer" -p "striker9" -g admin
user create "observer" -p "secret1"
user create "deployer" -p "secret2"
user create "Rigoberta Baldini" -p "secret3" -g monitor
```

To run the image using a local **identities.batch**, execute:

Docker

```
docker run -v $(pwd):/user-config -e IDENTITIES_BATCH="/user-config/identities.batch" -p
11222:11222 --name infinispan infinispan/server
```

Podman

```
podman run -v $(pwd):/user-config -e IDENTITIES_BATCH="/user-config/identities.batch" -p
11222:11222 --net=host --name infinispan infinispan/server
```

1.1.5. Server Configuration

The Infinispan image passes all container arguments to the created server, therefore it is possible to configure the server in the same manner as a non-containerised deployment.

Below shows how a docker volume can be created and mounted in order to run the Infinispan image with the local configuration file **my-infinispan-config.xml** located in the users current working directory.

Docker

```
docker run -v $(pwd):/user-config -e IDENTITIES_BATCH="/user-config/identities.batch" -p
11222:11222 --name infinispan infinispan/server -c /user-config/my-infinispan-config.xml
```

Podman

```
podman run -v $(pwd):/user-config -e IDENTITIES_BATCH="/user-config/identities.batch" -p
11222:11222 --net=host --name infinispan infinispan/server -c /user-config/my-infinispan-config.xml
```

1.1.5.1. Kubernetes/OpenShift Clustering

When running in a managed environment such as Kubernetes, it is not possible to utilise multicasting for initial node discovery, therefore we must utilise the JGroups DNS_PING protocol to discover cluster members. To enable this, we must provide the **jgroups.dnsPing.query** property and configure the **kubernetes** stack.

To utilise the tcp stack with DNS_PING, execute the following config:

Docker

```
docker run -v $(pwd):/user-config --name infinispan infinispan/server --bind-address=0.0.0.0  -
Dinfinispan.cluster.stack=kubernetes -Djgroups.dns.query="infinispan-dns-
ping.myproject.svc.cluster.local"
```

Podman

```
podman run -v $(pwd):/user-config --name infinispan infinispan/server --bind-address=0.0.0.0  -
Dinfinispan.cluster.stack=kubernetes -Djgroups.dns.query="infinispan-dns-
ping.myproject.svc.cluster.local"
```

1.1.5.2. Java Properties

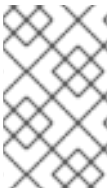
It is possible to provide additional Java properties and JVM options to the server images via the **JAVA_OPTIONS** env variable. For example, to quickly configure CORS without providing a server.yaml file, do the following:

Docker

```
docker run -e JAVA_OPTIONS="-Dinfinispan.cors.enableAll=https://host.domain:port" --name
infinispan infinispan/server
```

Podman

```
podman run -e JAVA_OPTIONS="-Dinfinispan.cors.enableAll=https://host.domain:port" --net=host --
name infinispan infinispan/server
```



NOTE

Using **JAVA_OPTIONS** will append the options to those determined by the server launch script, such as those that configure the JVM memory sizing. You can completely override these options by setting the **JAVA_OPTS** env variable.

1.1.5.3. Deploying artifacts to the server lib directory

Deploy artifacts to the server lib directory using the **SERVER_LIBS** env variable. For example, to add the PostgreSQL JDBC driver to the server:

Docker

```
docker run -e SERVER_LIBS="org.postgresql:postgresql:42.3.1" --name infinispan infinispan/server
```

Podman

```
podman run -e SERVER_LIBS="org.postgresql:postgresql:42.3.1" --name infinispan infinispan/server
```

The **SERVER_LIBS** variable supports multiple, space-separated artifacts represented as URLs or as Maven coordinates. Archive artifacts in **.tar**, **.tar.gz** or **.zip** formats will be extracted. Refer to the CLI **install** command help to learn about all possible arguments and options.

1.1.6. Kubernetes

1.1.6.1. Liveness and Readiness Probes

It is recommended to utilise Infinispan's REST endpoint in order to determine if the server is ready/live. To do this, you can utilise the Kubernetes httpGet probes as follows:

```
livenessProbe:
  httpGet:
    path: /rest/v2/cache-managers/default/health/status
    port: 11222
  failureThreshold: 5
  initialDelaySeconds: 10
  successThreshold: 1
  timeoutSeconds: 10
readinessProbe:
  httpGet:
    path: /rest/v2/cache-managers/default/health/status
    port: 11222
  failureThreshold: 5
  initialDelaySeconds: 10
  successThreshold: 1
  timeoutSeconds: 10
```

1.2. DATA GRID SERVER DISTRIBUTION

1.2.1. Data Grid Server distribution requirements

Data Grid Server requires a Java Virtual Machine. See the Data Grid Supported Configurations for details on supported versions.

1.2.2. Downloading Data Grid Server distributions

The Data Grid Server distribution is an archive of Java libraries (**JAR** files) and configuration files.

Procedure

1. Access the Red Hat customer portal.
2. Download Red Hat Data Grid 8.6 Server from the software downloads section.
3. Run the **md5sum** or **sha256sum** command with the server download archive as the argument, for example:

```
sha256sum jboss-datagrid-${version}-server.zip
```

4. Compare with the **MD5** or **SHA-256** checksum value on the Data Grid **Software Details** page.

Reference

- Data Grid Server README describes the contents of the server distribution.

1.2.3. Installing Data Grid Server

Install the Data Grid Server distribution on a host system.

Prerequisites

- Download a Data Grid Server distribution archive.

Procedure

- Use any appropriate tool to extract the Data Grid Server archive to the host filesystem.

```
unzip redhat-datagrid-8.6.0-server.zip
```

The resulting directory is your **$RHDG_HOME**.

1.2.4. Starting Data Grid Server

Run Data Grid Server instances in a Java Virtual Machine (JVM) on any supported host.

Prerequisites

- Download and install the server distribution.

Procedure

1. Open a terminal in **$RHDG_HOME**.
2. Start Data Grid Server instances with the **server** script.

   Linux

   ```
   bin/server.sh
   ```

   Microsoft Windows

   ```
   bin\server.bat
   ```

Data Grid Server is running successfully when it logs the following messages:

```
ISPN080004: Protocol SINGLE_PORT listening on 127.0.0.1:11222
ISPN080034: Server '...' listening on http://127.0.0.1:11222
ISPN080001: Data Grid Server <version> started in <mm>ms
```

Verification

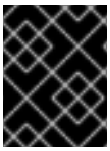
1. Open **127.0.0.1:11222/console/** in any browser.
2. Enter your credentials at the prompt and continue to Data Grid Console.

1.2.5. Passing Data Grid Server configuration at startup

Specify custom configuration when you start Data Grid Server.

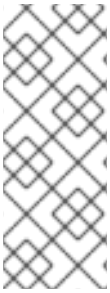
Data Grid Server can parse multiple configuration files that you overlay on startup with the **--server-config** argument. You can use as many configuration overlay files as required, in any order. Configuration overlay files:

- Must be valid Data Grid configuration and contain the root **server** element or field.
- Do not need to be full configuration as long as your combination of overlay files results in a full configuration.



IMPORTANT

Data Grid Server does not detect conflicting configuration between overlay files. Each overlay file overwrites any conflicting configuration in the preceding configuration.



NOTE

If you pass cache configuration to Data Grid Server on startup it does not dynamically create those cache across the cluster. You must manually propagate caches to each node.

Additionally, cache configuration that you pass to Data Grid Server on startup must include the **infinispan** and **cache-container** elements.

Prerequisites

- Download and install the server distribution.
- Add custom server configuration to the **server/conf** directory of your Data Grid Server installation.

Procedure

1. Open a terminal in **$RHDG_HOME**.
2. Specify one or more configuration files with the **--server-config=** or **-c** argument, for example:

```
bin/server.sh -c infinispan.xml -c datasources.yaml -c security-realms.json
```

1.3. CREATING DATA GRID USERS

Add credentials to authenticate with Data Grid Server deployments through Hot Rod and REST endpoints. Before you can access the Data Grid Console or perform cache operations you must create at least one user with the Data Grid command line interface (CLI).

TIP

Data Grid enforces security authorization with role-based access control (RBAC). Create an **admin** user the first time you add credentials to gain full **ADMIN** permissions to your Data Grid deployment.

Prerequisites

- Download and install Data Grid Server.

Procedure

1. Open a terminal in **$RHDG_HOME**.
2. Create an **admin** user, belonging to the **admin** group with the **user create** command.

```
bin/cli.sh user create admin -p changeme -g admin
```

TIP

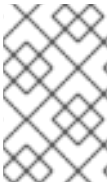
Run **help user** from a CLI session to get complete command details.

Verification

Open **user.properties** and confirm the user exists.

```
cat server/conf/users.properties

admin=scram-sha-1\:BYGcIAwvf6b...
```



NOTE

Adding credentials to a properties realm with the CLI creates the user only on the server instance to which you are connected. You must manually synchronize credentials in a properties realm to each node in the cluster.

1.3.1. Granting roles to users

Assign roles to users and grant them permissions to perform cache operations and interact with Data Grid resources.

TIP

Grant roles to groups instead of users if you want to assign the same role to multiple users and centrally maintain their permissions.

Prerequisites

- Have **ADMIN** permissions for Data Grid.
- Create Data Grid users.

Procedure

1. Create a CLI connection to Data Grid.
2. Assign roles to users with the **user roles grant** command, for example:

```
user roles grant --roles=deployer katie
```

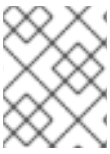
Verification

List roles that you grant to users with the **user roles ls** command.

```
user roles ls katie
["deployer"]
```

1.3.1.1. Adding users to groups

Groups let you change permissions for multiple users. You assign a role to a group and then add users to that group. Users inherit permissions from the group role.



NOTE

You use groups as part of a property realm in the Data Grid Server configuration. Each group is a special type of user that also requires a username and password.

Prerequisites

- Have **ADMIN** permissions for Data Grid.
- Create Data Grid users.

Procedure

1. Create a CLI connection to Data Grid.
2. Use the **user create** command to create a group.
   a. Specify a group name with the **--groups** argument.
   b. Set a username and password for the group.

```
user create --groups=developers developers -p changeme
```

3. List groups.

```
user ls --groups
```

4. Grant a role to the group.

```
user roles grant --roles=application developers
```

5. List roles for the group.

```
user roles ls developers
```

6. Add users to the group one at a time.

```
user groups john --groups=developers
```

Verification

Open **groups.properties** and confirm the group exists.

```
cat server/conf/groups.properties
```

1.3.2. Data Grid user roles and permissions

Data Grid includes several roles that provide users with permissions to access caches and Data Grid resources.

Role	Permissions	Description
admin	ALL	Superuser with all permissions including control of the Cache Manager lifecycle.
deployer	ALL_READ, ALL_WRITE, LISTEN, EXEC, MONITOR, CREATE	Can create and delete Data Grid resources in addition to **application** permissions.

Role	Permissions	Description
application	ALL_READ, ALL_WRITE, LISTEN, EXEC, MONITOR	Has read and write access to Data Grid resources in addition to **observer** permissions. Can also listen to events and execute server tasks and scripts.
observer	ALL_READ, MONITOR	Has read access to Data Grid resources in addition to **monitor** permissions.
monitor	MONITOR	Can view statistics via JMX and the **metrics** endpoint.

Additional resources

- org.infinispan.security.AuthorizationPermission Enum
- Data Grid configuration schema reference

1.4. VERIFYING CLUSTER VIEWS

Data Grid Server instances on the same network automatically discover each other and form clusters.

Complete this procedure to observe cluster discovery with the **MPING** protocol in the default **TCP** stack with locally running Data Grid Server instances. If you want to adjust cluster transport for custom network requirements, see the documentation for setting up Data Grid clusters.



NOTE

This procedure is intended to demonstrate the principle of cluster discovery and is not intended for production environments. Doing things like specifying a port offset on the command line is not a reliable way to configure cluster transport for production.

Prerequisites

Have one instance of Data Grid Server running.

Procedure

1. Open a terminal in **$RHDG_HOME**.
2. Copy the root directory to **server2**.

```
cp -r server server2
```

3. Specify a port offset and the **server2** directory.

```
bin/server.sh -o 100 -s server2
```

Verification

You can view cluster membership in the console at **127.0.0.1:11222/console/cluster-membership**.

Data Grid also logs the following messages when nodes join clusters:

```
INFO  [org.infinispan.CLUSTER] (jgroups-11,<server_hostname>)
ISPN000094: Received new cluster view for channel cluster:
[<server_hostname>|3] (2) [<server_hostname>, <server2_hostname>]

INFO  [org.infinispan.CLUSTER] (jgroups-11,<server_hostname>)
ISPN100000: Node <server2_hostname> joined the cluster
```

1.5. SHUTTING DOWN DATA GRID SERVER

Stop individually running servers or bring down clusters gracefully.

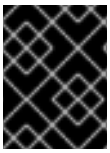
Procedure

1. Create a CLI connection to Data Grid.
2. Shut down Data Grid Server in one of the following ways:
   - Stop all nodes in a cluster with the **shutdown cluster** command, for example:

     ```
     shutdown cluster
     ```

     This command saves cluster state to the **data** folder for each node in the cluster. If you use a cache store, the **shutdown cluster** command also persists all data in the cache.
   - Stop individual server instances with the **shutdown server** command and the server hostname, for example:

     ```
     shutdown server <my_server01>
     ```



IMPORTANT

The **shutdown server** command does not wait for rebalancing operations to complete, which can lead to data loss if you specify multiple hostnames at the same time.

TIP

Run **help shutdown** for more details about using the command.

Verification

Data Grid logs the following messages when you shut down servers:

```
ISPN080002: Data Grid Server stopping
ISPN000080: Disconnecting JGroups channel cluster
ISPN000390: Persisted state, version=<$version> timestamp=YYYY-MM-DDTHH:MM:SS
ISPN080003: Data Grid Server stopped
```

1.5.1. Shutdown and restart of Data Grid clusters

Prevent data loss and ensure consistency of your cluster by properly shutting down and restarting nodes.

Cluster shutdown
Data Grid recommends using the **shutdown cluster** command to stop all nodes in a cluster while saving cluster state and persisting all data in the cache. You can use the **shutdown cluster** command also for clusters with a single node.

When you bring Data Grid clusters back online, all nodes and caches in the cluster will be unavailable until all nodes rejoin. To prevent inconsistencies or data loss, Data Grid restricts access to the data stored in the cluster and modifications of the cluster state until the cluster is fully operational again. Additionally, Data Grid disables cluster rebalancing and prevents local cache stores purging on startup.

During the cluster recovery process, the coordinator node logs messages for each new node joining, indicating which nodes are available and which are still missing. Other nodes in the Data Grid cluster have the view from the time they join. You can monitor availability of caches using the Data Grid Console or REST API.

However, in cases where waiting for all nodes is not necessary nor desired, it is possible to set a cache available with the current topology. This approach is possible through the CLI, see below, or the REST API.



IMPORTANT

Manually installing a topology can lead to data loss, only perform this operation if the initial topology cannot be recreated.

Server shutdown
After using the **shutdown server** command to bring nodes down, the first node to come back online will be available immediately without waiting for other members. The remaining nodes join the cluster immediately, triggering state transfer but loading the local persistence first, which might lead to stale entries. Local cache stores configured to purge on startup will be emptied when the server starts. Local cache stores marked as **purge=false** will be available after a server restarts but might contain stale entries.

If you shutdown clustered nodes with the **shutdown server** command, you must restart each server in reverse order to avoid potential issues related to data loss and stale entries in the cache.
For example, if you shutdown **server1** and then shutdown **server2**, you should first start **server2** and then start **server1**. However, restarting clustered nodes in reverse order does not completely prevent data loss and stale entries.

1.6. DATA GRID SERVER INSTALLATION DIRECTORY STRUCTURE

Data Grid Server uses the following folders on the host filesystem under **$RHDG_HOME**:

```
├── bin
├── boot
├── docs
├── lib
├── server
└── static
```

TIP

See the Data Grid Server README for descriptions of the each folder in your **$RHDG_HOME** directory as well as system properties you can use to customize the filesystem.

1.6.1. Server root directory

Apart from resources in the **bin** and **docs** folders, the only folder under **$RHDG_HOME** that you should interact with is the server root directory, which is named **server** by default.

You can create multiple nodes under the same **$RHDG_HOME** directory or in different directories, but each Data Grid Server instance must have its own server root directory. For example, a cluster of 5 nodes could have the following server root directories on the filesystem:

```
├── server
├── server1
├── server2
├── server3
└── server4
```

Each server root directory should contain the following folders:

```
├── server
│   ├── conf
│   ├── data
│   ├── lib
│   └── log
```

server/conf
Holds **infinispan.xml** configuration files for a Data Grid Server instance.

Data Grid separates configuration into two layers:

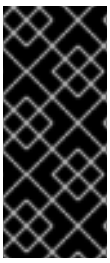
Dynamic

Create mutable cache configurations for data scalability.
Data Grid Server permanently saves the caches you create at runtime along with the cluster state that is distributed across nodes. Each joining node receives a complete cluster state that Data Grid Server synchronizes across all nodes whenever changes occur.

Static

Add configuration to **infinispan.xml** for underlying server mechanisms such as cluster transport, security, and shared datasources.

server/data
Provides internal storage that Data Grid Server uses to maintain cluster state.



IMPORTANT

Never directly delete or modify content in **server/data**.

Modifying files such as **caches.xml** while the server is running can cause corruption. Deleting content can result in an incorrect state, which means clusters cannot restart after shutdown.

server/lib
Contains extension **JAR** files for custom filters, custom event listeners, JDBC drivers, custom **ServerTask** implementations, and so on.

server/log
Holds Data Grid Server log files.

Additional resources

- Data Grid Server README
- What is stored in the <server>/data directory used by a RHDG server (Red Hat Knowledgebase)

Ansible collection

Automate installation of Data Grid clusters with our Ansible collection that optionally includes Keycloak caches and cross-site replication configuration. The Ansible collection also lets you inject Data Grid caches into the static configuration for each server instance during installation.

The Ansible collection for Data Grid is available from the Red Hat **Automation Hub**.

CHAPTER 2. NETWORK INTERFACES AND SOCKET BINDINGS

Expose Data Grid Server through a network interface by binding it to an IP address. You can then configure endpoints to use the interface so Data Grid Server can handle requests from remote client applications.

2.1. NETWORK INTERFACES

Data Grid Server multiplexes endpoints to a single TCP/IP port and automatically detects protocols of inbound client requests. You can configure how Data Grid Server binds to network interfaces to listen for client requests.

Internet Protocol (IP) address

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects a specific IPv4 address, which can be public, private, or loopback. This is the default network interface for Data Grid Server. -->
  <interfaces>
    <interface name="public">
      <inet-address value="${infinispan.bind.address:127.0.0.1}"/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "inet-address": {
        "value": "127.0.0.1"
      }
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      inetAddress:
        value: "127.0.0.1"
```

Loopback address

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects an IP address in an IPv4 or IPv6 loopback address block. -->
  <interfaces>
    <interface name="public">
      <loopback/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "loopback": null
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      loopback: ~
```

Non-loopback address

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects an IP address in an IPv4 or IPv6 non-loopback address block. -->
  <interfaces>
    <interface name="public">
      <non-loopback/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "non_loopback": null
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      nonLoopback: ~
```

Any address

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <!-- Uses the `INADDR_ANY` wildcard address which means Data Grid Server listens for inbound client requests on all interfaces. -->
  <interfaces>
    <interface name="public">
      <any-address/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "any_address": null
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      anyAddress: ~
```

Link local

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects a link-local IP address in an IPv4 or IPv6 address block. -->
  <interfaces>
    <interface name="public">
      <link-local/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "link_local": null
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      linkLocal: ~
```

Site local

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects a site-local (private) IP address in an IPv4 or IPv6 address block. -->
  <interfaces>
    <interface name="public">
      <site-local/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "site_local": null
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      siteLocal: ~
```

2.1.1. Match and fallback strategies

Data Grid Server can enumerate all network interfaces on the host system and bind to an interface, host, or IP address that matches a value, which can include regular expressions for additional flexibility.

Match host

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects an IP address that is assigned to a matching host name. -->
  <interfaces>
    <interface name="public">
      <match-host value="my_host_name"/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "match-host": {
        "value": "my_host_name"
      }
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      matchHost:
        value: "my_host_name"
```

Match interface

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <!--Selects an IP address assigned to a matching network interface. -->
  <interfaces>
    <interface name="public">
      <match-interface value="eth0"/>
    </interface>
  </interfaces>
</server>
```

JSON

-

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "match-interface": {
        "value": "eth0"
      }
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      matchInterface:
        value: "eth0"
```

Match address

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <!-- Selects an IP address that matches a regular expression. -->
  <interfaces>
    <interface name="public">
      <match-address value="132\..*"/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "match-address": {
        "value": "132\\..*"
      }
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      matchAddress:
        value: "127\\..*"
```

–

Fallback

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <!-- Includes multiple strategies that Data Grid Server tries in the declared order until it finds a match. -->
  <interfaces>
    <interface name="public">
      <match-host value="my_host_name"/>
      <match-address value="132\..*"/>
      <any-address/>
    </interface>
  </interfaces>
</server>
```

JSON

```
{
  "server": {
    "interfaces": [{
      "name": "public",
      "match-host": {
        "value": "my_host_name"
      },
      "match-address": {
        "value": "132\\..*"
      },
      "any_address": null
    }]
  }
}
```

YAML

```
server:
  interfaces:
    - name: "public"
      matchHost:
        value: "my_host_name"
      matchAddress:
        value: "132\\..*"
      anyAddress: ~
```

2.2. SOCKET BINDINGS

Socket bindings map endpoint connectors to network interfaces and ports. By default, Data Grid Server includes a socket binding configuration that listens on the localhost interface, **127.0.0.1**, at port **11222** for the REST and Hot Rod endpoints. If you enable the Memcached endpoint, the default socket bindings configure Data Grid Server to bind to port **11221**.

Default socket bindings

```
<server xmlns="urn:infinispan:server:16.0">
  <socket-bindings default-interface="public"
                   port-offset="${infinispan.socket.binding.port-offset:0}">
    <socket-binding name="default"
                    port="${infinispan.bind.port:11222}"/>
    <socket-binding name="memcached"
                    port="11221"/>
  </socket-bindings>
</server>
```

Configuration element or attribute	Description
socket-bindings	Root element that contains all network interfaces and ports to which Data Grid Server endpoints can bind and listen for client connections.
default-interface	Declare the network interface that Data Grid Server listens on by default.
port-offset	Specifies the offset that Data Grid Server applies to port declarations for socket bindings.
socket-binding	Configures Data Grid Server to bind to a port on a network interface.

Custom socket binding declarations

The following example configuration adds an **interface** declaration named "private" and a **socket-binding** declaration that binds Data Grid Server to the private IP address:

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <interfaces>
    <interface name="public">
      <inet-address value="${infinispan.bind.address:127.0.0.1}"/>
    </interface>
    <interface name="private">
      <inet-address value="10.1.2.3"/>
    </interface>
  </interfaces>

  <socket-bindings default-interface="public"
                   port-offset="${infinispan.socket.binding.port-offset:0}">
    <socket-binding name="private_binding"
                    interface="private"
                    port="49152"/>
  </socket-bindings>

  <endpoints socket-binding="private_binding"
             security-realm="default"/>
```

```
</server>
```

JSON

```
{
 "server": {
  "interfaces": [{
   "name": "private",
   "inet-address": {
    "value": "10.1.2.3"
   }
  }, {
   "name": "public",
   "inet-address": {
    "value": "127.0.0.1"
   }
  }],
  "socket-bindings": {
   "port-offset": "0",
   "default-interface": "public",
   "socket-binding": [{
    "name": "private_binding",
    "port": "1234",
    "interface": "private"
   }]
  },
  "endpoints": {
   "endpoint": {
    "socket-binding": "private_binding",
    "security-realm": "default"
   }
  }
 }
}
```

YAML

```
server:
  interfaces:
    - name: "private"
      inetAddress:
        value: "10.1.2.3"
    - name: "public"
      inetAddress:
        value: "127.0.0.1"
  socketBindings:
    portOffset: "0"
    defaultInterface: "public"
    socketBinding:
      - name: "private_binding"
        port: "49152"
        interface: "private"
  endpoints:
```

```
endpoint:
  socketBinding: "private_binding"
  securityRealm: "default"
```

2.3. CHANGING THE BIND ADDRESS FOR DATA GRID SERVER

Data Grid Server binds to a network IP address to listen for inbound client connections on the Hot Rod and REST endpoints. You can specify that IP address directly in your Data Grid Server configuration or when starting server instances.

Prerequisites

- Have at least one Data Grid Server installation.

Procedure

Specify the IP address to which Data Grid Server binds in one of the following ways:

- Open your Data Grid Server configuration and set the value for the **inet-address** element, for example:

```
<server xmlns="urn:infinispan:server:16.0">
  <interfaces>
    <interface name="custom">
      <inet-address value="${infinispan.bind.address:192.0.2.0}"/>
    </interface>
  </interfaces>
</server>
```

- Use the **-b** option or the **infinispan.bind.address** system property.

  Linux

```
bin/server.sh -b 192.0.2.0
```

  Windows

```
bin\server.bat -b 192.0.2.0
```

2.3.1. Listening on all addresses

If you specify the **0.0.0.0** meta-address, or **INADDR_ANY**, as the bind address in your Data Grid Server configuration, it listens for incoming client connections on all available network interfaces.

Client intelligence

Configuring Data Grid to listen on all addresses affects how it provides Hot Rod clients with cluster topology. If there are multiple interfaces to which Data Grid Server binds, then it sends a list of IP addresses for each interface.

For example, a cluster where each server node binds to:

- **10.0.0.0/8** subnet

- **192.168.0.0/16** subnet
- **127.0.0.1** loopback

Hot Rod clients receive IP addresses for server nodes that belong to the interface through which the clients connect. If a client connects to **192.168.0.0**, for example, it does not receive any cluster topology details for nodes that listen on **10.0.0.0**.

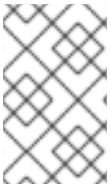
Netmask override

Kubernetes, and some other environments, divide the IP address space into subnets and use those different subnets as a single network. For example, **10.129.2.100/23** and **10.129.4.100/23** are in different subnets but belong to the **10.0.0.0/8** network.

For this reason, Data Grid Server overrides netmasks that the host system provides with netmasks that follow IANA conventions for private and reserved networks:

- IPv4: **10.0.0.0/8**, **100.64.0.0/10**, 192.168.0.0/16`, **172.16.0.0/12**, **169.254.0.0/16** and **240.0.0.0/4**
- IPv6: **fc00::/7** and **fe80::/10**

See **RFC 1112**, **RFC 1918**, **RFC 3927**, **RFC 6598** for IPv4 or **RFC 4193**, **RFC 3513** for IPv6.



NOTE

You can optionally configure the Hot Rod connector to use the netmask that the host system provides for interfaces with the **network-prefix-override** attribute in your Data Grid Server configuration.

Additional resources

- Data Grid Server schema reference
- RFC 1112
- RFC 1918
- RFC 3513
- RFC 3927
- RFC 4193
- RFC 6598

2.4. DATA GRID SERVER PORTS AND PROTOCOLS

Data Grid Server provides network endpoints that allow client access with different protocols.

Port	Protocol	Description
11222	TCP	Hot Rod and REST

Port	Protocol	Description
11221	TCP	Memcached (disabled by default)

Single port

Data Grid Server exposes multiple protocols through a single TCP port, **11222**. Handling multiple protocols with a single port simplifies configuration and reduces management complexity when deploying Data Grid clusters. Using a single port also enhances security by minimizing the attack surface on the network.

Data Grid Server handles HTTP/1.1, HTTP/2, and Hot Rod protocol requests from clients via the single port in different ways.

HTTP/1.1 upgrade headers

Client requests can include the **HTTP/1.1 upgrade** header field to initiate HTTP/1.1 connections with Data Grid Server. Client applications can then send the **Upgrade: protocol** header field, where **protocol** is a server endpoint.

Application-Layer Protocol Negotiation (ALPN)/Transport Layer Security (TLS)

Client requests include Server Name Indication (SNI) mappings for Data Grid Server endpoints to negotiate protocols over a TLS connection.

Automatic Hot Rod detection

Client requests that include Hot Rod headers automatically route to Hot Rod endpoints.

2.4.1. Configuring network firewalls for Data Grid traffic

Adjust firewall rules to allow traffic between Data Grid Server and client applications.

Procedure

On Red Hat Enterprise Linux (RHEL) workstations, for example, you can allow traffic to port **11222** with firewalld as follows:

```
# firewall-cmd --add-port=11222/tcp --permanent
success
# firewall-cmd --list-ports | grep 11222
11222/tcp
```

To configure firewall rules that apply across a network, you can use the nftables utility.

Reference

- Using and configuring firewalld
- Getting started with nftables

2.5. SPECIFYING PORT OFFSETS

Configure port offsets for multiple Data Grid Server instances on the same host. The default port offset is **0**.

Procedure

Use the **-o** switch with the Data Grid CLI or the **infinispan.socket.binding.port-offset** system property to set port offsets.

For example, start a server instance with an offset of **100** as follows. With the default configuration, this results in the Data Grid server listening on port **11322**.

Linux

```
bin/server.sh -o 100
```

Windows

```
bin\server.bat -o 100
```

CHAPTER 3. DATA GRID SERVER ENDPOINTS

Data Grid Server endpoints provide client access to the cache manager over different protocols. A server endpoint can multiplex multiple protocols on the same socket binding by detecting the incoming traffic type and routing it to the appropriate internal connector handler.

3.1. DATA GRID SERVER PROTOCOLS

3.1.1. Hot Rod

Hot Rod is a binary TCP client-server protocol designed to provide faster data access and improved performance in comparison to text-based protocols.

Data Grid provides Hot Rod client libraries in Java, C++, C#, Node.js and other programming languages.

Topology caches

Data Grid uses topology caches to provide clients with cluster views. Topology caches contain entries that map internal JGroups transport addresses to exposed Hot Rod endpoints.

When clients send requests, Data Grid servers compare the topology ID in request headers with the topology ID from the cache. Data Grid servers send new topology views if clients have older topology IDs.

Cluster topology views allow Hot Rod clients to immediately detect when nodes join and leave, which enables dynamic load balancing and failover.

In distributed cache modes, the consistent hashing algorithm also makes it possible to route Hot Rod client requests directly to primary owners.

Additional resources

- Hot Rod client documentation

3.1.2. REST

Data Grid exposes a RESTful interface that allows HTTP clients to access data, monitor and maintain clusters, and perform administrative operations.

You can use standard HTTP load balancers to provide clients with load balancing and failover capabilities. However, HTTP load balancers maintain static cluster views and require manual updates when cluster topology changes occur.

Additional resources

- REST API documentation

3.1.3. RESP

Data Grid provides an implementation of the RESP3 protocol used by Redis, Valkey, Elasticache and others.

The RESP connector supports a large subset of the Redis commands.

Additional resources

- RESP endpoint documentation

3.1.4. Memcached

Data Grid provides an implementation of the Memcached text and binary protocols for remote client access.

The Data Grid Memcached endpoint supports clustering with replicated and distributed cache modes.

There are some Memcached client implementations, such as the Cache::Memcached Perl client, that can offer load balancing and failover detection capabilities with static lists of Data Grid server addresses that require manual updates when cluster topology changes occur.

Additional resources

- Memcached endpoint documentation

3.1.5. Comparison of protocols

	Hot Rod	HTTP / REST	Memcached	RESP
Topology-aware	Y	N	N	N
Hash-aware	Y	N	N	N
Encryption	Y	Y	Y	Y
Authentication	Y	Y	Y	Y
Conditional ops	Y	Y	Y	N
Bulk ops	Y	N	Y	Y
Transactions	Y	N	N	N
Listeners	Y	N	N	Y
Query	Y	Y	N	N
Execution	Y	N	N	N
Cross-site failover	Y	N	N	N

3.1.6. Hot Rod client compatibility with Data Grid Server

Before diving into advanced configuration, here is a quick note on version compatibility. Data Grid Server allows you to connect Hot Rod clients with different versions. For instance, during a migration or upgrade to your Data Grid cluster, the Hot Rod client version might be lower than the Data Grid Server

version.

TIP

Data Grid recommends using the latest Hot Rod client version to benefit from the most recent capabilities and security enhancements.

Data Grid 8 and later

Hot Rod protocol version 3.x automatically negotiates the highest version possible for clients with Data Grid Server.

DATA GRID 7.3 AND EARLIER

The client no longer supports Hot Rod protocol versions older than 3.0. As such, you must use an older client or the **hotrod-client-legacy** module.

Clients that use a Hot Rod protocol version that is higher than the Data Grid Server version must set the **infinispan.client.hotrod.protocol_version** property.

Additional resources

- Hot Rod protocol reference
- Connecting Hot Rod clients to servers with different versions (Red Hat Knowledgebase)

3.2. CONFIGURING DATA GRID SERVER ENDPOINTS

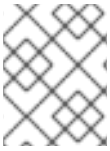
Control how the different server endpoints bind to sockets and use security realm configuration.

Prerequisites

- Add socket bindings and security realms to your Data Grid Server configuration.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Wrap multiple **endpoint** configurations with the **endpoints** element.
3. Specify the socket binding that the endpoint uses with the **socket-binding** attribute.
4. Specify the security realm that the endpoint uses with the **security-realm** attribute.
5. Disable administrator access with the **admin="false"** attribute, if required.
   With this configuration users cannot access Data Grid Console or the Command Line Interface (CLI) from the endpoint.
6. Save the changes to your configuration.



NOTE

Data Grid Server must be configured with at least one endpoint which has a REST connector with administrative capabilities.

Multiple endpoint configuration

The following Data Grid Server configuration creates endpoints on separate socket bindings with dedicated security realms and disabling administrative access on the **public** endpoint:

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <endpoints>
    <endpoint socket-binding="public"
              security-realm="application-realm"
              admin="false">
    </endpoint>
    <endpoint socket-binding="private"
              security-realm="management-realm">
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "server": {
    "endpoints": [
      {
        "socket-binding": "public",
        "security-realm": "application-realm",
        "admin": "false"
      },
      {
      "socket-binding": "private",
      "security-realm": "management-realm"
    }]
  }
}
```

YAML

```
server:
  endpoints:
    - socketBinding: public
      securityRealm: application-realm
      admin: false
    - socketBinding: private
      securityRealm: management-realm
```

Additional resources

- Network interfaces and socket bindings

3.3. ENDPOINT PROTOCOL CONNECTORS

Endpoints multiplex different protocol connectors through a single port. Protocol connectors employ detectors to inspect incoming traffic and direct client connections to the appropriate handler. An endpoint will enable all protocol connectors that support the configured security realm, unless connectors are declared explicitly. The default endpoint configuration, using the default property security realm, implicitly enables all the protocol connectors (Hot Rod, REST, RESP, Memcached).

XML

```
<endpoints socket-binding="default" security-realm="default"/>
```

JSON

```
{
  "endpoints": [
    {
      "socket-binding": "default",
      "security-realm": "default"
    }
  ]
}
```

YAML

```
server:
  endpoints:
    - socketBinding: default
      securityRealm: default
```

Enable specific protocol connectors using explicit declaration.

Configuration element or attribute	Description
endpoints	Wraps endpoint connector configuration.
endpoint	Declares a Data Grid Server endpoint that configures Hot Rod and REST connectors to use a socket binding and security realm.
hotrod-connector	Includes the Hot Rod endpoint in the **endpoint** configuration.
rest-connector	Includes the REST endpoint in the **endpoint** configuration.

Configuration element or attribute	Description
resp-connector	Includes the RESP endpoint in the **endpoint** configuration.
memcached-connector	Includes the Memcached endpoint in the **endpoint** configuration.

Expose a single endpoint with only the REST and Hot Rod connectors on the **default** socket binding with the **default** security realm:

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <endpoints>
    <endpoint socket-binding="default" security-realm="default">
      <rest-connector/>
      <hotrod-connector/>
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "endpoints": [
    {
      "endpoint": {
        "socket-binding": "default",
        "security-realm": "default",
        "connectors": {
          "rest": {
            "rest-connector": {}
          },
          "hotrod": {
            "hotrod-connector": {}
          }
        }
      }
    }
  ]
}
```

YAML

```
endpoints:
  - socketBinding: default
    securityRealm: default
    connectors:
      rest:
```

```
      restConnector: ~
    hotrod:
      hotrodConnector: ~
```

Expose a REST connector on the **default** socket binding with the **default** security realm and a RESP connector on the **resp** socket binding with the **none** security realm:

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <endpoints>
    <endpoint socket-binding="default" security-realm="default">
      <rest-connector/>
      <resp-connector socket-binding="resp" security-realm="none"/>
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "endpoints": [
    {
      "endpoint": {
        "socket-binding": "default",
        "security-realm": "default",
        "connectors": {
          "rest": {
            "rest-connector": {}
          },
          "resp": {
            "resp-connector": {
              "socket-binding": "resp",
              "security-realm": "none"
            }
          }
        }
      }
    }
  ]
}
```

YAML

```
endpoints:
  - socketBinding: default
    securityRealm: default
    connectors:
      rest:
        restConnector: ~
      resp:
```

```
    respConnector:
      socketBinding: resp
      securityRealm: none
```

3.3.1. Endpoint request limits

Endpoints, by default, impose a **10MB** limit on request sizes. If the client sends a request larger than the limit, the request may be met with an exception or the connection may be forcibly closed. The limit is configured via the **max-content-length** attributed. This is expressed in bytes and allows for a string value with a size suffix (e.g. 128KB, 10MB, 1.5GB). Note that this is the size of the entire request and is not limited to a single key or value.

Hot Rod, RESP & Memcached endpoints

```
<endpoints>
  <endpoint socket-binding="default" security-realm="default">
    <hotrod-connector name="hotrod" max-content-length="5MB">
      ...
    </hotrod-connector>
    <resp-connector name="resp" max-content-length="5MB">
      ...
    </hotrod-connector>
    <memcached-connector name="memcached" max-content-length="5MB">
      ...
    </hotrod-connector>
  </endpoint>
</endpoints>
```

REST endpoint

```
<endpoints>
  <endpoint socket-binding="default" security-realm="default">
    <rest-connector name="rest" max-content-length="5MB">
      ...
    </rest-connector>
  </endpoint>
</endpoints>
```

Additional resources

- Data Grid schema reference

3.4. ENDPOINT IP ADDRESS FILTERING RULES

Data Grid Server endpoints can use filtering rules that control whether clients can connect based on their IP addresses. Data Grid Server applies filtering rules in order until it finds a match for the client IP address.

A CIDR block is a compact representation of an IP address and its associated network mask. CIDR notation specifies an IP address, a slash ('/') character, and a decimal number. The decimal number is the count of leading 1 bits in the network mask. The number can also be thought of as the width, in bits, of the network prefix. The IP address in CIDR notation is always represented according to the standards for IPv4 or IPv6.

The address can denote a specific interface address, including a host identifier, such as **10.0.0.1/8**, or it can be the beginning address of an entire network interface range using a host identifier of 0, as in **10.0.0.0/8** or **10/8**.

For example:

- **192.168.100.14/24** represents the IPv4 address **192.168.100.14** and its associated network prefix **192.168.100.0**, or equivalently, its subnet mask **255.255.255.0**, which has 24 leading 1-bits.
- the IPv4 block **192.168.100.0/22** represents the 1024 IPv4 addresses from **192.168.100.0** to **192.168.103.255**.
- the IPv6 block **2001:db8::/48** represents the block of IPv6 addresses from **2001:db8:0:0:0:0:0:0** to **2001:db8:0:ffff:ffff:ffff:ffff:ffff**.
- **::1/128** represents the IPv6 loopback address. Its prefix length is 128 which is the number of bits in the address.

IP address filter configuration

In the following configuration, Data Grid Server accepts connections only from addresses in the **192.168.0.0/16** and **10.0.0.0/8** CIDR blocks. Data Grid Server rejects all other connections.

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <endpoints>
    <endpoint socket-binding="default" security-realm="default">
      <ip-filter>
        <accept from="192.168.0.0/16"/>
        <accept from="10.0.0.0/8"/>
        <reject from="/0"/>
      </ip-filter>
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "server": {
    "endpoints": {
      "endpoint": {
        "socket-binding": "default",
        "security-realm": "default",
        "ip-filter": {
          "accept-from": ["192.168.0.0/16", "10.0.0.0/8"],
          "reject-from": "/0"
        }
      }
    }
  }
}
```

YAML

YAML

```
server:
  endpoints:
    endpoint:
      socketBinding: "default"
      securityRealm: "default"
      ipFilter:
        acceptFrom: ["192.168.0.0/16","10.0.0.0/8"]
        rejectFrom: "/0"
```

3.5. INSPECTING AND MODIFYING RULES FOR FILTERING IP ADDRESSES

Configure IP address filtering rules on Data Grid Server endpoints to accept or reject connections based on client address.

Prerequisites

- Install Data Grid Command Line Interface (CLI).

Procedure

1. Create a CLI connection to Data Grid Server.
2. Inspect and modify the IP filter rules **server connector ipfilter** command as required.
   a. List all IP filtering rules active on a connector across the cluster:

```
server connector ipfilter ls endpoint-default
```

   b. Set IP filtering rules across the cluster.



NOTE

This command replaces any existing rules.

```
server connector ipfilter set endpoint-default --
rules=ACCEPT/192.168.0.0/16,REJECT/10.0.0.0/8`
```

   c. Remove all IP filtering rules on a connector across the cluster.

```
server connector ipfilter clear endpoint-default
```

CHAPTER 4. ENDPOINT AUTHENTICATION MECHANISMS

Data Grid Server can use custom authentication mechanisms for Hot Rod and binary Memcached (SASL) and REST (HTTP) endpoints. The RESP endpoint does not offer alternative authentication mechanisms, because of its specification.

4.1. DATA GRID SERVER AUTHENTICATION

Authentication restricts user access to endpoints as well as the Data Grid Console and Command Line Interface (CLI).

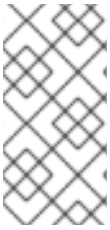
Data Grid Server includes a "default" security realm that enforces user authentication. Default authentication uses a property realm with user credentials stored in the **server/conf/users.properties** file. Data Grid Server also enables security authorization by default so you must assign users with permissions stored in the **server/conf/groups.properties** file.

TIP

Use the **user create** command with the Command Line Interface (CLI) to add users and assign permissions. Run **user create --help** for examples and more information.

4.2. CONFIGURING DATA GRID SERVER AUTHENTICATION MECHANISMS

You can explicitly configure Hot Rod and REST endpoints to use specific authentication mechanisms. Configuring authentication mechanisms is required only if you need to explicitly override the default mechanisms for a security realm.



NOTE

Each **endpoint** section in your configuration must include **hotrod-connector** and **rest-connector** elements or fields. For example, if you explicitly declare a **hotrod-connector** you must also declare a **rest-connector** even if it does not configure an authentication mechanism.

Prerequisites

- Add security realms to your Data Grid Server configuration as required.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Add an **endpoint** element or field and specify the security realm that it uses with the **security-realm** attribute.
3. Add a **hotrod-connector** element or field to configure the Hot Rod endpoint.
    a. Add an **authentication** element or field.
    b. Specify SASL authentication mechanisms for the Hot Rod endpoint to use with the **sasl mechanisms** attribute.

c. If applicable, specify SASL quality of protection settings with the **qop** attribute.

d. Specify the Data Grid Server identity with the **server-name** attribute if necessary.

4. Add a **rest-connector** element or field to configure the REST endpoint.

a. Add an **authentication** element or field.

b. Specify HTTP authentication mechanisms for the REST endpoint to use with the **mechanisms** attribute.

5. Save the changes to your configuration.

Authentication mechanism configuration

The following configuration specifies SASL mechanisms for the Hot Rod endpoint to use for authentication:

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <endpoints>
    <endpoint socket-binding="default"
              security-realm="my-realm">
      <hotrod-connector>
        <authentication>
          <sasl mechanisms="SCRAM-SHA-512 SCRAM-SHA-384 SCRAM-SHA-256
                            SCRAM-SHA-1 DIGEST-SHA-512 DIGEST-SHA-384
                            DIGEST-SHA-256 DIGEST-SHA DIGEST-MD5 PLAIN"
                server-name="infinispan"
                qop="auth"/>
        </authentication>
      </hotrod-connector>
      <rest-connector>
        <authentication mechanisms="DIGEST BASIC"/>
      </rest-connector>
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "server": {
    "endpoints": {
      "endpoint": {
        "socket-binding": "default",
        "security-realm": "my-realm",
        "hotrod-connector": {
          "authentication": {
            "security-realm": "default",
            "sasl": {
              "server-name": "infinispan",
              "mechanisms": ["SCRAM-SHA-512", "SCRAM-SHA-384", "SCRAM-SHA-256", "SCRAM-
SHA-1", "DIGEST-SHA-512", "DIGEST-SHA-384", "DIGEST-SHA-256", "DIGEST-SHA", "DIGEST-
MD5", "PLAIN"],
```

```
            "qop": ["auth"]
          }
        }
      },
      "rest-connector": {
        "authentication": {
          "mechanisms": ["DIGEST", "BASIC"],
          "security-realm": "default"
        }
      }
    }
  }
}
```

YAML

```
server:
  endpoints:
    endpoint:
      socketBinding: "default"
      securityRealm: "my-realm"
      hotrodConnector:
        authentication:
          securityRealm: "default"
          sasl:
            serverName: "infinispan"
            mechanisms:
              - "SCRAM-SHA-512"
              - "SCRAM-SHA-384"
              - "SCRAM-SHA-256"
              - "SCRAM-SHA-1"
              - "DIGEST-SHA-512"
              - "DIGEST-SHA-384"
              - "DIGEST-SHA-256"
              - "DIGEST-SHA"
              - "DIGEST-MD5"
              - "PLAIN"
            qop:
              - "auth"
      restConnector:
        authentication:
          mechanisms:
            - "DIGEST"
            - "BASIC"
          securityRealm: "default"
```

4.2.1. Disabling authentication

In local development environments or on isolated networks you can configure Data Grid to allow unauthenticated client requests. When you disable user authentication you should also disable authorization in your Data Grid security configuration.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Remove the **security-realm** attribute from the **endpoints** element or field.
3. Remove any **authorization** elements from the **security** configuration for the **cache-container** and each cache configuration.
4. Save the changes to your configuration.

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <endpoints socket-binding="default"/>
</server>
```

JSON

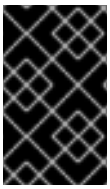
```
{
  "server": {
    "endpoints": {
      "endpoint": {
        "socket-binding": "default"
      }
    }
  }
}
```

YAML

```
server:
  endpoints:
    endpoint:
      socketBinding: "default"
```

4.3. DATA GRID SERVER AUTHENTICATION MECHANISMS

Data Grid Server automatically configures endpoints with authentication mechanisms that match your security realm configuration. For example, if you add a Kerberos security realm then Data Grid Server enables the **GSSAPI** and **GS2-KRB5** authentication mechanisms for the Hot Rod endpoint.



IMPORTANT

Currently, you cannot use the Lightweight Directory Access Protocol (LDAP) protocol with the **DIGEST** or **SCRAM** authentication mechanisms, because these mechanisms require access to specific hashed passwords.

Hot Rod endpoints

Data Grid Server enables the following SASL authentication mechanisms for Hot Rod endpoints when your configuration includes the corresponding security realm:

Security realm	SASL authentication mechanism
Property realms and LDAP realms	**SCRAM**, **DIGEST**
Token realms	**OAUTHBEARER**
Trust realms	**EXTERNAL**
Kerberos identities	**GSSAPI**, **GS2-KRB5**
SSL/TLS identities	**PLAIN**

REST endpoints

Data Grid Server enables the following HTTP authentication mechanisms for REST endpoints when your configuration includes the corresponding security realm:

Security realm	HTTP authentication mechanism
Property realms and LDAP realms	**DIGEST**
Token realms	**BEARER_TOKEN**
Trust realms	**CLIENT_CERT**
Kerberos identities	**SPNEGO**
SSL/TLS identities	**BASIC**

Memcached endpoints

Data Grid Server enables the following SASL authentication mechanisms for Memcached binary protocol endpoints when your configuration includes the corresponding security realm:

Security realm	SASL authentication mechanism
Property realms and LDAP realms	**SCRAM**, **DIGEST**
Token realms	**OAUTHBEARER**
Trust realms	**EXTERNAL**
Kerberos identities	**GSSAPI, GS2-KRB5**
SSL/TLS identities	**PLAIN**

Data Grid Server enables authentication on Memcached text protocol endpoints only with security realms which support password-based authentication:

Security realm	Memcached text authentication
Property realms and LDAP realms	Yes
Token realms	No
Trust realms	No
Kerberos identities	No
SSL/TLS identities	No

RESP endpoints

Data Grid Server enables authentication on RESP endpoints only with security realms which support password-based authentication:

Security realm	RESP authentication
Property realms and LDAP realms	Yes
Token realms	No
Trust realms	No
Kerberos identities	No
SSL/TLS identities	No

4.3.1. SASL authentication mechanisms

Data Grid Server supports the following SASL authentications mechanisms with Hot Rod and Memcached binary protocol endpoints:

Authentication mechanism	Description	Security realm type	Related details
PLAIN	Uses credentials in plain-text format. You should use **PLAIN** authentication with encrypted connections only.	Property realms and LDAP realms	Similar to the **BASIC** HTTP mechanism.

Authentication mechanism	Description	Security realm type	Related details
DIGEST-*	Uses hashing algorithms and nonce values. Hot Rod connectors support **DIGEST-MD5**, **DIGEST-SHA**, **DIGEST-SHA-256**, **DIGEST-SHA-384**, and **DIGEST-SHA-512** hashing algorithms, in order of strength.	Property realms and LDAP realms	Similar to the **Digest** HTTP mechanism.
SCRAM-*	Uses *salt* values in addition to hashing algorithms and nonce values. Hot Rod connectors support **SCRAM-SHA**, **SCRAM-SHA-256**, **SCRAM-SHA-384**, and **SCRAM-SHA-512** hashing algorithms, in order of strength.	Property realms and LDAP realms	Similar to the **Digest** HTTP mechanism.
GSSAPI	Uses Kerberos tickets and requires a Kerberos Domain Controller. You must add a corresponding **kerberos** server identity in the realm configuration. In most cases, you also specify an **ldap-realm** to provide user membership information.	Kerberos realms	Similar to the **SPNEGO** HTTP mechanism.

Authentication mechanism	Description	Security realm type	Related details
GS2-KRB5	Uses Kerberos tickets and requires a Kerberos Domain Controller. You must add a corresponding **kerberos** server identity in the realm configuration. In most cases, you also specify an **ldap-realm** to provide user membership information.	Kerberos realms	Similar to the **SPNEGO** HTTP mechanism.
EXTERNAL	Uses client certificates.	Trust store realms	Similar to the **CLIENT_CERT** HTTP mechanism.
OAUTHBEARER	Uses OAuth tokens and requires a **token-realm** configuration.	Token realms	Similar to the **BEARER_TOKEN** HTTP mechanism.

4.3.2. SASL quality of protection (QoP)

If SASL mechanisms support integrity and privacy protection (QoP) settings, you can add them to your Hot Rod and Memcached endpoint configuration with the **qop** attribute.

QoP setting	Description
auth	Authentication only.
auth-int	Authentication with integrity protection.
auth-conf	Authentication with integrity and privacy protection.

4.3.3. SASL policies

SASL policies provide fine-grain control over Hot Rod and Memcached authentication mechanisms.

TIP

Data Grid cache authorization restricts access to caches based on roles and permissions. Configure cache authorization and then set **<no-anonymous value=false />** to allow anonymous login and delegate access logic to cache authorization.

Policy	Description	Default value
forward-secrecy	Use only SASL mechanisms that support forward secrecy between sessions. This means that breaking into one session does not automatically provide information for breaking into future sessions.	false
pass-credentials	Use only SASL mechanisms that require client credentials.	false
no-plain-text	Do not use SASL mechanisms that are susceptible to simple plain passive attacks.	false
no-active	Do not use SASL mechanisms that are susceptible to active, non-dictionary, attacks.	false
no-dictionary	Do not use SASL mechanisms that are susceptible to passive dictionary attacks.	false
no-anonymous	Do not use SASL mechanisms that accept anonymous logins.	true

SASL policy configuration

In the following configuration the Hot Rod endpoint uses the **GSSAPI** mechanism for authentication because it is the only mechanism that complies with all SASL policies:

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <endpoints>
    <endpoint socket-binding="default"
              security-realm="default">
      <hotrod-connector>
        <authentication>
          <sasl mechanisms="PLAIN DIGEST-MD5 GSSAPI EXTERNAL"
                server-name="infinispan"
                qop="auth"
                policy="no-active no-plain-text"/>
        </authentication>
      </hotrod-connector>
      <rest-connector/>
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "server": {
    "endpoints" : {
      "endpoint" : {
        "socket-binding" : "default",
        "security-realm" : "default",
        "hotrod-connector" : {
          "authentication" : {
            "sasl" : {
              "server-name" : "infinispan",
              "mechanisms" : [ "PLAIN","DIGEST-MD5","GSSAPI","EXTERNAL" ],
              "qop" : [ "auth" ],
              "policy" : [ "no-active","no-plain-text" ]
            }
          }
        },
        "rest-connector" : ""
      }
    }
  }
}
```

YAML

```
server:
  endpoints:
    endpoint:
      socketBinding: "default"
      securityRealm: "default"
      hotrodConnector:
        authentication:
          sasl:
            serverName: "infinispan"
            mechanisms:
              - "PLAIN"
              - "DIGEST-MD5"
              - "GSSAPI"
              - "EXTERNAL"
            qop:
              - "auth"
            policy:
              - "no-active"
              - "no-plain-text"
      restConnector: ~
```

4.3.4. HTTP authentication mechanisms

Data Grid Server supports the following HTTP authentication mechanisms with REST endpoints:

Authentication mechanism	Description	Security realm type	Related details
BASIC	Uses credentials in plain-text format. You should use **BASIC** authentication with encrypted connections only.	Property realms and LDAP realms	Corresponds to the **Basic** HTTP authentication scheme and is similar to the **PLAIN** SASL mechanism.
DIGEST	Uses hashing algorithms and nonce values. REST connectors support **SHA-512**, **SHA-256** and **MD5** hashing algorithms.	Property realms and LDAP realms	Corresponds to the **Digest** HTTP authentication scheme and is similar to **DIGEST-*** SASL mechanisms.
SPNEGO	Uses Kerberos tickets and requires a Kerberos Domain Controller. You must add a corresponding **kerberos** server identity in the realm configuration. In most cases, you also specify an **ldap-realm** to provide user membership information.	Kerberos realms	Corresponds to the **Negotiate** HTTP authentication scheme and is similar to the **GSSAPI** and **GS2-KRB5** SASL mechanisms.
BEARER_TOKEN	Uses OAuth tokens and requires a **token-realm** configuration.	Token realms	Corresponds to the **Bearer** HTTP authentication scheme and is similar to **OAUTHBEARER** SASL mechanism.
CLIENT_CERT	Uses client certificates.	Trust store realms	Similar to the **EXTERNAL** SASL mechanism.

CHAPTER 5. SECURITY REALMS

Security realms integrate Data Grid Server deployments with the network protocols and infrastructure in your environment that control access and verify user identities.

5.1. CREATING SECURITY REALMS

Add security realms to Data Grid Server configuration to control access to deployments. You can add one or more security realm to your configuration.



NOTE

When you add security realms to your configuration, Data Grid Server automatically enables the matching authentication mechanisms for the Hot Rod and REST endpoints.

Prerequisites

- Add socket bindings to your Data Grid Server configuration as required.
- Create keystores, or have a PEM file, to configure the security realm with TLS/SSL encryption. Data Grid Server can also generate keystores at startup.
- Provision the resources or services that the security realm configuration relies on. For example, if you add a token realm, you need to provision OAuth services.

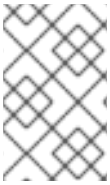
This procedure demonstrates how to configure multiple property realms. Before you begin, you need to create properties files that add users and assign permissions with the Command Line Interface (CLI). Use the **user create** commands as follows:

```
user create <username> -p <changeme> -g <role> \
    --users-file=application-users.properties \
    --groups-file=application-groups.properties

user create <username> -p <changeme> -g <role> \
    --users-file=management-users.properties \
    --groups-file=management-groups.properties
```

TIP

Run **user create --help** for examples and more information.



NOTE

Adding credentials to a properties realm with the CLI creates the user only on the server instance to which you are connected. You must manually synchronize credentials in a properties realm to each node in the cluster.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Use the **security-realms** element in the **security** configuration to contain create multiple security realms.

3. Add a security realm with the **security-realm** element and give it a unique name with the **name** attribute.
   To follow the example, create one security realm named **application-realm** and another named **management-realm**.

4. Provide the TLS/SSL identify for Data Grid Server with the **server-identities** element and configure a keystore as required.

5. Specify the type of security realm by adding one the following elements or fields:

   - **properties-realm**
   - **ldap-realm**
   - **token-realm**
   - **truststore-realm**

6. Specify properties for the type of security realm you are configuring as appropriate.
   To follow the example, specify the ***.properties** files you created with the CLI using the **path** attribute on the **user-properties** and **group-properties** elements or fields.

7. If you add multiple different types of security realm to your configuration, include the **distributed-realm** element or field so that Data Grid Server uses the realms in combination with each other.

8. Configure Data Grid Server endpoints to use the security realm with the with the **security-realm** attribute.

9. Save the changes to your configuration.

Multiple property realms

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="application-realm">
        <properties-realm groups-attribute="Roles">
          <user-properties path="application-users.properties"/>
          <group-properties path="application-groups.properties"/>
        </properties-realm>
      </security-realm>
      <security-realm name="management-realm">
        <properties-realm groups-attribute="Roles">
          <user-properties path="management-users.properties"/>
          <group-properties path="management-groups.properties"/>
        </properties-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "management-realm",
        "properties-realm": {
          "groups-attribute": "Roles",
          "user-properties": {
            "digest-realm-name": "management-realm",
            "path": "management-users.properties"
          },
          "group-properties": {
            "path": "management-groups.properties"
          }
        }
      }, {
        "name": "application-realm",
        "properties-realm": {
          "groups-attribute": "Roles",
          "user-properties": {
            "digest-realm-name": "application-realm",
            "path": "application-users.properties"
          },
          "group-properties": {
            "path": "application-groups.properties"
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "management-realm"
        propertiesRealm:
          groupsAttribute: "Roles"
          userProperties:
            digestRealmName: "management-realm"
            path: "management-users.properties"
          groupProperties:
            path: "management-groups.properties"
      - name: "application-realm"
        propertiesRealm:
          groupsAttribute: "Roles"
          userProperties:
            digestRealmName: "application-realm"
            path: "application-users.properties"
          groupProperties:
            path: "application-groups.properties"
```

5.2. SETTING UP KERBEROS IDENTITIES

Add Kerberos identities to a security realm in your Data Grid Server configuration to use *keytab* files that contain service principal names and encrypted keys, derived from Kerberos passwords.

Prerequisites

- Have Kerberos service account principals.



NOTE

keytab files can contain both user and service account principals. However, Data Grid Server uses service account principals only which means it can provide identity to clients and allow clients to authenticate with Kerberos servers.

In most cases, you create unique principals for the Hot Rod and REST endpoints. For example, if you have a "datagrid" server in the "INFINISPAN.ORG" domain you should create the following service principals:

- **hotrod/datagrid@INFINISPAN.ORG** identifies the Hot Rod service.
- **HTTP/datagrid@INFINISPAN.ORG** identifies the REST service.

Procedure

1. Create keytab files for the Hot Rod and REST services.

   Linux

   ```
   ktutil
   ktutil:  addent -password -p datagrid@INFINISPAN.ORG -k 1 -e aes256-cts
   Password for datagrid@INFINISPAN.ORG: [enter your password]
   ktutil:  wkt http.keytab
   ktutil:  quit
   ```

   Microsoft Windows

   ```
   ktpass -princ HTTP/datagrid@INFINISPAN.ORG -pass * -mapuser
   INFINISPAN\USER_NAME
   ktab -k http.keytab -a HTTP/datagrid@INFINISPAN.ORG
   ```

2. Copy the keytab files to the **server/conf** directory of your Data Grid Server installation.
3. Open your Data Grid Server configuration for editing.
4. Add a **server-identities** definition to the Data Grid server security realm.
5. Specify the location of keytab files that provide service principals to Hot Rod and REST connectors.
6. Name the Kerberos service principals.
7. Save the changes to your configuration.

Kerberos identity configuration

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="kerberos-realm">
        <server-identities>
          <!-- Specifies a keytab file that provides a Kerberos identity. -->
          <!-- Names the Kerberos service principal for the Hot Rod endpoint. -->
          <!-- The required="true" attribute specifies that the keytab file must be present when the server starts. -->
          <kerberos keytab-path="hotrod.keytab"
                    principal="hotrod/datagrid@INFINISPAN.ORG"
                    required="true"/>
          <!-- Specifies a keytab file and names the Kerberos service principal for the REST endpoint. -->
          <kerberos keytab-path="http.keytab"
                    principal="HTTP/localhost@INFINISPAN.ORG"
                    required="true"/>
        </server-identities>
      </security-realm>
    </security-realms>
  </security>
  <endpoints>
    <endpoint socket-binding="default"
              security-realm="kerberos-realm">
      <hotrod-connector>
        <authentication>
          <sasl server-name="datagrid"
                server-principal="hotrod/datagrid@INFINISPAN.ORG"/>
        </authentication>
      </hotrod-connector>
      <rest-connector>
        <authentication server-principal="HTTP/localhost@INFINISPAN.ORG"/>
      </rest-connector>
    </endpoint>
  </endpoints>
</server>
```

JSON

```json
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "kerberos-realm",
        "server-identities": [{
          "kerberos": {
            "principal": "hotrod/datagrid@INFINISPAN.ORG",
            "keytab-path": "hotrod.keytab",
            "required": true
          },
          "kerberos": {
```

```
            "principal": "HTTP/localhost@INFINISPAN.ORG",
            "keytab-path": "http.keytab",
            "required": true
          }
        }]
      }]
    },
    "endpoints": {
      "endpoint": {
        "socket-binding": "default",
        "security-realm": "kerberos-realm",
        "hotrod-connector": {
          "authentication": {
            "security-realm": "kerberos-realm",
            "sasl": {
              "server-name": "datagrid",
              "server-principal": "hotrod/datagrid@INFINISPAN.ORG"
            }
          }
        },
        "rest-connector": {
          "authentication": {
            "server-principal": "HTTP/localhost@INFINISPAN.ORG"
          }
        }
      }
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "kerberos-realm"
        serverIdentities:
          - kerberos:
              principal: "hotrod/datagrid@INFINISPAN.ORG"
              keytabPath: "hotrod.keytab"
              required: "true"
          - kerberos:
              principal: "HTTP/localhost@INFINISPAN.ORG"
              keytabPath: "http.keytab"
              required: "true"
  endpoints:
    endpoint:
      socketBinding: "default"
      securityRealm: "kerberos-realm"
      hotrodConnector:
        authentication:
          sasl:
            serverName: "datagrid"
            serverPrincipal: "hotrod/datagrid@INFINISPAN.ORG"
      restConnector:
```

```
      authentication:
        securityRealm: "kerberos-realm"
        serverPrincipal" : "HTTP/localhost@INFINISPAN.ORG"
```

5.3. PROPERTY REALMS

Property realms use property files to define users and groups.

- **users.properties** contains Data Grid user credentials. Passwords can be pre-digested with the **DIGEST-MD5** and **DIGEST** authentication mechanisms.
- **groups.properties** associates users with roles and permissions.

users.properties

```
myuser=a_password
user2=another_password
```

groups.properties

```
myuser=supervisor,reader,writer
user2=supervisor
```

Property realm configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="default">
        <!-- groups-attribute configures the "groups.properties" file to contain security authorization roles.
-->
        <properties-realm groups-attribute="Roles">
          <user-properties path="users.properties"
                           relative-to="infinispan.server.config.path"
                           plain-text="true"/>
          <group-properties path="groups.properties"
                            relative-to="infinispan.server.config.path"/>
        </properties-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "default",
```

```
        "properties-realm": {
          "groups-attribute": "Roles",
          "user-properties": {
            "digest-realm-name": "default",
            "path": "users.properties",
            "relative-to": "infinispan.server.config.path",
            "plain-text": true
          },
          "group-properties": {
            "path": "groups.properties",
            "relative-to": "infinispan.server.config.path"
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "default"
        propertiesRealm:
          # groupsAttribute configures the "groups.properties" file
          # to contain security authorization roles.
          groupsAttribute: "Roles"
          userProperties:
            digestRealmName: "default"
            path: "users.properties"
            relative-to: 'infinispan.server.config.path'
            plainText: "true"
          groupProperties:
            path: "groups.properties"
            relative-to: 'infinispan.server.config.path'
```

5.3.1. Property realm file structure

User properties files are structured as follows:

users.properties structure

```
#$REALM_NAME=default$
#$ALGORITHM=encrypted$
#Wed Jul 31 08:32:08 CEST 2024
admin=algorithm-1\:hash-1;algorithm-2\:hash-2;...
```

The first three lines are special comments that define the name of the realm (**$REALM_NAME**), whether the passwords are stored in **clear** or **encrypted** format (**$ALGORITHM**) and the timestamp of the last update.

User credentials are stored in traditional key/value format: the key corresponds to the username and the value corresponds to the password. Encrypted passwords are represented as semi-colon-separated algorithm/hash pairs, with the hash encoded in Base64.

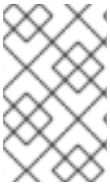


WARNING

Credentials are encrypted using the realm name. Changing a realm's name requires re-encrypting all the passwords. Use the Data Grid CLI to enter the correct security realm name to the file.

5.4. LDAP REALMS

LDAP realms connect to LDAP servers, such as OpenLDAP, Red Hat Directory Server, Apache Directory Server, or Microsoft Active Directory, to authenticate users and obtain membership information.



NOTE

LDAP servers can have different entry layouts, depending on the type of server and deployment. It is beyond the scope of this document to provide examples for all possible configurations.

5.4.1. LDAP connection properties

Specify the LDAP connection properties in the LDAP realm configuration.

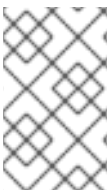
The following properties are required:

url	Specifies the URL of the LDAP server. The URL should be in format **ldap://hostname:port** or **ldaps://hostname:port** for secure connections using TLS.
principal	Specifies a distinguished name (DN) of a valid user in the LDAp server. The DN uniquely identifies the user within the LDAP directory structure.
credential	Corresponds to the password associated with the principal mentioned above.



IMPORTANT

The principal for LDAP connections must have necessary privileges to perform LDAP queries and access specific attributes.



NOTE

If the **url** uses LDAPS, it may be necessary to define a server identity containing an appropriate truststore and reference it in the **ldap-realm.client-ssl-context** configuration attribute.

TIP

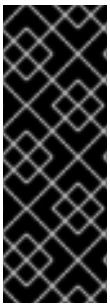
Enabling **connection-pooling** significantly improves the performance of authentication to LDAP servers. The connection pooling mechanism is provided by the JDK. For more information see Connection Pooling Configuration and Java Tutorials: Pooling.

5.4.2. LDAP realm user authentication methods

Configure the user authentication method in the LDAP realm.

The LDAP realm can authenticate users in two ways:

Hashed password comparison	by comparing the hashed password stored in a user's password attribute (usually **userPassword**)
Direct verification	by authenticating against the LDAP server using the supplied credentials Direct verification is the only approach that works with Active Directory, because access to the **password** attribute is forbidden.



IMPORTANT

You cannot use endpoint authentication mechanisms that performs hashing with the **direct-verification** attribute, since this method requires having the password in clear text. As a result you must use the **BASIC** authentication mechanism with the REST endpoint and **PLAIN** with the Hot Rod endpoint to integrate with Active Directory Server. A more secure alternative is to use Kerberos, which allows the **SPNEGO**, **GSSAPI**, and **GS2-KRB5** authentication mechanisms.

The LDAP realm searches the directory to find the entry which corresponds to the authenticated user. The **rdn-identifier** attribute specifies an LDAP attribute that finds the user entry based on a provided identifier, which is typically a username; for example, the **uid** or **sAMAccountName** attribute. Add **search-recursive="true"** to the configuration to search the directory recursively. By default, the search for the user entry uses the **(rdn_identifier={0})** filter. You can specify a different filter using the **filter-name** attribute.

5.4.3. Mapping user entries to their associated groups

In the LDAP realm configuration, specify the **attribute-mapping** element to retrieve and associate all groups that a user is a member of.

The membership information is stored typically in two ways:

- Under group entries that usually have class **groupOfNames** or **groupOfUniqueNames** in the **member** attribute. This is the default behavior in most LDAP installations, except for Active Directory. In this case, you can use an attribute filter. This filter searches for entries that match the supplied filter, which locates groups with a **member** attribute equal to the user's DN. The filter then extracts the group entry's CN as specified by **from**, and adds it to the user's **Roles**.
- In the user entry in the **memberOf** attribute. This is typically the case for Active Directory. In this case you should use an attribute reference such as the following:

<attribute-reference reference="memberOf" from="cn" to="Roles" />

This reference gets all **memberOf** attributes from the user's entry, extracts the CN as specified by **from**, and adds them to the user's groups (**Roles** is the internal name used to map the groups).

5.4.4. LDAP realm configuration reference

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="ldap-realm">
        <!-- Specifies connection properties. -->
        <ldap-realm url="ldap://my-ldap-server:10389"
                    principal="uid=admin,ou=People,dc=infinispan,dc=org"
                    credential="strongPassword"
                    connection-timeout="3s"
                    read-timeout="30s"
                    connection-pooling="true"
                    referral-mode="ignore"
                    page-size="30"
                    direct-verification="true">
          <!-- Defines how principals are mapped to LDAP entries. -->
          <identity-mapping rdn-identifier="uid"
                            search-dn="ou=People,dc=infinispan,dc=org"
                            search-recursive="false">
            <!-- Retrieves all the groups of which the user is a member. -->
            <attribute-mapping>
              <attribute from="cn" to="Roles"
                         filter="(&amp;(objectClass=groupOfNames)(member={1}))"
                         filter-dn="ou=Roles,dc=infinispan,dc=org"/>
            </attribute-mapping>
          </identity-mapping>
        </ldap-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```json
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "ldap-realm",
        "ldap-realm": {
          "url": "ldap://my-ldap-server:10389",
          "principal": "uid=admin,ou=People,dc=infinispan,dc=org",
          "credential": "strongPassword",
          "connection-timeout": "3000",
          "read-timeout": "30000",
```

```
          "connection-pooling": "true",
          "referral-mode": "ignore",
          "page-size": "30",
          "direct-verification": "true",
          "identity-mapping": {
            "rdn-identifier": "uid",
            "search-dn": "ou=People,dc=infinispan,dc=org",
            "search-recursive": "false",
            "attribute-mapping": [{
              "from": "cn",
              "to": "Roles",
              "filter": "(&(objectClass=groupOfNames)(member={1}))",
              "filter-dn": "ou=Roles,dc=infinispan,dc=org"
            }]
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: ldap-realm
        ldapRealm:
          url: 'ldap://my-ldap-server:10389'
          principal: 'uid=admin,ou=People,dc=infinispan,dc=org'
          credential: strongPassword
          connectionTimeout: '3000'
          readTimeout: '30000'
          connectionPooling: true
          referralMode: ignore
          pageSize: '30'
          directVerification: true
          identityMapping:
            rdnIdentifier: uid
            searchDn: 'ou=People,dc=infinispan,dc=org'
            searchRecursive: false
            attributeMapping:
              - filter: '(&(objectClass=groupOfNames)(member={1}))'
                filterDn: 'ou=Roles,dc=infinispan,dc=org'
                from: cn
                to: Roles
```

5.4.4.1. LDAP realm principal rewriting

Principals obtained by SASL authentication mechanisms such as **GSSAPI**, **GS2-KRB5** and **Negotiate** usually include the domain name, for example **myuser@INFINISPAN.ORG**. Before using these principals in LDAP queries, it is necessary to transform them to ensure their compatibility. This process is called rewriting.

Data Grid includes the following transformers:

case-principal-transformer	rewrites the principal to either all uppercase or all lowercase. For example **MyUser** would be rewritten as **MYUSER** in uppercase mode and **myuser** in lowercase mode.
common-name-principal-transformer	rewrites principals in the LDAP Distinguished Name format (as defined by RFC 4514). It extracts the first attribute of type **CN** (commonName). For example, **DN=CN=myuser,OU=myorg,DC=mydomain** would be rewritten as**myuser**.
regex-principal-transformer	rewrites principals using a regular expression with capturing groups, allowing, for example, for extractions of any substring.

5.4.4.2. LDAP principal rewriting configuration reference

Case principal transformer

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="ldap-realm">
        <ldap-realm url="ldap://${org.infinispan.test.host.address}:10389"
                    principal="uid=admin,ou=People,dc=infinispan,dc=org"
                    credential="strongPassword">
          <name-rewriter>
            <!-- Defines a rewriter that transforms usernames to lowercase -->
            <case-principal-transformer uppercase="false"/>
          </name-rewriter>
          <!-- further configuration omitted -->
        </ldap-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```json
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "ldap-realm",
        "ldap-realm": {
          "principal": "uid=admin,ou=People,dc=infinispan,dc=org",
          "url": "ldap://${org.infinispan.test.host.address}:10389",
          "credential": "strongPassword",
          "name-rewriter": {
            "case-principal-transformer": {
```

```
            "uppercase": false
          }
        }
      }
    }]
  }
 }
}
```

YAML

```
server:
 security:
  securityRealms:
   - name: "ldap-realm"
     ldapRealm:
       principal: "uid=admin,ou=People,dc=infinispan,dc=org"
       url: "ldap://${org.infinispan.test.host.address}:10389"
       credential: "strongPassword"
       nameRewriter:
         casePrincipalTransformer:
           uppercase: false
       # further configuration omitted
```

Common name principal transformer

XML

```
<server xmlns="urn:infinispan:server:16.0">
 <security>
  <security-realms>
   <security-realm name="ldap-realm">
    <ldap-realm url="ldap://${org.infinispan.test.host.address}:10389"
                principal="uid=admin,ou=People,dc=infinispan,dc=org"
                credential="strongPassword">
     <name-rewriter>
      <!-- Defines a rewriter that obtains the first CN from a DN -->
      <common-name-principal-transformer />
     </name-rewriter>
     <!-- further configuration omitted -->
    </ldap-realm>
   </security-realm>
  </security-realms>
 </security>
</server>
```

JSON

```
{
 "server": {
  "security": {
   "security-realms": [{
    "name": "ldap-realm",
```

```
        "ldap-realm": {
          "principal": "uid=admin,ou=People,dc=infinispan,dc=org",
          "url": "ldap://${org.infinispan.test.host.address}:10389",
          "credential": "strongPassword",
          "name-rewriter": {
            "common-name-principal-transformer": {}
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "ldap-realm"
        ldapRealm:
          principal: "uid=admin,ou=People,dc=infinispan,dc=org"
          url: "ldap://${org.infinispan.test.host.address}:10389"
          credential: "strongPassword"
          nameRewriter:
            commonNamePrincipalTransformer: ~
          # further configuration omitted
```

Regex principal transformer

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="ldap-realm">
        <ldap-realm url="ldap://${org.infinispan.test.host.address}:10389"
                    principal="uid=admin,ou=People,dc=infinispan,dc=org"
                    credential="strongPassword">
          <name-rewriter>
            <!-- Defines a rewriter that extracts the username from the principal using a regular
expression. -->
            <regex-principal-transformer pattern="(.*)@INFINISPAN\.ORG"
                                         replacement="$1"/>
          </name-rewriter>
          <!-- further configuration omitted -->
        </ldap-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "ldap-realm",
        "ldap-realm": {
          "principal": "uid=admin,ou=People,dc=infinispan,dc=org",
          "url": "ldap://${org.infinispan.test.host.address}:10389",
          "credential": "strongPassword",
          "name-rewriter": {
            "regex-principal-transformer": {
              "pattern": "(.*)@INFINISPAN\\.ORG",
              "replacement": "$1"
            }
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "ldap-realm"
        ldapRealm:
          principal: "uid=admin,ou=People,dc=infinispan,dc=org"
          url: "ldap://${org.infinispan.test.host.address}:10389"
          credential: "strongPassword"
          nameRewriter:
            regexPrincipalTransformer:
              pattern: (.*)@INFINISPAN\.ORG
              replacement: "$1"
          # further configuration omitted
```

5.4.4.3. LDAP user and group mapping process with Data Grid

This example illustrates the process of loading and internally mapping LDAP users and groups to Data Grid subjects. The following is a LDIF (LDAP Data Interchange Format) file, which describes multiple LDAP entries:

LDIF

```
# Users

dn: uid=root,ou=People,dc=infinispan,dc=org
objectclass: top
objectclass: uidObject
objectclass: person
uid: root
cn: root
sn: root
```

```
userPassword: strongPassword

# Groups

dn: cn=admin,ou=Roles,dc=infinispan,dc=org
objectClass: top
objectClass: groupOfNames
cn: admin
description: the Infinispan admin group
member: uid=root,ou=People,dc=infinispan,dc=org

dn: cn=monitor,ou=Roles,dc=infinispan,dc=org
objectClass: top
objectClass: groupOfNames
cn: monitor
description: the Infinispan monitor group
member: uid=root,ou=People,dc=infinispan,dc=org
```

The **root** user is a member of the **admin** and **monitor** groups.

When a request to authenticate the user **root** with the password **strongPassword** is made on one of the endpoints, the following operations are performed:

- The username is optionally rewritten using the chosen principal transformer.
- The realm searches within the **ou=People,dc=infinispan,dc=org** tree for an entry whose **uid** attribute is equal to **root** and finds the entry with DN **uid=root,ou=People,dc=infinispan,dc=org**, which becomes the user principal.
- The realm searches within the **u=Roles,dc=infinispan,dc=org** tree for entries of **objectClass=groupOfNames** that include **uid=root,ou=People,dc=infinispan,dc=org** in the **member** attribute. In this case it finds two entries: **cn=admin,ou=Roles,dc=infinispan,dc=org** and **cn=monitor,ou=Roles,dc=infinispan,dc=org**. From these entries, it extracts the **cn** attributes which become the group principals.

The resulting subject will therefore look like:

- NamePrincipal: **uid=root,ou=People,dc=infinispan,dc=org**
- RolePrincipal: **admin**
- RolePrincipal: **monitor**

At this point, the global authorization mappers are applied on the above subject to convert the principals into roles. The roles are then expanded into a set of permissions, which are validated against the requested cache and operation.

5.5. TOKEN REALMS

Token realms use external services to validate tokens and require providers that are compatible with RFC-7662 (OAuth2 Token Introspection), such as Red Hat SSO.

Token realm configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="token-realm">
        <!-- Specifies the URL of the authentication server. -->
        <token-realm name="token"
                     auth-server-url="https://oauth-server/auth/">
          <!-- Specifies the URL of the token introspection endpoint. -->
          <oauth2-introspection introspection-url="https://oauth-
server/auth/realms/infinispan/protocol/openid-connect/token/introspect"
                                client-id="infinispan-server"
                                client-secret="1fdca4ec-c416-47e0-867a-3d471af7050f"/>
        </token-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "token-realm",
        "token-realm": {
          "auth-server-url": "https://oauth-server/auth/",
          "oauth2-introspection": {
            "client-id": "infinispan-server",
            "client-secret": "1fdca4ec-c416-47e0-867a-3d471af7050f",
            "introspection-url": "https://oauth-server/auth/realms/infinispan/protocol/openid-
connect/token/introspect"
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: token-realm
        tokenRealm:
          authServerUrl: 'https://oauth-server/auth/'
          oauth2Introspection:
            clientId: infinispan-server
            clientSecret: '1fdca4ec-c416-47e0-867a-3d471af7050f'
            introspectionUrl: 'https://oauth-server/auth/realms/infinispan/protocol/openid-
connect/token/introspect'
```

If the **introspection-url** uses HTTPS, it may be necessary to define a server identity containing an appropriate truststore and reference it in the **oauth2-introspection.client-ssl-context** configuration attribute.

Token realm configuration with TLS

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="keycloak">
        <server-identities>
          <ssl>
            <truststore path="keycloak-trust.pfx" password="secret"/>
          </ssl>
        </server-identities>
      </security-realm>
      <security-realm name="default">
        <!-- The TLS identity of this server -->
        <server-identities>
          <ssl>
            <keystore path="server.pfx" password="secret" alias="server"/>1
          </ssl>
        </server-identities>
        <!-- Specifies the URL of the authentication server. -->
        <token-realm name="token"
                     auth-server-url="https://oauth-server/auth/">
          <!-- Specifies the URL of the token introspection endpoint and uses the `keycloak` SSL
context defined above to trust the server -->
          <oauth2-introspection introspection-url="https://oauth-
server/auth/realms/infinispan/protocol/openid-connect/token/introspect"
                                client-id="infinispan-server"
                                client-secret="1fdca4ec-c416-47e0-867a-3d471af7050f"
                                client-ssl-context="keycloak"
          />
        </token-realm>
      </security-realm>
    </security-realms>
  </security>
  <endpoints socket-binding="default" security-realm="default"/>
</server>
```

JSON

```
{
  "infinispan" : {
    "server" : {
      "security" : {
        "security-realms" : [ {
          "name" : "keycloak",
          "server-identities" : {
            "ssl" : {
              "truststore" : {
```

```
            "path" : "ca.pfx",
            "password" : "secret"
          }
        }
      }
    },{
      "name" : "default",
      "server-identities" : {
        "ssl" : {
          "keystore" : {
            "path" : "server.pfx",
            "password" : "secret"
          }
        }
      },
      "token-realm" : {
        "name" : "token-realm",
        "auth-server-url" : "https://127.0.0.1:14568/auth/",
        "client-id" : "infinispan-console",
        "oauth2-introspection" : {
          "client-id" : "infinispan-server",
          "client-ssl-context" : "keycloak",
          "introspection-url" : "https://127.0.0.1:14568/realms/infinispan/protocol/openid-
connect/token/introspect",
          "client-secret" : "1fdca4ec-c416-47e0-867a-3d471af7050f"
        }
      }
    } ]
  },
  "endpoints" : {
    "endpoint" : {
      "socket-binding" : "default",
      "security-realm" : "default"
    }
  }
 }
}
```

YAML

```
infinispan:
  server:
    security:
      securityRealms:
        - name: "keycloak"
          serverIdentities:
            ssl:
              truststore:
                path: "ca.pfx"
                password: "secret"
        - name: "default"
          serverIdentities:
            ssl:
              keystore:
```

```
            path: "server.pfx"
            password: "secret"
        tokenRealm:
          name: "token-realm"
          authServerUrl: "https://127.0.0.1:14568/auth/"
          clientId: "infinispan-console"
          # Specifies the URL of the token introspection endpoint and uses the `keycloak` SSL context
defined above to trust the server
          oauth2Introspection:
            clientId: "infinispan-server"
            clientSslContext: "keycloak"
            introspectionUrl: "https://127.0.0.1:14568/realms/infinispan/protocol/openid-
connect/token/introspect"
            clientSecret: "1fdca4ec-c416-47e0-867a-3d471af7050f"
    endpoints:
      socketBinding: "default"
      securityRealm: "default"
```

5.6. TRUST STORE REALMS

Trust store realms use certificates, or certificates chains, that verify Data Grid Server and client identities when they negotiate connections.

Keystores

Contain server certificates that provide a Data Grid Server identity to clients. If you configure a keystore with server certificates, Data Grid Server encrypts traffic using industry standard SSL/TLS protocols.

Trust stores

Contain client certificates, or certificate chains, that clients present to Data Grid Server. Client trust stores are optional and allow Data Grid Server to perform client certificate authentication.

Client certificate authentication

You must add the **require-ssl-client-auth="true"** attribute to the endpoint configuration if you want Data Grid Server to validate or authenticate client certificates.

Trust store realm configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="trust-store-realm">
        <server-identities>
          <ssl>
            <!-- Provides an SSL/TLS identity with a keystore that contains server certificates. -->
            <keystore path="server.p12"
                      relative-to="infinispan.server.config.path"
                      keystore-password="secret"
                      alias="server"/>
            <!-- Configures a trust store that contains client certificates or part of a certificate chain. -->
            <truststore path="trust.p12"
                        relative-to="infinispan.server.config.path"
```

```
                    password="secret"/>
        </ssl>
      </server-identities>
      <!-- Authenticates client certificates against the trust store. If you configure this, the trust store
must contain the public certificates for all clients. -->
      <truststore-realm/>
    </security-realm>
  </security-realms>
 </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "trust-store-realm",
        "server-identities": {
          "ssl": {
            "keystore": {
              "path": "server.p12",
              "relative-to": "infinispan.server.config.path",
              "keystore-password": "secret",
              "alias": "server"
            },
            "truststore": {
              "path": "trust.p12",
              "relative-to": "infinispan.server.config.path",
              "password": "secret"
            }
          }
        },
        "truststore-realm": {}
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "trust-store-realm"
        serverIdentities:
          ssl:
            keystore:
              path: "server.p12"
              relative-to: "infinispan.server.config.path"
              keystore-password: "secret"
              alias: "server"
            truststore:
              path: "trust.p12"
```

```
            relative-to: "infinispan.server.config.path"
            password: "secret"
        trustStoreRealm: ~
```

5.7. DISTRIBUTED SECURITY REALMS

Distributed realms combine multiple different types of security realms. When users attempt to access the Hot Rod or REST endpoints, Data Grid Server uses each security realm in turn until it finds one that can perform the authentication.

Distributed realm configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="distributed-realm">
        <ldap-realm url="ldap://my-ldap-server:10389"
                    principal="uid=admin,ou=People,dc=infinispan,dc=org"
                    credential="strongPassword">
          <identity-mapping rdn-identifier="uid"
                            search-dn="ou=People,dc=infinispan,dc=org"
                            search-recursive="false">
            <attribute-mapping>
              <attribute from="cn" to="Roles"
                         filter="(&amp;(objectClass=groupOfNames)(member={1}))"
                         filter-dn="ou=Roles,dc=infinispan,dc=org"/>
            </attribute-mapping>
          </identity-mapping>
        </ldap-realm>
        <properties-realm groups-attribute="Roles">
          <user-properties path="users.properties"
                           relative-to="infinispan.server.config.path"/>
          <group-properties path="groups.properties"
                            relative-to="infinispan.server.config.path"/>
        </properties-realm>
        <distributed-realm/>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "distributed-realm",
        "ldap-realm": {
          "principal": "uid=admin,ou=People,dc=infinispan,dc=org",
          "url": "ldap://my-ldap-server:10389",
          "credential": "strongPassword",
```

```
        "identity-mapping": {
          "rdn-identifier": "uid",
          "search-dn": "ou=People,dc=infinispan,dc=org",
          "search-recursive": false,
          "attribute-mapping": {
            "attribute": {
              "filter": "(&(objectClass=groupOfNames)(member={1}))",
              "filter-dn": "ou=Roles,dc=infinispan,dc=org",
              "from": "cn",
              "to": "Roles"
            }
          }
        }
      },
      "properties-realm": {
        "groups-attribute": "Roles",
        "user-properties": {
          "digest-realm-name": "distributed-realm",
          "path": "users.properties"
        },
        "group-properties": {
          "path": "groups.properties"
        }
      },
      "distributed-realm": {}
    }]
  }
 }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "distributed-realm"
        ldapRealm:
          principal: "uid=admin,ou=People,dc=infinispan,dc=org"
          url: "ldap://my-ldap-server:10389"
          credential: "strongPassword"
          identityMapping:
            rdnIdentifier: "uid"
            searchDn: "ou=People,dc=infinispan,dc=org"
            searchRecursive: "false"
            attributeMapping:
              attribute:
                filter: "(&(objectClass=groupOfNames)(member={1}))"
                filterDn: "ou=Roles,dc=infinispan,dc=org"
                from: "cn"
                to: "Roles"
        propertiesRealm:
          groupsAttribute: "Roles"
          userProperties:
            digestRealmName: "distributed-realm"
            path: "users.properties"
```

```
      groupProperties:
        path: "groups.properties"
    distributedRealm: ~
```

5.8. AGGREGATE SECURITY REALMS

Aggregate realms combine multiple realms: the first one for the authentication steps and the others for loading the identity for the authorization steps. For example, this can be used to authenticate users via a client certificate, and retrieve identity from a properties or LDAP realm.

Aggregate realm configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="default" default-realm="aggregate">
        <server-identities>
          <ssl>
            <keystore path="server.pfx" password="secret" alias="server"/>
            <truststore path="trust.pfx" password="secret"/>
          </ssl>
        </server-identities>
        <properties-realm name="properties" groups-attribute="Roles">
          <user-properties path="users.properties" relative-to="infinispan.server.config.path"/>
          <group-properties path="groups.properties" relative-to="infinispan.server.config.path"/>
        </properties-realm>
        <truststore-realm name="trust"/>
        <aggregate-realm authentication-realm="trust" authorization-realms="properties">
          <name-rewriter>
            <common-name-principal-transformer/>
          </name-rewriter>
        </aggregate-realm>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [
        {
          "name": "aggregate-realm",
          "default-realm": "aggregate",
          "server-identities": {
            "ssl": {
              "keystore": {
                "path": "server.p12",
                "relative-to": "infinispan.server.config.path",
                "keystore-password": "secret",
```

```
            "alias": "server"
          },
          "truststore": {
            "path": "trust.p12",
            "relative-to": "infinispan.server.config.path",
            "password": "secret"
          }
        }
      },
      "properties-realm": {
        "name": "properties",
        "groups-attribute": "Roles",
        "user-properties": {
          "digest-realm-name": "distributed-realm",
          "path": "users.properties"
        },
        "group-properties": {
          "path": "groups.properties"
        }
      },
      "truststore-realm": {
        "name": "trust"
      },
      "aggregate-realm": {
        "authentication-realm": "trust",
        "authorization-realms": ["properties"],
        "name-rewriter": {
          "common-name-principal-transformer": {}
        }
      }
    }
  ]
 }
}
```

YAML

```
server:
 security:
  securityRealms:
   - name: "aggregate-realm"
     defaultRealm: "aggregate"
     serverIdentities:
       ssl:
         keystore:
           path: "server.p12"
           relative-to: "infinispan.server.config.path"
           keystore-password: "secret"
           alias: "server"
         truststore:
           path: "trust.p12"
           relative-to: "infinispan.server.config.path"
           password: "secret"
     truststoreRealm:
```

```
      name: "trust"
    propertiesRealm:
      name: "properties"
      groupsAttribute: "Roles"
      userProperties:
        digestRealmName: "distributed-realm"
        path: "users.properties"
      groupProperties:
        path: "groups.properties"
    aggregateRealm:
      authenticationRealm: "trust"
      authorizationRealms:
        - "properties"
      nameRewriter:
        common-name-principal-transformer: ~
```

5.8.1. Name rewriters

Principal names may have different forms, depending on the security realm type:

- Properties and Token realms may return simple strings
- Trust and LDAP realms may return X.500-style distinguished names
- Kerberos realms may return **user@domain**-style names

Names must be normalized to a common form when using the aggregate realm using one of the following transformers.

5.8.1.1. Case Principal Transformer

The **case-principal-transformer** transforms a name to all uppercase or all lowercase letters.

XML

```
<aggregate-realm authentication-realm="trust" authorization-realms="properties">
  <name-rewriter>
    <case-principal-transformer uppercase="false"/>
  </name-rewriter>
</aggregate-realm>
```

JSON

```
{
  "aggregate-realm": {
    "authentication-realm": "trust",
    "authorization-realms": [
      "properties"
    ],
    "name-rewriter": {
      "case-principal-transformer": {
        "uppercase": "false"
      }
```

```
    }
  }
}
```

YAML

```
aggregateRealm:
  authenticationRealm: "trust"
  authorizationRealms:
    - "properties"
  nameRewriter:
    casePrincipalTransformer:
      uppercase: false
```

5.8.1.2. Common Name Principal Transformer

The **common-name-principal-transformer** extracts the first **CN** element from a **DN** used by LDAP or Certificates. For example, given a principal in the form **CN=app1,CN=serviceA,OU=applications,DC=infinispan,DC=org**, the following configuration will extract **app1** as the principal.

XML

```
<aggregate-realm authentication-realm="trust" authorization-realms="properties">
  <name-rewriter>
    <common-name-principal-transformer/>
  </name-rewriter>
</aggregate-realm>
```

JSON

```
{
  "aggregate-realm": {
    "authentication-realm": "trust",
    "authorization-realms": [
      "properties"
    ],
    "name-rewriter": {
      "common-name-principal-transformer": {}
    }
  }
}
```

YAML

```
aggregateRealm:
  authenticationRealm: "trust"
  authorizationRealms:
    - "properties"
  nameRewriter:
    commonNamePrincipalTransformer: ~
```

5.8.1.3. Regex Principal Transformer

The **regex-principal-transformer** can perform find and replace using a regular expression. The example shows how to extract the local-part from a **user@domain.com** identifier.

XML

```
<aggregate-realm authentication-realm="trust" authorization-realms="properties">
  <name-rewriter>
    <regex-principal-transformer pattern="([^@]+)@.*" replacement="$1" replace-all="false"/>
  </name-rewriter>
</aggregate-realm>
```

JSON

```
{
  "aggregate-realm": {
    "authentication-realm": "trust",
    "authorization-realms": [
      "properties"
    ],
    "name-rewriter": {
      "regex-principal-transformer": {
        "pattern" : "([^@]+)@.*",
        "replacement": "$1",
        "replace-all": false
      }
    }
  }
}
```

YAML

```
aggregateRealm:
  authenticationRealm: "trust"
  authorizationRealms:
    - "properties"
  nameRewriter:
    regexPrincipalTransformer:
      pattern: "([^@]+)@.*"
      replacement: "$1"
      replaceAll: false
```

5.9. SECURITY REALM CACHING

Security realms implement caching to avoid having to repeatedly retrieve data which usually changes very infrequently. By default

Realm caching realm configuration

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="default" cache-max-size="1024" cache-lifespan="20m">
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```json
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "default",
        "cache-max-size": 1024,
        "cache-lifespan": 120000
      }]
    }
  }
}
```

YAML

```yaml
server:
  security:
    securityRealms:
      - name: "default"
        cache-max-size: 1024
        cache-lifespan: 120000
```

5.9.1. Flushing realm caches

Use the CLI to flush security realm caches across the whole cluster.

```
[node-1@mycluster//containers/default]> server aclcache flush
```

CHAPTER 6. CONFIGURING TLS/SSL ENCRYPTION

You can secure Data Grid Server connections using SSL/TLS encryption by configuring a keystore that contains public and private keys for Data Grid. You can also configure client certificate authentication if you require mutual TLS.

6.1. CONFIGURING DATA GRID SERVER KEYSTORES

Add keystores to Data Grid Server and configure it to present SSL/TLS certificates that verify its identity to clients. If a security realm contains TLS/SSL identities, it encrypts any connections to Data Grid Server endpoints that use that security realm.

Prerequisites

- Create a keystore that contains certificates, or certificate chains, for Data Grid Server.

Data Grid Server supports the following keystore formats: JKS, JCEKS, PKCS12/PFX and PEM. BKS, BCFKS, and UBER are also supported if the Bouncy Castle library is present.

Certificates should include the **subjectAltName** extension of type **dNSName** and/or **iPAddress** so that clients can correctly perform hostname validation, according to the rules defined by the RFC 2818 specification. The server will issue a warning if it is started with a certificate which does not include such an extension.



IMPORTANT

In production environments, server certificates should be signed by a trusted Certificate Authority, either Root or Intermediate CA.

TIP

You can use PEM files as keystores if they contain both of the following:

- A private key in PKCS#1 or PKCS#8 format.
- One or more certificates.

You should also configure PEM file keystores with an empty password (**password=""**).

Procedure

1. Open your Data Grid Server configuration for editing.
2. Add the keystore that contains SSL/TLS identities for Data Grid Server to the **$RHDG_HOME/server/conf** directory.
3. Add a **server-identities** definition to the Data Grid Server security realm.
4. Specify the keystore file name with the **path** attribute.
5. Provide the keystore password and certificate alias with the **keystore-password** and **alias** attributes.
6. Save the changes to your configuration.

Next steps

Configure clients with a trust store so they can verify SSL/TLS identities for Data Grid Server.

Keystore configuration

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="default">
        <server-identities>
          <ssl>
            <!-- Adds a keystore that contains server certificates that provide SSL/TLS identities to clients.
-->
            <keystore path="server.p12"
                      relative-to="infinispan.server.config.path"
                      password="secret"
                      alias="my-server"/>
          </ssl>
        </server-identities>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "default",
        "server-identities": {
          "ssl": {
            "keystore": {
              "alias": "my-server",
              "path": "server.p12",
              "password": "secret"
            }
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "default"
        serverIdentities:
```

```
ssl:
  keystore:
    alias: "my-server"
    path: "server.p12"
    password: "secret"
```

6.1.1. SSL/TLS Certificate rotation

SSL/TLS certificates have an expiration date, after which they are no longer valid. The process of renewing a certificate is also known as "rotation". Data Grid monitors the keystore files for changes and automatically reloads them without requiring a server or client restart.



NOTE

To ensure seamless operations during certificate rotation, use certificates signed by a Certificate Authority (CA) and configure both server and client trust stores with the CA certificate. Using self-signed certificates will cause temporary handshake failures until all clients and servers have been updated.

Additional resources

- Configuring Hot Rod client encryption

6.1.2. Generating Data Grid Server keystores

Configure Data Grid Server to automatically generate keystores at startup.



IMPORTANT

Automatically generated keystores:

- Should not be used in production environments.
- Are generated whenever necessary; for example, while obtaining the first connection from a client.
- Contain certificates that you can use directly in Hot Rod clients.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Include the **generate-self-signed-certificate-host** attribute for the **keystore** element in the server configuration.
3. Specify a hostname for the server certificate as the value.
4. Save the changes to your configuration.

Generated keystore configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
```

```
  <security>
    <security-realms>
      <security-realm name="generated-keystore">
        <server-identities>
          <ssl>
            <!-- Generates a keystore that includes a self-signed certificate with the specified hostname. --
>
            <keystore path="server.p12"
                      relative-to="infinispan.server.config.path"
                      password="secret"
                      alias="server"
                      generate-self-signed-certificate-host="localhost"/>
          </ssl>
        </server-identities>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "generated-keystore",
        "server-identities": {
          "ssl": {
            "keystore": {
              "alias": "server",
              "generate-self-signed-certificate-host": "localhost",
              "path": "server.p12",
              "password": "secret"
            }
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "generated-keystore"
        serverIdentities:
          ssl:
            keystore:
              alias: "server"
              generateSelfSignedCertificateHost: "localhost"
              path: "server.p12"
              password: "secret"
```

6.1.3. Configuring TLS versions and cipher suites

When using SSL/TLS encryption to secure your deployment, you can configure Data Grid Server to use specific versions of the TLS protocol as well as specific cipher suites within the protocol.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Add the **engine** element to the SSL configuration for Data Grid Server.
3. Configure Data Grid to use one or more TLS versions with the **enabled-protocols** attribute. Data Grid Server supports TLS version 1.2 and 1.3 by default. If appropriate you can set **TLSv1.3** only to restrict the security protocol for client connections. Data Grid does not recommend enabling **TLSv1.1** because it is an older protocol with limited support and provides weak security. You should never enable any version of TLS older than 1.1.



WARNING

If you modify the SSL **engine** configuration for Data Grid Server you must explicitly configure TLS versions with the **enabled-protocols** attribute. Omitting the **enabled-protocols** attribute allows any TLS version.

```
<engine enabled-protocols="TLSv1.3 TLSv1.2" />
```

4. Configure Data Grid to use one or more cipher suites with the **enabled-ciphersuites** attribute (for TLSv1.2 and below) and the **enabled-ciphersuites-tls13** attribute (for TLSv1.3). You must ensure that you set a cipher suite that supports any protocol features you plan to use; for example **HTTP/2 ALPN**.
5. Save the changes to your configuration.

SSL engine configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="default">
        <server-identities>
          <ssl>
            <keystore path="server.p12"
                      relative-to="infinispan.server.config.path"
                      password="secret"
                      alias="server"/>
            <!-- Configures Data Grid Server to use specific TLS versions and cipher suites. -->
            <engine enabled-protocols="TLSv1.3 TLSv1.2"
                    enabled-
```

```
ciphersuites="TLS_DHE_RSA_WITH_AES_128_CBC_SHA,TLS_DHE_RSA_WITH_AES_128_CBC_
SHA256"
                enabled-ciphersuites-
tls13="TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_S
HA256"/>
        </ssl>
       </server-identities>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "default",
        "server-identities": {
          "ssl": {
            "keystore": {
              "alias": "server",
              "path": "server.p12",
              "password": "secret"
            },
            "engine": {
              "enabled-protocols": ["TLSv1.3"],
              "enabled-ciphersuites":
"TLS_DHE_RSA_WITH_AES_128_CBC_SHA,TLS_DHE_RSA_WITH_AES_128_CBC_SHA256",
              "enabled-ciphersuites-tls13":
"TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_SHA25
6"
            }
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "default"
        serverIdentities:
          ssl:
            keystore:
              alias: "server"
              path: "server.p12"
              password: "secret"
            engine:
              enabledProtocols:
```

```
    - "TLSv1.3"
enabledCiphersuites: "TLS_AES_256_GCM_SHA384,TLS_AES_128_GCM_SHA256"
enabledCiphersuitesTls13: "TLS_AES_256_GCM_SHA384"
```

6.2. CONFIGURING DATA GRID SERVER ON A SYSTEM WITH FIPS 140-2 COMPLIANT CRYPTOGRAPHY

FIPS (Federal Information Processing Standards) are standards and guidelines for US federal computer systems. Although FIPS are developed for use by the US federal government, many in the private sector voluntarily use these standards.

FIPS 140-2 defines security requirements for cryptographic modules. You can configure your Data Grid Server to use encryption ciphers that adhere to the FIPS 140-2 specification by using alternative JDK security providers.

Additional resources

- Java PKCS#11 cryptographic provider
- The Legion of the Bouncy Castle cryptographic provider

6.2.1. Configuring the PKCS11 cryptographic provider

You can configure the PKCS11 cryptographic provider by specifying the PKCS11 keystore with the **SunPKCS11-NSS-FIPS** provider.

Prerequisites

- Configure your system for FIPS mode. You can check if your system has FIPS Mode enabled by issuing the **fips-mode-setup --check** command in your Data Grid command-line Interface (CLI)
- Initialize the system-wide NSS database by using the **certutil** tool.
- Install the JDK with the **java.security** file configured to enable the **SunPKCS11** provider. This provider points to the NSS database and the SSL provider.
- Install a certificate in the NSS database.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Add a **server-identities** definition to the Data Grid Server security realm.
3. Specify the PKCS11 keystore with the **SunPKCS11-NSS-FIPS** provider.
4. Save the changes to your configuration.

Keystore configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
```

```
    <security-realms>
      <security-realm name="default">
        <server-identities>
          <ssl>
            <!-- Adds a keystore that reads certificates from the NSS database. -->
            <keystore provider="SunPKCS11-NSS-FIPS" type="PKCS11"/>
          </ssl>
        </server-identities>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "default",
        "server-identities": {
          "ssl": {
            "keystore": {
              "provider": "SunPKCS11-NSS-FIPS",
              "type": "PKCS11"
            }
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "default"
        serverIdentities:
          ssl:
            keystore:
              provider: "SunPKCS11-NSS-FIPS"
              type: "PKCS11"
```

6.2.2. Configuring the Bouncy Castle FIPS cryptographic provider

You can configure the Bouncy Castle FIPS (Federal Information Processing Standards) cryptographic provider in your Data Grid server's configuration.

Prerequisites

- Configure your system for FIPS mode. You can check if your system has FIPS Mode enabled by issuing the **fips-mode-setup --check** command in your Data Grid command-line Interface (CLI).

- Create a keystore in BCFKS format that contains a certificate.

Procedure

1. Download the Bouncy Castle FIPS JAR file, and add the file to the **server/lib** directory of your Data Grid Server installation.
2. To install Bouncy Castle, issue the **install** command:

```
[disconnected]> install org.bouncycastle:bc-fips:1.0.2.3
```

3. Open your Data Grid Server configuration for editing.
4. Add a **server-identities** definition to the Data Grid Server security realm.
5. Specify the BCFKS keystore with the **BCFIPS** provider.
6. Save the changes to your configuration.

Keystore configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="default">
        <server-identities>
          <ssl>
            <!-- Adds a keystore that reads certificates from the BCFKS keystore. -->
            <keystore path="server.bcfks" password="secret" alias="server" provider="BCFIPS"
type="BCFKS"/>
          </ssl>
        </server-identities>
      </security-realm>
    </security-realms>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "default",
        "server-identities": {
          "ssl": {
            "keystore": {
              "path": "server.bcfks",
              "password": "secret",
              "alias": "server",
              "provider": "BCFIPS",
              "type": "BCFKS"
            }
```

```
        }
      }
    }]
  }
 }
}
```

YAML

```
server:
 security:
  securityRealms:
   - name: "default"
     serverIdentities:
       ssl:
         keystore:
           path: "server.bcfks"
           password: "secret"
           alias: "server"
           provider: "BCFIPS"
           type: "BCFKS"
```

6.3. CONFIGURING CLIENT CERTIFICATE AUTHENTICATION

Configure Data Grid Server to use mutual TLS to secure client connections.

You can configure Data Grid to verify client identities from certificates in a trust store in two ways:

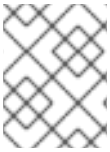
- Require a trust store that contains only the signing certificate, which is typically a Certificate Authority (CA). Any client that presents a certificate signed by the CA can connect to Data Grid.
- Require a trust store that contains all client certificates in addition to the signing certificate. Only clients that present a signed certificate that is present in the trust store can connect to Data Grid.

TIP

Alternatively to providing trust stores you can use shared system certificates.

Prerequisites

- Create a client trust store that contains either the CA certificate or all public certificates.
- Create a keystore for Data Grid Server and configure an SSL/TLS identity.



NOTE

PEM files can be used as trust stores provided they contain one or more certificates. These trust stores should be configured with an empty password: **password=""**.

Procedure

1. Open your Data Grid Server configuration for editing.

2. Add the **require-ssl-client-auth="true"** parameter to your **endpoints** configuration.
3. Add the client trust store to the **$RHDG_HOME/server/conf** directory.
4. Specify the **path** and **password** attributes for the **truststore** element in the Data Grid Server security realm configuration.
5. Add the **<truststore-realm/>** element to the security realm if you want Data Grid Server to authenticate each client certificate.
6. Save the changes to your configuration.

Next steps

- Set up authorization with client certificates in the Data Grid Server configuration if you control access with security roles and permissions.
- Configure clients to negotiate SSL/TLS connections with Data Grid Server.

Client certificate authentication configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="trust-store-realm">
        <server-identities>
          <ssl>
            <!-- Provides an SSL/TLS identity with a keystore that
                 contains server certificates. -->
            <keystore path="server.p12"
                      relative-to="infinispan.server.config.path"
                      keystore-password="secret"
                      alias="server"/>
            <!-- Configures a trust store that contains client certificates
                 or part of a certificate chain. -->
            <truststore path="trust.p12"
                        relative-to="infinispan.server.config.path"
                        password="secret"/>
          </ssl>
        </server-identities>
        <!-- Authenticates client certificates against the trust store. If you configure this, the trust store
must contain the public certificates for all clients. -->
        <truststore-realm/>
      </security-realm>
    </security-realms>
  </security>
  <endpoints>
    <endpoint socket-binding="default"
              security-realm="trust-store-realm"
              require-ssl-client-auth="true">
      <hotrod-connector>
        <authentication>
          <sasl mechanisms="EXTERNAL"
```

```
            server-name="infinispan"
            qop="auth"/>
        </authentication>
      </hotrod-connector>
      <rest-connector>
        <authentication mechanisms="CLIENT_CERT"/>
      </rest-connector>
    </endpoint>
  </endpoints>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "security-realms": [{
        "name": "trust-store-realm",
        "server-identities": {
          "ssl": {
            "keystore": {
              "path": "server.p12",
              "relative-to": "infinispan.server.config.path",
              "keystore-password": "secret",
              "alias": "server"
            },
            "truststore": {
              "path": "trust.p12",
              "relative-to": "infinispan.server.config.path",
              "password": "secret"
            }
          }
        },
        "truststore-realm": {}
      }]
    },
    "endpoints": [{
      "socket-binding": "default",
      "security-realm": "trust-store-realm",
      "require-ssl-client-auth": "true",
      "connectors": {
        "hotrod": {
          "hotrod-connector": {
            "authentication": {
              "sasl": {
                "mechanisms": "EXTERNAL",
                "server-name": "infinispan",
                "qop": "auth"
              }
            }
          },
          "rest": {
            "rest-connector": {
              "authentication": {
                "mechanisms": "CLIENT_CERT"
```

```
            }
          }
        }
      }
    }
  }]
 }
}
```

YAML

```
server:
  security:
    securityRealms:
      - name: "trust-store-realm"
        serverIdentities:
          ssl:
            keystore:
              path: "server.p12"
              relative-to: "infinispan.server.config.path"
              keystore-password: "secret"
              alias: "server"
            truststore:
              path: "trust.p12"
              relative-to: "infinispan.server.config.path"
              password: "secret"
        truststoreRealm: ~
  endpoints:
    socketBinding: "default"
    securityRealm: "trust-store-realm"
    requireSslClientAuth: "true"
    connectors:
      - hotrod:
          hotrodConnector:
            authentication:
              sasl:
                mechanisms: "EXTERNAL"
                serverName: "infinispan"
                qop: "auth"
      - rest:
          restConnector:
            authentication:
              mechanisms: "CLIENT_CERT"
```

Additional resources

- Configuring Hot Rod client encryption
- Using Shared System Certificates (Red Hat Enterprise Linux 7 Security Guide)

6.4. CONFIGURING AUTHORIZATION WITH CLIENT CERTIFICATES

Enabling client certificate authentication means you do not need to specify Data Grid user credentials in client configuration, which means you must associate roles with the Common Name (CN) field in the client certificate(s).

Prerequisites

- Provide clients with a Java keystore that contains either their public certificates or part of the certificate chain, typically a public CA certificate.
- Configure Data Grid Server to perform client certificate authentication.

Procedure

1. Open your Data Grid Server configuration for editing.
2. Enable the **common-name-role-mapper** in the security authorization configuration.
3. Assign the Common Name (**CN**) from the client certificate a role with the appropriate permissions.
4. Save the changes to your configuration.



NOTE

Data Grid creates the identity of the client by extracting the certificate principal. Any other Subject Alternative Names (SANs) which may be present in the certificate are currently ignored. For this reason, the **authorization.group-only-mapping** attribute below must be set to **false**.

Client certificate authorization configuration

XML

```
<infinispan>
  <cache-container name="certificate-authentication" statistics="true">
    <security>
      <authorization group-only-mapping="false">
        <!-- Declare a role mapper that associates the common name (CN) field in client certificate trust stores with authorization roles. -->
        <common-name-role-mapper/>
        <!-- In this example, if a client certificate contains `CN=Client1` then clients with matching certificates get ALL permissions. -->
        <role name="Client1" permissions="ALL"/>
      </authorization>
    </security>
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan": {
    "cache-container": {
```

```
    "name": "certificate-authentication",
    "security": {
      "authorization": {
        "group-only-mapping": false,
        "common-name-role-mapper": null,
        "roles": {
          "Client1": {
            "role": {
              "permissions": "ALL"
            }
          }
        }
      }
    }
  }
}
```

YAML

```
infinispan:
  cacheContainer:
    name: "certificate-authentication"
    security:
      authorization:
        groupOnlyMapping: false
        commonNameRoleMapper: ~
        roles:
          Client1:
            role:
              permissions:
                - "ALL"
```

CHAPTER 7. STORING DATA GRID SERVER CREDENTIALS IN KEYSTORES

External services require credentials to authenticate with Data Grid Server. To protect sensitive text strings such as passwords, add them to a credential keystore rather than directly in Data Grid Server configuration files.

You can then configure Data Grid Server to decrypt passwords for establishing connections with services such as databases or LDAP directories.



IMPORTANT

Plain-text passwords in **$RHDG_HOME/server/conf** are unencrypted. Any user account with read access to the host filesystem can view plain-text passwords.

While credential keystores are password-protected store encrypted passwords, any user account with write access to the host filesystem can tamper with the keystore itself.

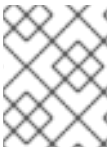
To completely secure Data Grid Server credentials, you should grant read-write access only to user accounts that can configure and run Data Grid Server.

7.1. SETTING UP CREDENTIAL KEYSTORES

Create keystores that encrypt credential for Data Grid Server access.

A credential keystore contains at least one alias that is associated with an encrypted password. After you create a keystore, you specify the alias in a connection configuration such as a database connection pool. Data Grid Server then decrypts the password for that alias from the keystore when the service attempts authentication.

You can create as many credential keystores with as many aliases as required.



NOTE

As a security best practice, keystores should be readable only by the user who runs the process for Data Grid Server.

Procedure

1. Open a terminal in **$RHDG_HOME**.
2. Create a keystore and add credentials to it with the **credentials** command.

TIP

By default, keystores are of type PKCS12. Run **help credentials** for details on changing keystore defaults.

The following example shows how to create a keystore that contains an alias of "dbpassword" for the password "changeme". When you create a keystore you also specify a password to access the keystore with the **-p** argument.

Linux

```
bin/cli.sh credentials add dbpassword -c changeme -p "secret1234!"
```

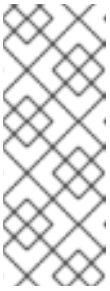
Microsoft Windows

```
bin\cli.bat credentials add dbpassword -c changeme -p "secret1234!"
```

3. Check that the alias is added to the keystore.

```
bin/cli.sh credentials ls -p "secret1234!"
dbpassword
```

4. Open your Data Grid Server configuration for editing.
5. Configure Data Grid to use the credential keystore.
   a. Add a **credential-stores** section to the **security** configuration.
   b. Specify the name and location of the credential keystore.
   c. Specify the password to access the credential keystore with the **clear-text-credential** configuration.



NOTE

Instead of adding a clear-text password for the credential keystore to your Data Grid Server configuration you can use an external command or masked password for additional security.

You can also use a password in one credential store as the master password for another credential store.

6. Reference the credential keystore in configuration that Data Grid Server uses to connect with an external system such as a datasource or LDAP server.
   a. Add a **credential-reference** section.
   b. Specify the name of the credential keystore with the **store** attribute.
   c. Specify the password alias with the **alias** attribute.

TIP

Attributes in the **credential-reference** configuration are optional.

- **store** is required only if you have multiple keystores.
- **alias** is required only if the keystore contains multiple password aliases.

7. Save the changes to your configuration.

7.2. SECURING PASSWORDS FOR CREDENTIAL KEYSTORES

Data Grid Server requires a password to access credential keystores. You can add that password to Data Grid Server configuration in clear text or, as an added layer of security, you can use an external command for the password or you can mask the password.

Prerequisites

- Set up a credential keystore for Data Grid Server.

Procedure

Do one of the following:

- Use the **credentials mask** command to obscure the password, for example:

```
bin/cli.sh credentials mask -i 100 -s pepper99 "secret1234!"
```

  Masked passwords use Password Based Encryption (PBE) and must be in the following format in your Data Grid Server configuration: <MASKED_VALUE;SALT;ITERATION>.

- Use an external command that provides the password as standard output.
  An external command can be any executable, such as a shell script or binary, that uses **java.lang.Runtime#exec(java.lang.String)**.
  If the command requires parameters, provide them as a space-separated list of strings.

7.3. CREDENTIAL KEYSTORE CONFIGURATION

You can add credential keystores to Data Grid Server configuration and use clear-text passwords, masked passwords, or external commands that supply passwords.

Credential keystore with a clear text password

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <credential-stores>
      <credential-store name="credentials" path="credentials.pfx">
        <clear-text-credential clear-text="secret1234!"/>
      </credential-store>
    </credential-stores>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "credential-stores": [{
        "name": "credentials",
        "path": "credentials.pfx",
        "clear-text-credential": {
          "clear-text": "secret1234!"
        }
```

```
      }]
    }
  }
}
```

YAML

```
server:
  security:
    credentialStores:
      - name: credentials
        path: credentials.pfx
        clearTextCredential:
          clearText: "secret1234!"
```

Credential keystore with a masked password

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <credential-stores>
      <credential-store name="credentials"
                        path="credentials.pfx">
        <masked-credential masked="1oTMDZ5JQj6DVepJviXMnX;pepper99;100"/>
      </credential-store>
    </credential-stores>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "credential-stores": [{
        "name": "credentials",
        "path": "credentials.pfx",
        "masked-credential": {
          "masked": "1oTMDZ5JQj6DVepJviXMnX;pepper99;100"
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    credentialStores:
      - name: credentials
```

```
      path: credentials.pfx
      maskedCredential:
        masked: "1oTMDZ5JQj6DVepJviXMnX;pepper99;100"
```

External command passwords

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <credential-stores>
      <credential-store name="credentials"
                        path="credentials.pfx">
        <command-credential command="/path/to/executable.sh arg1 arg2"/>
      </credential-store>
    </credential-stores>
  </security>
</server>
```

JSON

```
{
  "server": {
    "security": {
      "credential-stores": [{
        "name": "credentials",
        "path": "credentials.pfx",
        "command-credential": {
          "command": "/path/to/executable.sh arg1 arg2"
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    credentialStores:
      - name: credentials
        path: credentials.pfx
        commandCredential:
          command: "/path/to/executable.sh arg1 arg2"
```

7.4. CREDENTIAL KEYSTORE REFERENCES

After you add credential keystores to Data Grid Server you can reference them in connection configurations.

Datasource connections

XML

```xml
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <credential-stores>
      <credential-store name="credentials"
                        path="credentials.pfx">
        <clear-text-credential clear-text="secret1234!"/>
      </credential-store>
    </credential-stores>
  </security>
  <data-sources>
    <data-source name="postgres"
                 jndi-name="jdbc/postgres">
      <!-- Specifies the database username in the connection factory. -->
      <connection-factory driver="org.postgresql.Driver"
                          username="dbuser"
                          url="${org.infinispan.server.test.postgres.jdbcUrl}">
        <!-- Specifies the credential keystore that contains an encrypted password and the alias for it. --
>
        <credential-reference store="credentials"
                              alias="dbpassword"/>
      </connection-factory>
      <connection-pool max-size="10"
                       min-size="1"
                       background-validation="1000"
                       idle-removal="1"
                       initial-size="1"
                       leak-detection="10000"/>
    </data-source>
  </data-sources>
</server>
```

JSON

```json
{
  "server": {
    "security": {
      "credential-stores": [{
        "name": "credentials",
        "path": "credentials.pfx",
        "clear-text-credential": {
          "clear-text": "secret1234!"
        }
      }],
      "data-sources": [{
        "name": "postgres",
        "jndi-name": "jdbc/postgres",
        "connection-factory": {
          "driver": "org.postgresql.Driver",
          "username": "dbuser",
          "url": "${org.infinispan.server.test.postgres.jdbcUrl}",
          "credential-reference": {
            "store": "credentials",
            "alias": "dbpassword"
```

```
        }
      }
    }]
  }
 }
}
```

YAML

```
server:
 security:
  credentialStores:
   - name: credentials
     path: credentials.pfx
     clearTextCredential:
       clearText: "secret1234!"
 dataSources:
  - name: postgres
    jndiName: jdbc/postgres
    connectionFactory:
     driver: org.postgresql.Driver
     username: dbuser
     url: '${org.infinispan.server.test.postgres.jdbcUrl}'
     credentialReference:
       store: credentials
       alias: dbpassword
```

LDAP connections

XML

```
<server xmlns="urn:infinispan:server:16.0">
 <security>
  <credential-stores>
   <credential-store name="credentials"
                     path="credentials.pfx">
    <clear-text-credential clear-text="secret1234!"/>
   </credential-store>
  </credential-stores>
  <security-realms>
   <security-realm name="default">
    <!-- Specifies the LDAP principal in the connection factory. -->
    <ldap-realm name="ldap"
                url="ldap://my-ldap-server:10389"
                principal="uid=admin,ou=People,dc=infinispan,dc=org">
     <!-- Specifies the credential keystore that contains an encrypted password and the alias for it. -
->
      <credential-reference store="credentials"
                            alias="ldappassword"/>
    </ldap-realm>
   </security-realm>
  </security-realms>
 </security>
</server>
```

JSON

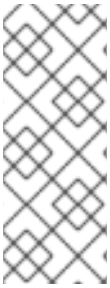
```
{
  "server": {
    "security": {
      "credential-stores": [{
        "name": "credentials",
        "path": "credentials.pfx",
        "clear-text-credential": {
          "clear-text": "secret1234!"
        }
      }],
      "security-realms": [{
        "name": "default",
        "ldap-realm": {
          "name": "ldap",
          "url": "ldap://my-ldap-server:10389",
          "principal": "uid=admin,ou=People,dc=infinispan,dc=org",
          "credential-reference": {
            "store": "credentials",
            "alias": "ldappassword"
          }
        }
      }]
    }
  }
}
```

YAML

```
server:
  security:
    credentialStores:
      - name: credentials
        path: credentials.pfx
        clearTextCredential:
          clearText: "secret1234!"
    securityRealms:
      - name: "default"
        ldapRealm:
          name: ldap
          url: 'ldap://my-ldap-server:10389'
          principal: 'uid=admin,ou=People,dc=infinispan,dc=org'
          credentialReference:
            store: credentials
            alias: ldappassword
```

CHAPTER 8. SECURITY AUTHORIZATION WITH ROLE-BASED ACCESS CONTROL

Role-based access control (RBAC) capabilities use different permissions levels to restrict user interactions with Data Grid.



NOTE

For information on creating users and configuring authorization specific to remote or embedded caches, see:

- Configuring user roles and permissions with Data Grid Server
- Programmatically configuring user roles and permissions

8.1. DATA GRID USER ROLES AND PERMISSIONS

Data Grid includes several roles that provide users with permissions to access caches and Data Grid resources.

Role	Permissions	Description
admin	ALL	Superuser with all permissions including control of the Cache Manager lifecycle.
deployer	ALL_READ, ALL_WRITE, LISTEN, EXEC, MONITOR, CREATE	Can create and delete Data Grid resources in addition to **application** permissions.
application	ALL_READ, ALL_WRITE, LISTEN, EXEC, MONITOR	Has read and write access to Data Grid resources in addition to **observer** permissions. Can also listen to events and execute server tasks and scripts.
observer	ALL_READ, MONITOR	Has read access to Data Grid resources in addition to **monitor** permissions.
monitor	MONITOR	Can view statistics via JMX and the **metrics** endpoint.

Additional resources

- org.infinispan.security.AuthorizationPermission Enum
- Data Grid configuration schema reference

8.1.1. Permissions

User roles are sets of permissions with different access levels.

Table 8.1. Cache Manager permissions

Permission	Function	Description
CONFIGURATION	**defineConfiguration**	Defines new cache configurations.
LISTEN	**addListener**	Registers listeners against a Cache Manager.
LIFECYCLE	**stop**	Stops the Cache Manager.
CREATE	**createCache**, **removeCache**	Create and remove container resources such as caches, counters, schemas, and scripts.
MONITOR	**getStats**	Allows access to JMX statistics and the **metrics** endpoint.
ALL	-	Includes all Cache Manager permissions.

Table 8.2. Cache permissions

Permission	Function	Description
READ	**get**, **contains**	Retrieves entries from a cache.
WRITE	**put**, **putIfAbsent**, **replace**, **remove**, **evict**	Writes, replaces, removes, evicts data in a cache.
EXEC	**distexec**, **streams**	Allows code execution against a cache.
LISTEN	**addListener**	Registers listeners against a cache.
BULK_READ	**keySet**, **values**, **entrySet**, **query**	Executes bulk retrieve operations.
BULK_WRITE	**clear**, **putAll**	Executes bulk write operations.
LIFECYCLE	**start**, **stop**	Starts and stops a cache.

ADMIN	**getVersion**, **addInterceptor***, **removeInterceptor**, **getInterceptorChain**, **getEvictionManager**, **getComponentRegistry**, **getDistributionManager**, **getAuthorizationManager**, **evict**, **getRpcManager**, **getCacheConfiguration**, **getCacheManager**, **getInvocationContextContainer**, **setAvailability**, **getDataContainer**, **getStats**, **getXAResource**	Allows access to underlying components and internal structures.
MONITOR	**getStats**	Allows access to JMX statistics and the **metrics** endpoint.
ALL	-	Includes all cache permissions.
ALL_READ	-	Combines the READ and BULK_READ permissions.
ALL_WRITE	-	Combines the WRITE and BULK_WRITE permissions.

Additional resources

- Data Grid Security API

8.1.2. Role and permission mappers

Data Grid implements users as a collection of principals. Principals represent either an individual user identity, such as a username, or a group to which the users belong. Internally, these are implemented with the **javax.security.auth.Subject** class.

To enable authorization, the principals must be mapped to role names, which are then expanded into a set of permissions.

Data Grid includes the **PrincipalRoleMapper** API for associating security principals to roles, and the **RolePermissionMapper** API for associating roles with specific permissions.

Data Grid provides the following role and permission mapper implementations:

Cluster role mapper

Stores principal to role mappings in the cluster registry.

Cluster permission mapper

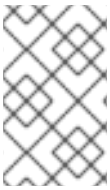
Stores role to permission mappings in the cluster registry. Allows you to dynamically modify user roles and permissions.

Identity role mapper

Uses the principal name as the role name. The type or format of the principal name depends on the source. For example, in an LDAP directory the principal name could be a Distinguished Name (DN).

Common name role mapper

Uses the Common Name (CN) as the role name. You can use this role mapper with an LDAP directory or with client certificates that contain Distinguished Names (DN); for example **cn=managers,ou=people,dc=example,dc=com** maps to the **managers** role.



NOTE

By default, principal-to-role mapping is only applied to principals which represent groups. It is possible to configure Data Grid to also perform the mapping for user principals by setting the **authorization.group-only-mapping** configuration attribute to **false**.

8.1.2.1. Mapping users to roles and permissions in Data Grid

Consider the following user retrieved from an LDAP server, as a collection of DNs:

```
CN=myapplication,OU=applications,DC=mycompany
CN=dataprocessors,OU=groups,DC=mycompany
CN=finance,OU=groups,DC=mycompany
```

Using the **Common name role mapper**, the user would be mapped to the following roles:

```
dataprocessors
finance
```

Data Grid has the following role definitions:

```
dataprocessors: ALL_WRITE ALL_READ
finance: LISTEN
```

The user would have the following permissions:

```
ALL_WRITE ALL_READ LISTEN
```

Additional resources

- Data Grid Security API
- org.infinispan.security.PrincipalRoleMapper
- org.infinispan.security.RolePermissionMapper
- org.infinispan.security.mappers.IdentityRoleMapper
- org.infinispan.security.mappers.CommonNameRoleMapper

8.1.3. Configuring role mappers

Data Grid enables the cluster role mapper and cluster permission mapper by default. To use a different implementation for role mapping, you must configure the role mappers.

Procedure

Procedure

1. Open your Data Grid configuration for editing.
2. Declare the role mapper as part of the security authorization in the Cache Manager configuration.
3. Save the changes to your configuration.

Role mapper configuration

XML

```
<cache-container>
  <security>
    <authorization>
      <common-name-role-mapper />
    </authorization>
  </security>
</cache-container>
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
      "security" : {
        "authorization" : {
          "common-name-role-mapper": {}
        }
      }
    }
  }
}
```

YAML

```
infinispan:
  cacheContainer:
    security:
      authorization:
        commonNameRoleMapper: ~
```

Additional resources

- Data Grid configuration schema reference

8.1.4. Configuring the cluster role and permission mappers

The cluster role mapper maintains a dynamic mapping between principals and roles. The cluster permission mapper maintains a dynamic set of role definitions. In both cases, the mappings are stored in the cluster registry and can be manipulated at runtime using either the CLI or the REST API.

Prerequisites

Prerequisites

- Have **ADMIN** permissions for Data Grid.
- Start the Data Grid CLI.
- Connect to a running Data Grid cluster.

8.1.4.1. Creating new roles

Create new roles and set the permissions.

Procedure

- Create roles with the **user roles create** command, for example:

```
user roles create --permissions=ALL_READ,ALL_WRITE simple
```

Verification

List roles that you grant to users with the **user roles ls** command.

```
user roles ls
["observer","application","admin","monitor","simple","deployer"]
```

Describe roles with the **user roles describe** command.

```
user roles describe simple
{
  "name" : "simple",
  "permissions" : [ "ALL_READ","ALL_WRITE" ]
}
```

8.1.4.2. Granting roles to users

Assign roles to users and grant them permissions to perform cache operations and interact with Data Grid resources.

TIP

Grant roles to groups instead of users if you want to assign the same role to multiple users and centrally maintain their permissions.

Prerequisites

- Have **ADMIN** permissions for Data Grid.
- Create Data Grid users.

Procedure

1. Create a CLI connection to Data Grid.
2. Assign roles to users with the **user roles grant** command, for example:

```
user roles grant --roles=deployer katie
```

Verification

List roles that you grant to users with the **user roles ls** command.

```
user roles ls katie
["deployer"]
```

8.1.4.3. Cluster role mapper name rewriters

By default, the mapping is performed using a strict string equivalence between principal names and roles. It is possible to configure the cluster role mapper to apply transformation to the principal name before performing a lookup.

Procedure

1. Open your Data Grid configuration for editing.
2. Specify a name rewriter for the cluster role mapper as part of the security authorization in the Cache Manager configuration.
3. Save the changes to your configuration.

Principal names may have different forms, depending on the security realm type:

- Properties and Token realms may return simple strings
- Trust and LDAP realms may return X.500-style distinguished names
- Kerberos realms may return **user@domain**-style names

Names can be normalized to a common form using one of the following transformers:

8.1.4.3.1. Case Principal Transformer

XML

```
<cache-container>
  <security>
    <authorization>
      <cluster-role-mapper>
        <name-rewriter>
          <case-principal-transformer uppercase="false"/>
        </name-rewriter>
      </cluster-role-mapper>
    </authorization>
  </security>
</cache-container>
```

JSON

```
{
```

```
  "cache-container": {
    "security": {
      "authorization": {
        "cluster-role-mapper": {
          "name-rewriter": {
            "case-principal-transformer": {}
          }
        }
      }
    }
  }
}
```

YAML

```
cacheContainer:
  security:
    authorization:
      clusterRoleMapper:
        nameRewriter:
          casePrincipalTransformer:
            uppercase: false
```

8.1.4.3.2. Regex Principal Transformer

XML

```
<cache-container>
  <security>
    <authorization>
      <cluster-role-mapper>
        <name-rewriter>
          <regex-principal-transformer pattern="cn=([^,]+),.*" replacement="$1"/>
        </name-rewriter>
      </cluster-role-mapper>
    </authorization>
  </security>
</cache-container>
```

JSON

```
{
  "cache-container": {
    "security": {
      "authorization": {
        "cluster-role-mapper": {
          "name-rewriter": {
            "regex-principal-transformer": {
              "pattern": "cn=([^,]+),.*",
              "replacement": "$1"
            }
          }
        }
```

```
      }
    }
  }
}
```

YAML

```
cacheContainer:
  security:
    authorization:
      clusterRoleMapper:
        nameRewriter:
          regexPrincipalTransformer:
            pattern: "cn=([^,]+),.*"
            replacement: "$1"
```

Additional resources

- Data Grid configuration schema reference

8.2. CONFIGURING CACHES WITH SECURITY AUTHORIZATION

Add security authorization to caches to enforce role-based access control (RBAC). This requires Data Grid users to have a role with a sufficient level of permission to perform cache operations.

Prerequisites

- Create Data Grid users and either grant them with roles or assign them to groups.

Procedure

1. Open your Data Grid configuration for editing.
2. Add a **security** section to the configuration.
3. Specify roles that users must have to perform cache operations with the **authorization** element.
   You can implicitly add all roles defined in the Cache Manager or explicitly define a subset of roles.
4. Save the changes to your configuration.

Implicit role configuration

The following configuration implicitly adds every role defined in the Cache Manager:

XML

```
<distributed-cache>
  <security>
    <authorization/>
  </security>
</distributed-cache>
```

JSON

```
{
  "distributed-cache": {
    "security": {
      "authorization": {
        "enabled": true
      }
    }
  }
}
```

YAML

```
distributedCache:
  security:
    authorization:
      enabled: true
```

Explicit role configuration

The following configuration explicitly adds a subset of roles defined in the Cache Manager. In this case Data Grid denies cache operations for any users that do not have one of the configured roles.

XML

```
<distributed-cache>
  <security>
    <authorization roles="admin supervisor"/>
  </security>
</distributed-cache>
```

JSON

```
{
  "distributed-cache": {
    "security": {
      "authorization": {
        "enabled": true,
        "roles": ["admin","supervisor"]
      }
    }
  }
}
```

YAML

```
distributedCache:
  security:
    authorization:
      enabled: true
      roles: ["admin","supervisor"]
```

CHAPTER 9. ENABLING AND CONFIGURING DATA GRID STATISTICS AND JMX MONITORING

Data Grid can provide Cache Manager and cache statistics as well as export JMX MBeans.

9.1. ENABLING STATISTICS IN REMOTE CACHES

Data Grid Server automatically enables statistics for the default Cache Manager. However, you must explicitly enable statistics for your caches.

Procedure

1. Open your Data Grid configuration for editing.
2. Add the **statistics** attribute or field and specify **true** as the value.
3. Save and close your Data Grid configuration.

Remote cache statistics

XML

```
<distributed-cache statistics="true" />
```

JSON

```
{
  "distributed-cache": {
    "statistics": "true"
  }
}
```

YAML

```
distributedCache:
  statistics: true
```

9.2. ENABLING HOT ROD CLIENT STATISTICS

Hot Rod Java clients can provide statistics that include remote cache and near-cache hits and misses.

Procedure

1. Open your Hot Rod Java client configuration for editing.
2. Set **true** as the value for the **statistics** property or invoke the **statistics().enable()** methods.
3. Export JMX MBeans for your Hot Rod client with the **jmx** and **jmx_domain** properties or invoke the **jmxEnable()** and **jmxDomain()** methods.

4. Save and close your client configuration.

Hot Rod Java client statistics

ConfigurationBuilder

```
ConfigurationBuilder builder = new ConfigurationBuilder();
builder.statistics().enable()
         .jmxEnable()
         .jmxDomain("my.domain.org")
       .addServer()
         .host("127.0.0.1")
         .port(11222);
RemoteCacheManager remoteCacheManager = new RemoteCacheManager(builder.build());
```

hotrod-client.properties

```
infinispan.client.hotrod.statistics = true
infinispan.client.hotrod.jmx = true
infinispan.client.hotrod.jmx_domain = my.domain.org
```

9.3. CONFIGURING DATA GRID METRICS

Data Grid generates metrics that are compatible with any monitoring system.

- Gauges provide values such as the average number of nanoseconds for write operations or JVM uptime.
- Histograms provide details about operation execution times such as read, write, and remove times.

By default, Data Grid generates gauges when you enable statistics but you can also configure it to generate histograms.



NOTE

Data Grid metrics are provided at the **vendor** scope. Metrics related to the JVM are provided in the **base** scope.

Procedure

1. Open your Data Grid configuration for editing.
2. Add the **metrics** element or object to the cache container.
3. Enable or disable gauges with the **gauges** attribute or field.
4. Enable or disable histograms with the **histograms** attribute or field.
5. Enable or disable the export of metrics in legacy format using the **legacy** attribute or field.
6. Save and close your client configuration.

Metrics configuration

XML

```
<infinispan>
  <cache-container statistics="true">
    <metrics gauges="true"
             histograms="true" />
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
      "statistics" : "true",
      "metrics" : {
        "gauges" : "true",
        "histograms" : "true"
      }
    }
  }
}
```

YAML

```
infinispan:
  cacheContainer:
    statistics: "true"
    metrics:
      gauges: "true"
      histograms: "true"
```

Legacy format
For backwards compatibility, metrics are currently exported in a legacy format by default. The setting **legacy** should be set to **false** to export metrics in the new format.

XML

```
<infinispan>
  <cache-container statistics="true">
    <metrics legacy="false" />
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
```

```
      "statistics" : "true",
      "metrics" : {
        "legacy" : "false"
      }
    }
  }
}
```

YAML

```
infinispan:
  cacheContainer:
    statistics: "true"
    metrics:
      legacy: "false"
```

Verification

Data Grid Server exposes statistics through the **metrics** endpoint that you can collect with monitoring tools such as Prometheus. To verify that statistics are exported to the **metrics** endpoint, you can do the following:

Prometheus format

```
curl -v http://localhost:11222/metrics \
--digest -u username:password
```

OpenMetrics format

```
curl -v http://localhost:11222/metrics \
--digest -u username:password \
-H "Accept: application/openmetrics-text"
```



NOTE

Data Grid no longer provides metrics in MicroProfile JSON format.

Additional resources

- Data Grid Metrics
- Micrometer Prometheus

9.4. REGISTERING JMX MBEANS

Data Grid can register JMX MBeans that you can use to collect statistics and perform administrative operations. You must also enable statistics otherwise Data Grid provides **0** values for all statistic attributes in JMX MBeans.



IMPORTANT

Use JMX Mbeans for collecting statistics only when Data Grid is embedded in applications and not with a remote Data Grid server.

When you use JMX Mbeans for collecting statistics from a remote Data Grid server, the data received from JMX Mbeans might differ from the data received from other APIs such as REST. In such cases the data received from the other APIs is more accurate.

Procedure

1. Open your Data Grid configuration for editing.
2. Add the **jmx** element or object to the cache container and specify **true** as the value for the **enabled** attribute or field.
3. Add the **domain** attribute or field and specify the domain where JMX MBeans are exposed, if required.
4. Save and close your client configuration.

JMX configuration

XML

```
<infinispan>
  <cache-container statistics="true">
    <jmx enabled="true"
         domain="example.com"/>
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
      "statistics" : "true",
      "jmx" : {
        "enabled" : "true",
        "domain" : "example.com"
      }
    }
  }
}
```

YAML

```
infinispan:
  cacheContainer:
    statistics: "true"
```

```
jmx:
  enabled: "true"
  domain: "example.com"
```

9.4.1. Enabling JMX remote ports

Provide unique remote JMX ports to expose Data Grid MBeans through connections in JMXServiceURL format.



NOTE

Data Grid Server does not expose JMX remotely via the single port endpoint. If you want to remotely access Data Grid Server via JMX you must enable a remote port.

You can enable remote JMX ports using one of the following approaches:

- Enable remote JMX ports that require authentication to one of the Data Grid Server security realms.
- Enable remote JMX ports manually using the standard Java management configuration options.

Prerequisites

- For remote JMX with authentication, define JMX specific user roles using the default security realm. Users must have **controlRole** with read/write access or the **monitorRole** with read-only access to access any JMX resources. Data Grid automatically maps global **ADMIN** and **MONITOR** permissions to the JMX **controlRole** and **monitorRole** roles.

Procedure

Start Data Grid Server with a remote JMX port enabled using one of the following ways:

- Enable remote JMX through port **9999**.

```
bin/server.sh --jmx 9999
```



WARNING

Using remote JMX with SSL disabled is not intended for production environments.

- Pass the following system properties to Data Grid Server at startup.

```
bin/server.sh -Dcom.sun.management.jmxremote.port=9999 -
Dcom.sun.management.jmxremote.authenticate=false -
Dcom.sun.management.jmxremote.ssl=false
```



WARNING

Enabling remote JMX with no authentication or SSL is not secure and not recommended in any environment. Disabling authentication and SSL allows unauthorized users to connect to your server and access the data hosted there.

Additional resources

- Creating security realms

9.4.2. Data Grid MBeans

Data Grid exposes JMX MBeans that represent manageable resources.

org.infinispan:type=Cache

Attributes and operations available for cache instances.

org.infinispan:type=CacheManager

Attributes and operations available for Cache Managers, including Data Grid cache and cluster health statistics.

For a complete list of available JMX MBeans along with descriptions and available operations and attributes, see the *Data Grid JMX Components* documentation.

Additional resources

- Data Grid JMX Components

9.4.3. Registering MBeans in custom MBean servers

Data Grid includes an **MBeanServerLookup** interface that you can use to register MBeans in custom MBeanServer instances.

Prerequisites

- Create an implementation of **MBeanServerLookup** so that the **getMBeanServer()** method returns the custom MBeanServer instance.
- Configure Data Grid to register JMX MBeans.

Procedure

1. Open your Data Grid configuration for editing.
2. Add the **mbean-server-lookup** attribute or field to the JMX configuration for the Cache Manager.
3. Specify fully qualified name (FQN) of your **MBeanServerLookup** implementation.

4. Save and close your client configuration.

JMX MBean server lookup configuration

XML

```
<infinispan>
  <cache-container statistics="true">
    <jmx enabled="true"
         domain="example.com"
         mbean-server-lookup="com.example.MyMBeanServerLookup"/>
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
      "statistics" : "true",
      "jmx" : {
        "enabled" : "true",
        "domain" : "example.com",
        "mbean-server-lookup" : "com.example.MyMBeanServerLookup"
      }
    }
  }
}
```

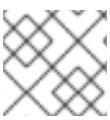
YAML

```
infinispan:
  cacheContainer:
    statistics: "true"
    jmx:
      enabled: "true"
      domain: "example.com"
      mbeanServerLookup: "com.example.MyMBeanServerLookup"
```

9.5. EXPORTING METRICS DURING A STATE TRANSFER OPERATION

You can export time metrics for clustered caches that Data Grid redistributes across nodes.

A state transfer operation occurs when a clustered cache topology changes, such as a node joining or leaving a cluster. During a state transfer operation, Data Grid exports metrics from each cache, so that you can determine a cache's status. A state transfer exposes attributes as properties, so that Data Grid can export metrics from each cache.



NOTE

You cannot perform a state transfer operation in invalidation mode.

Data Grid generates time metrics that are compatible with the REST API and the JMX API.

Prerequisites

- Configure Data Grid metrics.
- Enable metrics for your cache type, such as embedded cache or remote cache.
- Initiate a state transfer operation by changing your clustered cache topology.

Procedure

- Choose one of the following methods:
  - Configure Data Grid to use the REST API to collect metrics.
  - Configure Data Grid to use the JMX API to collect metrics.

Additional resources

- Enabling and configuring Data Grid statistics and JMX monitoring (Data Grid caches)
- **StateTransferManager** (Data Grid 16.0 API)

9.6. MONITORING THE STATUS OF CROSS-SITE REPLICATION

Monitor the site status of your backup locations to detect interruptions in the communication between the sites. When a remote site status changes to **offline**, Data Grid stops replicating your data to the backup location. Your data become out of sync and you must fix the inconsistencies before bringing the clusters back online.

Monitoring cross-site events is necessary for early problem detection. Use one of the following monitoring strategies:

- Monitoring cross-site replication with the REST API
- Monitoring cross-site replication with the Prometheus metrics or any other monitoring system

Monitoring cross-site replication with the REST API

Monitor the status of cross-site replication for all caches using the REST endpoint. You can implement a custom script to poll the REST endpoint or use the following example.

Prerequisites

- Enable cross-site replication.

Procedure

1. Implement a script to poll the REST endpoint.
   The following example demonstrates how you can use a Python script to poll the site status every five seconds.

```
#!/usr/bin/python3
import time
import requests
```

```
from requests.auth import HTTPDigestAuth


class InfinispanConnection:

    def __init__(self, server: str = 'http://localhost:11222', cache_manager: str = 'default',
                 auth: tuple = ('admin', 'change_me')) -> None:
        super().__init__()
        self.__url = f'{server}/rest/v2/container/x-site/backups/'
        self.__auth = auth
        self.__headers = {
            'accept': 'application/json'
        }

    def get_sites_status(self):
        try:
            rsp = requests.get(self.__url, headers=self.__headers, auth=HTTPDigestAuth(self.__auth[0],
self.__auth[1]))
            if rsp.status_code != 200:
                return None
            return rsp.json()
        except:
            return None


# Specify credentials for Data Grid user with permission to access the REST endpoint
USERNAME = 'admin'
PASSWORD = 'change_me'
# Set an interval between cross-site status checks
POLL_INTERVAL_SEC = 5
# Provide a list of servers
SERVERS = [
    InfinispanConnection('http://127.0.0.1:11222', auth=(USERNAME, PASSWORD)),
    InfinispanConnection('http://127.0.0.1:12222', auth=(USERNAME, PASSWORD))
]
#Specify the names of remote sites
REMOTE_SITES = [
    'nyc'
]
#Provide a list of caches to monitor
CACHES = [
    'work',
    'sessions'
]


def on_event(site: str, cache: str, old_status: str, new_status: str):
    # TODO implement your handling code here
    print(f'site={site} cache={cache} Status changed {old_status} -> {new_status}')


def __handle_mixed_state(state: dict, site: str, site_status: dict):
    if site not in state:
        state[site] = {c: 'online' if c in site_status['online'] else 'offline' for c in CACHES}
        return
```

```
    for cache in CACHES:
        __update_cache_state(state, site, cache, 'online' if cache in site_status['online'] else 'offline')


def __handle_online_or_offline_state(state: dict, site: str, new_status: str):
    if site not in state:
        state[site] = {c: new_status for c in CACHES}
        return

    for cache in CACHES:
        __update_cache_state(state, site, cache, new_status)


def __update_cache_state(state: dict, site: str, cache: str, new_status: str):
    old_status = state[site].get(cache)
    if old_status != new_status:
        on_event(site, cache, old_status, new_status)
        state[site][cache] = new_status


def update_state(state: dict):
    rsp = None
    for conn in SERVERS:
        rsp = conn.get_sites_status()
        if rsp:
            break
    if rsp is None:
        print('Unable to fetch site status from any server')
        return

    for site in REMOTE_SITES:
        site_status = rsp.get(site, {})
        new_status = site_status.get('status')
        if new_status == 'mixed':
            __handle_mixed_state(state, site, site_status)
        else:
            __handle_online_or_offline_state(state, site, new_status)


if __name__ == '__main__':
    _state = {}
    while True:
        update_state(_state)
        time.sleep(POLL_INTERVAL_SEC)
```

When a site status changes from **online** to **offline** or vice-versa, the function **on_event** is invoked.

If you want to use this script, you must specify the following variables:

- **USERNAME** and **PASSWORD**: The username and password of Data Grid user with permission to access the REST endpoint.
- **POLL_INTERVAL_SEC**: The number of seconds between polls.
- **SERVERS**: The list of Data Grid Servers at this site. The script only requires a single valid response but the list is provided to allow fail over.

- **REMOTE_SITES**: The list of remote sites to monitor on these servers.
- **CACHES**: The list of cache names to monitor.

Additional resources

- REST API: Getting status of backup locations

Monitoring cross-site replication with the Prometheus metrics

Prometheus, and other monitoring systems, let you configure alerts to detect when a site status changes to **offline**.

TIP

Monitoring cross-site latency metrics can help you to discover potential issues.

Prerequisites

- Enable cross-site replication.

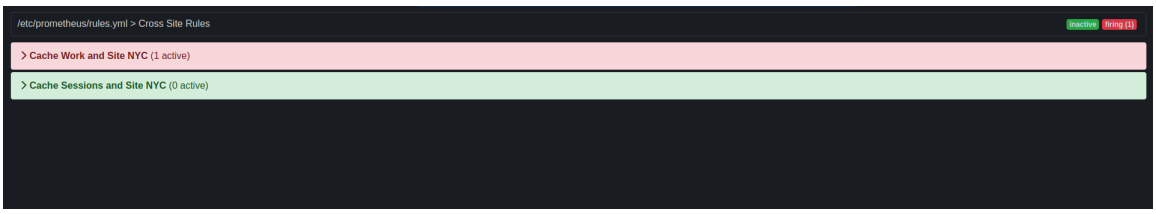
Procedure

1. Configure Data Grid metrics.
2. Configure alerting rules using the Prometheus metrics format.
   - For the site status, use **1** for **online** and **0** for **offline**.
   - For the **expr** filed, use the following format:
     infinispan_x_site_admin_status{cache=\"<cache name>\",site=\"<site name>\"}.
     In the following example, Prometheus alerts you when the **NYC** site gets **offline** for cache named **work** or **sessions**.

```
groups:
- name: Cross Site Rules
  rules:
  - alert: Cache Work and Site NYC
    expr: infinispan_x_site_admin_status{cache=\"Work\",site=\"NYC\"} == 0
  - alert: Cache Sessions and Site NYC
    expr: infinispan_x_site_admin_status{cache=\"Sessions\",site=\"NYC\"} == 0
```

The following image shows an alert that the **NYC** site is **offline** for cache **work**.

Figure 9.1. Prometheus Alert



Additional resources

- Configuring Data Grid metrics

- Prometheus Alerting Overview
- Grafana Alerting Documentation
- Openshift Managing Alerts

CHAPTER 10. ENABLING AND CONFIGURING DATA GRID OPENTELEMETRY TRACING

Data Grid generates tracing spans compatible with the OpenTelemetry standard, allowing you to export, visualize, and analyze tracing data related to the most important cache operations.

10.1. CONFIGURING DATA GRID TRACING

Configure OpenTelemetry tracing, to enable monitoring and tracing of cache operations.

Procedure

1. Open your Data Grid configuration for editing.
2. Add the **tracing** element or object to the cache container.
3. Define the endpoint URL of the OpenTelemetry collector with the **collector_endpoint** attribute or field. It is mandatory to enable tracing. **4318** is typically the **http/protobuf** OTLP standard port.
4. Enable or disable tracing globally with the **enable** attribute or field.
5. Enable or disable security event tracing with the **security** attribute or field.
6. Optionally change the tracing exporter protocol changing the **exporter_protocol** attribute or field. By default, it is **otlp** (OpenTelemetry protocol).
7. Optionally change the tracing service name associated with the generated tracing span changing the **service_name** attribute or field. By default, it is **infinispan-server**.
8. Save and close your client configuration.

Next steps

To apply any global tracing configuration changes, stop the server and repeat the procedure.

Tracing configuration

Data Grid applies the tracing configuration globally to all caches.

XML

```
<infinispan>
  <cache-container statistics="true">
    <tracing collector-endpoint="http://localhost:4318"
             enabled="true"
             exporter-protocol="OTLP"
             service-name="infinispan-server"
             security="false" />
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan" : {
    "cache-container" : {
      "statistics" : true,
      "tracing" : {
        "collector-endpoint" : "http://localhost:4318",
        "enabled" : true,
        "exporter-protocol" : "OTLP",
        "service-name" : "infinispan-server",
        "security" : false
      }
    }
  }
}
```

YAML

```
infinispan:
  cacheContainer:
    statistics: true
    tracing:
      collector-endpoint: "http://localhost:4318"
      enabled: true
      exporter-protocol: "OTLP"
      service-name: "infinispan-server"
      security: false
```

10.1.1. Further Tracing Options

To configure further tracing options it is possible to pass system properties or to set environment variables to the Data Grid server at startup to configure directly the OpenTelemetry SDK Autoconfigure that is used by the Data Grid server to configure the OpenTelemetry tracing.

Procedure

- Pass the system properties to Data Grid Server at startup.
  Use **-D<property-name>=<property-value>** arguments like in the following example:

  ```
  bin/server.sh -Dotel.exporter.otlp.timeout=10000
  ```

Additional resources

- OpenTelemetry SDK Autoconfigure

Tracing data format

The Data Grid Server, by default, exports tracing data using the OTLP **http/protobuf** protocol.

tracing.properties

```
otel.exporter.otlp.protocol = http/protobuf
```

To use a different protocol, you must copy JAR files or dependencies to the **$ISPN_HOME/server/lib** directory of your Data Grid Server installation.

10.2. CONFIGURE TRACING AT CACHE LEVEL

Once the tracing is configured at server level, it will be automatically enabled by default for all caches. A cache configuration level of tracing allows on the other hand to enable or disable it at cache level and at runtime.

Tracing categories

Several categories are potentially traced:

- Container. That are all the main cache operations, such as replace, put, clear, getForReplace, remove operations and size. With the exception of all the getting operation.
- Cluster. Operations that are replicated to another node in the same cluster.
- X-Site. Operations that are replicated to another external site.
- Persistence. All the operations involving persistence via a cache store and/or cache loader.

Each of category can be enabled/disabled at start time or runtime listing them in the **categories** list attribute. By default only the container category is enabled.

There is also the **Security** category, to trace security audit events. This category is configured globally, not only at cache level, since their events can have different scopes (cache, container or server), not only cache scope.

Enable/disable tracing for a given cache

XML

```
<replicated-cache>
   <tracing enabled="true" categories="container cluster x-site persistence" />
</replicated-cache>
```

JSON

```
{
  "distributed-cache": {
    "tracing": {
      "enabled" : true,
      "categories": [
        "container",
        "cluster",
        "x-site",
        "persistence"
      ]
    }
  }
}
```

YAML

```
distributedCache:
  tracing:
    enabled: true
    categories:
      - "container"
      - "cluster"
      - "x-site"
      - "persistence"
```

Enable/disable tracing at runtime

The cache-level tracing attribute **enable** is a mutable attribute, it means it can be changed at runtime without the need to restart the Infinispan cluster.

To change a mutable attribute both HotRod and REST APIs can be used.

HotRod

```
remoteCacheManager.administration()
   .updateConfigurationAttribute(CACHE_A, "tracing.enabled", "false");
```

REST

```
restClient.cache(CACHE_A)
   .updateConfigurationAttribute("tracing.enabled", "false");
```

Additional resources

- Updating cache configuration attributes

CHAPTER 11. ADDING MANAGED DATASOURCES TO DATA GRID SERVER

Optimize connection pooling and performance for JDBC database connections by adding managed datasources to your Data Grid Server configuration.

11.1. CONFIGURING MANAGED DATASOURCES

Create managed datasources as part of your Data Grid Server configuration to optimize connection pooling and performance for JDBC database connections. You can then specify the JDNI name of the managed datasources in your caches, which centralizes JDBC connection configuration for your deployment.

Prerequisites

- Copy database drivers to the **server/lib** directory in your Data Grid Server installation.

  #### TIP

  Use the **install** command with the Data Grid Command Line Interface (CLI) to download the required drivers to the **server/lib** directory, for example:

  ```
  install org.postgresql:postgresql:42.4.3
  ```

Procedure

1. Open your Data Grid Server configuration for editing.
2. Add a new **data-source** to the **data-sources** section.
3. Uniquely identify the datasource with the **name** attribute or field.
4. Specify a JNDI name for the datasource with the **jndi-name** attribute or field.

   #### TIP

   You use the JNDI name to specify the datasource in your JDBC cache store configuration.

5. Set **true** as the value of the **statistics** attribute or field to enable statistics for the datasource through the /**metrics** endpoint.
6. Provide JDBC driver details that define how to connect to the datasource in the **connection-factory** section.
   a. Specify the name of the database driver with the **driver** attribute or field.
   b. Specify the JDBC connection url with the **url** attribute or field.
   c. Specify credentials with the **username** and **password** attributes or fields.
   d. Provide any other configuration as appropriate.

7. Define how Data Grid Server nodes pool and reuse connections with connection pool tuning properties in the **connection-pool** section.

8. Save the changes to your configuration.

Verification

Use the Data Grid Command Line Interface (CLI) to test the datasource connection, as follows:

1. Start a CLI session.

```
bin/cli.sh
```

2. List all datasources and confirm the one you created is available.

```
server datasource ls
```

3. Test a datasource connection.

```
server datasource test my-datasource
```

Managed datasource configuration

XML

```
<server xmlns="urn:infinispan:server:16.0">
  <data-sources>
    <!-- Defines a unique name for the datasource and JNDI name that you
         reference in JDBC cache store configuration.
         Enables statistics for the datasource, if required. -->
    <data-source name="ds"
                 jndi-name="jdbc/postgres"
                 statistics="true">
      <!-- Specifies the JDBC driver that creates connections. -->
      <connection-factory driver="org.postgresql.Driver"
                          url="jdbc:postgresql://localhost:5432/postgres"
                          username="postgres"
                          password="changeme">
        <!-- Sets optional JDBC driver-specific connection properties. -->
        <connection-property name="name">value</connection-property>
      </connection-factory>
      <!-- Defines connection pool tuning properties. -->
      <connection-pool initial-size="1"
                       max-size="10"
                       min-size="3"
                       background-validation="1s"
                       idle-removal="1m"
                       blocking-timeout="1s"
                       leak-detection="10s"/>
    </data-source>
  </data-sources>
</server>
```

JSON

```
{
  "server": {
    "data-sources": [{
      "name": "ds",
      "jndi-name": "jdbc/postgres",
      "statistics": true,
      "connection-factory": {
        "driver": "org.postgresql.Driver",
        "url": "jdbc:postgresql://localhost:5432/postgres",
        "username": "postgres",
        "password": "changeme",
        "connection-properties": {
          "name": "value"
        }
      },
      "connection-pool": {
        "initial-size": 1,
        "max-size": 10,
        "min-size": 3,
        "background-validation": 1000,
        "idle-removal": 1,
        "blocking-timeout": 1000,
        "leak-detection": 10000
      }
    }]
  }
}
```

YAML

```
server:
  dataSources:
    - name: ds
      jndiName: 'jdbc/postgres'
      statistics: true
      connectionFactory:
        driver: "org.postgresql.Driver"
        url: "jdbc:postgresql://localhost:5432/postgres"
        username: "postgres"
        password: "changeme"
        connectionProperties:
          name: value
      connectionPool:
        initialSize: 1
        maxSize: 10
        minSize: 3
        backgroundValidation: 1000
        idleRemoval: 1
        blockingTimeout: 1000
        leakDetection: 10000
```

11.2. CONFIGURING CACHES WITH JNDI NAMES

When you add a managed datasource to Data Grid Server you can add the JNDI name to a JDBC-based cache store configuration.

Prerequisites

- Configure Data Grid Server with a managed datasource.

Procedure

1. Open your cache configuration for editing.
2. Add the **data-source** element or field to the JDBC-based cache store configuration.
3. Specify the JNDI name of the managed datasource as the value of the **jndi-url** attribute.
4. Configure the JDBC-based cache stores as appropriate.
5. Save the changes to your configuration.

JNDI name in cache configuration

XML

```
<distributed-cache>
  <persistence>
    <jdbc:string-keyed-jdbc-store>
      <!-- Specifies the JNDI name of a managed datasource on Data Grid Server. -->
      <jdbc:data-source jndi-url="jdbc/postgres"/>
      <jdbc:string-keyed-table drop-on-exit="true" create-on-start="true" prefix="TBL">
        <jdbc:id-column name="ID" type="VARCHAR(255)"/>
        <jdbc:data-column name="DATA" type="BYTEA"/>
        <jdbc:timestamp-column name="TS" type="BIGINT"/>
        <jdbc:segment-column name="S" type="INT"/>
      </jdbc:string-keyed-table>
    </jdbc:string-keyed-jdbc-store>
  </persistence>
</distributed-cache>
```

JSON

```
{
  "distributed-cache": {
    "persistence": {
      "string-keyed-jdbc-store": {
        "data-source": {
          "jndi-url": "jdbc/postgres"
          },
        "string-keyed-table": {
          "prefix": "TBL",
          "drop-on-exit": true,
          "create-on-start": true,
          "id-column": {
            "name": "ID",
            "type": "VARCHAR(255)"
```

```
          },
          "data-column": {
            "name": "DATA",
            "type": "BYTEA"
          },
          "timestamp-column": {
            "name": "TS",
            "type": "BIGINT"
          },
          "segment-column": {
            "name": "S",
            "type": "INT"
          }
        }
      }
    }
  }
}
```

YAML

```
distributedCache:
  persistence:
    stringKeyedJdbcStore:
      dataSource:
        jndi-url: "jdbc/postgres"
      stringKeyedTable:
        prefix: "TBL"
        dropOnExit: true
        createOnStart: true
        idColumn:
          name: "ID"
          type: "VARCHAR(255)"
        dataColumn:
          name: "DATA"
          type: "BYTEA"
        timestampColumn:
          name: "TS"
          type: "BIGINT"
        segmentColumn:
          name: "S"
          type: "INT"
```

11.3. CONNECTION POOL TUNING PROPERTIES

You can tune JDBC connection pools for managed datasources in your Data Grid Server configuration.

Property	Description
initial-size	Initial number of connections the pool should hold.
max-size	Maximum number of connections in the pool.

Property	Description
min-size	Minimum number of connections the pool should hold.
blocking-timeout	Maximum time in milliseconds to block while waiting for a connection before throwing an exception. This will never throw an exception if creating a new connection takes an inordinately long period of time. Default is **0** meaning that a call will wait indefinitely.
background-validation	Time in milliseconds between background validation runs. A duration of **0** means that this feature is disabled.
validate-on-acquisition	Connections idle for longer than this time, specified in milliseconds, are validated before being acquired (foreground validation). A duration of **0** means that this feature is disabled.
idle-removal	Time in minutes a connection has to be idle before it can be removed.
leak-detection	Time in milliseconds a connection has to be held before a leak warning.

CHAPTER 12. SETTING UP DATA GRID CLUSTER TRANSPORT

Data Grid requires a transport layer so nodes can automatically join and leave clusters. The transport layer also enables Data Grid nodes to replicate or distribute data across the network and perform operations such as re-balancing and state transfer.

12.1. DEFAULT JGROUPS STACKS

Data Grid provides default JGroups stack files, **default-jgroups-*.xml**, in the **default-configs** directory inside the **infinispan-core-16.0.6.Final-redhat-00001.jar** file.

You can find this JAR file in the **$RHDG_HOME/lib** directory.

File name	Stack name	Description
default-jgroups-udp.xml	**udp**	Uses UDP for transport and UDP multicast for discovery. Suitable for larger clusters (over 100 nodes) or if you are using replicated caches or invalidation mode. Minimizes the number of open sockets.
default-jgroups-tcp.xml	**tcp**	Uses TCP for transport and the **MPING** protocol for discovery, which uses **UDP** multicast. Suitable for smaller clusters (under 100 nodes) *only if* you are using distributed caches because TCP is more efficient than UDP as a point-to-point protocol.
default-jgroups-kubernetes.xml	**kubernetes**	Uses TCP for transport and **DNS_PING** for discovery. Suitable for Kubernetes and Red Hat OpenShift nodes where UDP multicast is not always available.
default-jgroups-ec2.xml	**ec2**	Uses TCP for transport and **aws.S3_PING** for discovery. Suitable for Amazon EC2 nodes where UDP multicast is not available. Requires additional dependencies.
default-jgroups-google.xml	**google**	Uses TCP for transport and **GOOGLE_PING2** for discovery. Suitable for Google Cloud Platform nodes where UDP multicast is not available. Requires additional dependencies.
default-jgroups-azure.xml	**azure**	Uses TCP for transport and **AZURE_PING** for discovery. Suitable for Microsoft Azure nodes where UDP multicast is not available. Requires additional dependencies.

File name	Stack name	Description
default-jgroups-tunnel.xml	**tunnel**	Uses **TUNNEL** for transport. Suitable for environments where the Data Grid is behind a firewall and direct connection between Data Grid nodes is impossible. It requires an external and accessible service (Gossip Router) to redirect the traffic. It requires **jgroups.tunnel.hosts** property to be set in the format **host1[port],host2[port],...** with the Gossip Router(s) hosts and ports.

Additional resources

- JGroups Protocols

12.2. CLUSTER DISCOVERY PROTOCOLS

Data Grid supports different protocols that allow nodes to automatically find each other on the network and form clusters.

There are two types of discovery mechanisms that Data Grid can use:

- Generic discovery protocols that work on most networks and do not rely on external services.
- Discovery protocols that rely on external services to store and retrieve topology information for Data Grid clusters.
  For instance the DNS_PING protocol performs discovery through DNS server records.



NOTE

Running Data Grid on hosted platforms requires using discovery mechanisms that are adapted to network constraints that individual cloud providers impose.

Additional resources

- JGroups Discovery Protocols
- JGroups cluster transport configuration for Data Grid 8.x (Red Hat knowledgebase article)

12.2.1. PING

PING, or UDPPING is a generic JGroups discovery mechanism that uses dynamic multicasting with the UDP protocol.

When joining, nodes send PING requests to an IP multicast address to discover other nodes already in the Data Grid cluster. Each node responds to the PING request with a packet that contains the address of the coordinator node and its own address. C=coordinator's address and A=own address. If no nodes respond to the PING request, the joining node becomes the coordinator node in a new cluster.

PING configuration example

```
<PING num_discovery_runs="3"/>
```

Additional resources

- JGroups PING

12.2.2. TCPPING

TCPPING is a generic JGroups discovery mechanism that uses a list of static addresses for cluster members.

With TCPPING, you manually specify the IP address or hostname of each node in the Data Grid cluster as part of the JGroups stack, rather than letting nodes discover each other dynamically.

TCPPING configuration example

```
<TCP bind_port="7800" />
<TCPPING timeout="3000"
         initial_hosts="${jgroups.tcpping.initial_hosts:hostname1[port1],hostname2[port2]}"
         port_range="0"
         num_initial_members="3"/>
```

Additional resources

- JGroups TCPPING

12.2.3. MPING

MPING uses IP multicast to discover the initial membership of Data Grid clusters.

You can use MPING to replace TCPPING discovery with TCP stacks and use multicasing for discovery instead of static lists of initial hosts. However, you can also use MPING with UDP stacks.

MPING configuration example

```
<MPING mcast_addr="${jgroups.mcast_addr:239.6.7.8}"
       mcast_port="${jgroups.mcast_port:46655}"
       num_discovery_runs="3"
       ip_ttl="${jgroups.udp.ip_ttl:2}"/>
```

Additional resources

- JGroups MPING

12.2.4. TCPGOSSIP

Gossip routers provide a centralized location on the network from which your Data Grid cluster can retrieve addresses of other nodes.

You inject the address (**IP:PORT**) of the Gossip router into Data Grid nodes as follows:

1. Pass the address as a system property to the JVM; for example, **-DGossipRouterAddress="10.10.2.4[12001]"**.
2. Reference that system property in the JGroups configuration file.

Gossip router configuration example

```
<TCP bind_port="7800" />
<TCPGOSSIP timeout="3000"
        initial_hosts="${GossipRouterAddress}"
        num_initial_members="3" />
```

Additional resources

- JGroups Gossip Router

12.2.5. JDBC_PING2

JDBC_PING2 uses shared databases to store information about Data Grid clusters. This protocol supports any database that can use a JDBC connection.

Nodes write their IP addresses to the shared database so joining nodes can find the Data Grid cluster on the network. When nodes leave Data Grid clusters, they delete their IP addresses from the shared database.

JDBC_PING2 configuration example

```
<JDBC_PING2 connection_url="jdbc:mysql://localhost:3306/database_name"
        connection_username="user"
        connection_password="password"
        connection_driver="com.mysql.jdbc.Driver"/>
```



IMPORTANT

Add the appropriate JDBC driver to the classpath so Data Grid can use JDBC_PING2.

12.2.5.1. Using a server datasource for JDBC_PING2 discovery

Add a managed datasource to a Data Grid Server and use it to provide database connections for the cluster transport JDBC_PING2 discovery protocol.

Prerequisites

- Install a Data Grid Server cluster.

Procedure

1. Deploy a JDBC driver JAR to your Data Grid Server **server/lib** directory
2. Create a datasource for your database.

```
<server xmlns="urn:infinispan:server:16.0">
  <data-sources>
    <!-- Defines a unique name for the datasource and JNDI name that you
         reference in JDBC cache store configuration.
         Enables statistics for the datasource, if required. -->
    <data-source name="ds"
                 jndi-name="jdbc/postgres"
```

```
            statistics="true">
        <!-- Specifies the JDBC driver that creates connections. -->
        <connection-factory driver="org.postgresql.Driver"
                            url="jdbc:postgresql://localhost:5432/postgres"
                            username="postgres"
                            password="changeme">
          <!-- Sets optional JDBC driver-specific connection properties. -->
          <connection-property name="name">value</connection-property>
        </connection-factory>
        <!-- Defines connection pool tuning properties. -->
        <connection-pool initial-size="1"
                         max-size="10"
                         min-size="3"
                         background-validation="1s"
                         idle-removal="1m"
                         blocking-timeout="1s"
                         leak-detection="10s"/>
      </data-source>
    </data-sources>
  </server>
```

3. Create a JGroups stack which uses the **JDBC_PING2** protocol for discovery.

4. Configure cluster transport to use the datasource by specifying the name of the datasource with the **server:data-source** attribute.

```
<infinispan>
  <jgroups>
    <stack name="jdbc" extends="tcp">
      <JDBC_PING2 stack.combine="REPLACE" stack.position="MPING" />
    </stack>
  </jgroups>
  <cache-container>
    <transport stack="jdbc" server:data-source="ds" />
  </cache-container>
</infinispan>
```

Additional resources

- JDBC_PING2 documentation

12.2.6. DNS_PING

JGroups DNS_PING queries DNS servers to discover Data Grid cluster members in Kubernetes environments such as OKD and Red Hat OpenShift.

DNS_PING configuration example

```
<dns.DNS_PING dns_query="myservice.myproject.svc.cluster.local" />
```

Additional resources

- JGroups DNS_PING

- DNS for Services and Pods (Kubernetes documentation for adding DNS entries)

12.2.7. Cloud discovery protocols

Data Grid includes default JGroups stacks that use discovery protocol implementations that are specific to cloud providers.

Discovery protocol	Default stack file	Artifact	Version
aws.S3_PING	**default-jgroups-ec2.xml**	**org.jgroups.aws:jgroups-aws**	**3.0.0.Final**
GOOGLE_PING2	**default-jgroups-google.xml**	**org.jgroups.google:jgroups-google**	**2.0.0.Final**
azure.AZURE_PING	**default-jgroups-azure.xml**	**org.jgroups.azure:jgroups-azure**	**2.0.2.Final**

Providing dependencies for cloud discovery protocols
To use **aws.S3_PING**, **GOOGLE_PING2**, or **azure.AZURE_PING** cloud discovery protocols, you need to provide dependent libraries to Data Grid.

Procedure

1. Download the artifact JAR file and all dependencies.
2. Add the artifact JAR file and all dependencies to the **$RHDG_HOME/server/lib** directory of your Data Grid Server installation.
   For more details see the Downloading artifacts for JGroups cloud discover protocols for Data Grid Server (Red Hat knowledgebase article)

You can then configure the cloud discovery protocol as part of a JGroups stack file or with system properties.

Additional resources

- JGroups aws.S3_PING
- JGroups GOOGLE_PING2
- JGroups azure.AZURE_PING

12.3. USING THE DEFAULT JGROUPS STACKS

Data Grid uses JGroups protocol stacks so nodes can send each other messages on dedicated cluster channels.

Data Grid provides preconfigured JGroups stacks for **UDP** and **TCP** protocols. You can use these default stacks as a starting point for building custom cluster transport configuration that is optimized for your network requirements.

Procedure

Do one of the following to use one of the default JGroups stacks:

- Use the **stack** attribute in your **infinispan.xml** file.

```
<infinispan>
  <cache-container default-cache="replicatedCache">
    <!-- Use the default UDP stack for cluster transport. -->
    <transport cluster="${infinispan.cluster.name}"
               stack="udp"
               node-name="${infinispan.node.name:}"/>
  </cache-container>
</infinispan>
```

- Use the **cluster-stack** argument to set the JGroups stack file when Data Grid Server starts:

```
bin/server.sh --cluster-stack=udp
```

Verification

Data Grid logs the following message to indicate which stack it uses:

```
[org.infinispan.CLUSTER] ISPN000078: Starting JGroups channel cluster with stack udp
```

Additional resources

- JGroups cluster transport configuration for Data Grid 8.x (Red Hat knowledgebase article)

12.4. CUSTOMIZING JGROUPS STACKS

Adjust and tune properties to create a cluster transport configuration that works for your network requirements.

Data Grid provides attributes that let you extend the default JGroups stacks for easier configuration. You can inherit properties from the default stacks while combining, removing, and replacing other properties.

Procedure

1. Create a new JGroups stack declaration in your **infinispan.xml** file.
2. Add the **extends** attribute and specify a JGroups stack to inherit properties from.
3. Use the **stack.combine** attribute to modify properties for protocols configured in the inherited stack.
4. Use the **stack.position** attribute to define the location for your custom stack.
5. Specify the stack name as the value for the **stack** attribute in the **transport** configuration.
   For example, you might evaluate using a Gossip router and symmetric encryption with the default TCP stack as follows:

```
<infinispan>
  <jgroups>
    <!-- Creates a custom JGroups stack named "my-stack". -->
```

```
    <!-- Inherits properties from the default TCP stack. -->
    <stack name="my-stack" extends="tcp">
      <!-- Uses TCPGOSSIP as the discovery mechanism instead of MPING -->
      <TCPGOSSIP initial_hosts="${jgroups.tunnel.gossip_router_hosts:localhost[12001]}"
            stack.combine="REPLACE"
            stack.position="MPING" />
      <!-- Removes the FD_SOCK2 protocol from the stack. -->
      <FD_SOCK2 stack.combine="REMOVE"/>
      <!-- Modifies the timeout value for the VERIFY_SUSPECT2 protocol. -->
      <VERIFY_SUSPECT2 timeout="2s"/>
      <!-- Adds SYM_ENCRYPT to the stack after VERIFY_SUSPECT2. -->
      <SYM_ENCRYPT sym_algorithm="AES"
                   keystore_name="mykeystore.p12"
                   keystore_type="PKCS12"
                   store_password="changeit"
                   key_password="changeit"
                   alias="myKey"
                   stack.combine="INSERT_AFTER"
                   stack.position="VERIFY_SUSPECT2" />
    </stack>
  </jgroups>
  <cache-container name="default" statistics="true">
    <!-- Uses "my-stack" for cluster transport. -->
    <transport cluster="${infinispan.cluster.name}"
               stack="my-stack"
               node-name="${infinispan.node.name:}"/>
  </cache-container>
</infinispan>
```

6. Check Data Grid logs to ensure it uses the stack.

```
[org.infinispan.CLUSTER] ISPN000078: Starting JGroups channel cluster with stack my-stack
```

Reference

- JGroups cluster transport configuration for Data Grid 8.x (Red Hat knowledgebase article)

12.4.1. Inheritance attributes

When you extend a JGroups stack, inheritance attributes let you adjust protocols and properties in the stack you are extending.

- **stack.position** specifies protocols to modify.
- **stack.combine** uses the following values to extend JGroups stacks:

Value	Description
COMBINE	Overrides protocol properties.
REPLACE	Replaces protocols.

Value	Description
INSERT_AFTER	Adds a protocol into the stack after another protocol. Does not affect the protocol that you specify as the insertion point. Protocols in JGroups stacks affect each other based on their location in the stack. For example, you should put a protocol such as **NAKACK2** after the **SYM_ENCRYPT** or **ASYM_ENCRYPT** protocol so that **NAKACK2** is secured.
INSERT_BEFORE	Inserts a protocols into the stack before another protocol. Affects the protocol that you specify as the insertion point.
REMOVE	Removes protocols from the stack.

12.5. USING JGROUPS SYSTEM PROPERTIES

Pass system properties to Data Grid at startup to tune cluster transport.

Procedure

- Use **-D<property-name>=<property-value>** arguments to set JGroups system properties as required.

For example, set a custom bind port and IP address as follows:

```
bin/server.sh -Djgroups.bind.port=1234 -Djgroups.bind.address=192.0.2.0
```

12.5.1. Cluster transport properties

Use the following properties to customize JGroups cluster transport.

System Property	Description	Default Value	Required/ Optional
jgroups.bind.address	Bind address for cluster transport.	**SITE_LOCAL**	Optional
jgroups.bind.port	Bind port for the socket.	**7800**	Optional
jgroups.mcast_addr	IP address for multicast when using UDP transport, both discovery and inter-cluster communication. The IP address must be a valid "class D" address that is suitable for IP multicast.	**239.6.7.8**	Optional

System Property	Description	Default Value	Required/ Optional
jgroups.mcast_port	Port for the multicast socket when using UDP transport.	**46655**	Optional
jgroups.ip_ttl	Time-to-live (TTL) for IP multicast packets when using UDP transport. The value defines the number of network hops a packet can make before it is dropped.	2	Optional
jgroups.thread_pool.min_threads	Minimum number of threads for the thread pool.	0	Optional
jgroups.thread_pool.max_threads	Maximum number of threads for the thread pool.	200	Optional
jgroups.join_timeout	Maximum number of milliseconds to wait for join requests to succeed.	2000	Optional
jgroups.thread_dumps_enabled"	Dump threads when the thread pool is full.	**false**	Optional
jgroups.fd.port-offset	Offset from **jgroups.bind.port** port for the FD (failure detection protocol) socket when using UDP transport.	**50000** (port **57800**)	Optional
jgroups.frag_size	Maximum number of bytes in a message. Messages larger than that are fragmented.	60000	Optional
jgroups.diag.enabled	Enables JGroups diagnostic probing.	false	Optional

Additional resources

- JGroups system properties
- JGroups protocol list

12.5.2. System properties for cloud discovery protocols

Use the following properties to configure JGroups discovery protocols for hosted platforms.

12.5.2.1. Amazon EC2

System properties for configuring **aws.S3_PING**.

System Property	Description	Default Value	Required/ Optional
jgroups.s3.region_name	Name of the Amazon S3 region.	No default value.	Optional
jgroups.s3.bucket_name	Name of the Amazon S3 bucket. The name must exist and be unique.	No default value.	Optional

12.5.2.2. Google Cloud Platform

System properties for configuring **GOOGLE_PING2**.

System Property	Description	Default Value	Required/ Optional
jgroups.google.bucket_name	Name of the Google Compute Engine bucket. The name must exist and be unique.	No default value.	Required

12.5.2.3. Azure

System properties for azure.AZURE_PING`.

System Property	Description	Default Value	Required/ Optional
jboss.jgroups.azure_ping.storage_account_name	Name of the Azure storage account. The name must exist and be unique.	No default value.	Required
jboss.jgroups.azure_ping.storage_access_key	Name of the Azure storage access key.	No default value.	Required

System Property	Description	Default Value	Required/ Optional
jboss.jgroups.azure_ping.container	Valid DNS name of the container that stores ping information.	No default value.	Required

12.5.2.4. OpenShift

System properties for **DNS_PING**.

System Property	Description	Default Value	Required/ Optional
jgroups.dns.query	Sets the DNS record that returns cluster members.	No default value.	Required
jgroups.dns.record	Sets the DNS record type.	A	Optional

12.6. USING INLINE JGROUPS STACKS

You can insert complete JGroups stack definitions into **infinispan.xml** files.

Procedure

- Embed a custom JGroups stack declaration in your **infinispan.xml** file.

```
<infinispan>
  <!-- Contains one or more JGroups stack definitions. -->
  <jgroups>
    <!-- Defines a custom JGroups stack named "prod". -->
    <stack name="prod">
      <TCP bind_port="7800" port_range="30" recv_buf_size="20000000"
send_buf_size="640000"/>
      <RED/>
      <MPING break_on_coord_rsp="true"
             mcast_addr="${jgroups.mping.mcast_addr:239.2.4.6}"
             mcast_port="${jgroups.mping.mcast_port:43366}"
             num_discovery_runs="3"
             ip_ttl="${jgroups.udp.ip_ttl:2}"/>
      <MERGE3 />
      <FD_SOCK2 />
      <FD_ALL3 timeout="3000" interval="1000" timeout_check_interval="1000" />
      <VERIFY_SUSPECT2 timeout="1s" />
      <pbcast.NAKACK2 use_mcast_xmit="false" xmit_interval="200ms"
xmit_table_num_rows="50"
```

```
                xmit_table_msgs_per_row="1024" xmit_table_max_compaction_time="30s" />
    <UNICAST3 conn_close_timeout="5s" xmit_interval="200ms" xmit_table_num_rows="50"
            xmit_table_msgs_per_row="1024" xmit_table_max_compaction_time="30s" />
    <pbcast.STABLE desired_avg_gossip="2s" max_bytes="1M" />
    <pbcast.GMS print_local_addr="false" join_timeout="${jgroups.join_timeout:2s}" />
    <UFC max_credits="4m" min_threshold="0.40" />
    <MFC max_credits="4m" min_threshold="0.40" />
    <FRAG4 />
  </stack>
 </jgroups>
 <cache-container default-cache="replicatedCache">
  <!-- Uses "prod" for cluster transport. -->
  <transport cluster="${infinispan.cluster.name}"
        stack="prod"
        node-name="${infinispan.node.name:}"/>
 </cache-container>
</infinispan>
```

12.7. USING EXTERNAL JGROUPS STACKS

Reference external files that define custom JGroups stacks in **infinispan.xml** files.

Procedure

1. Add custom JGroups stack files to the **$RHDG_HOME/server/conf** directory.
   Alternatively you can specify an absolute path when you declare the external stack file.
2. Reference the external stack file with the **stack-file** element.

```
<infinispan>
 <jgroups>
   <!-- Creates a "prod-tcp" stack that references an external file. -->
   <stack-file name="prod-tcp" path="prod-jgroups-tcp.xml"/>
 </jgroups>
 <cache-container default-cache="replicatedCache">
  <!-- Use the "prod-tcp" stack for cluster transport. -->
  <transport stack="prod-tcp" />
  <replicated-cache name="replicatedCache"/>
 </cache-container>
 <!-- Cache configuration goes here. -->
</infinispan>
```

12.8. ENCRYPTING CLUSTER TRANSPORT

Secure cluster transport so that nodes communicate with encrypted messages. You can also configure Data Grid clusters to perform certificate authentication so that only nodes with valid identities can join.

12.8.1. Securing cluster transport with TLS identities

Add SSL/TLS identities to a Data Grid Server security realm and use them to secure cluster transport. Nodes in the Data Grid Server cluster then exchange SSL/TLS certificates to encrypt JGroups messages, including RELAY messages if you configure cross-site replication.

Prerequisites

- Install a Data Grid Server cluster.

Procedure

1. Create a TLS keystore that contains a single certificate to identify Data Grid Server. You can also use a PEM file if it contains a private key in PKCS#1 or PKCS#8 format, a certificate, and has an empty password: **password=""**.

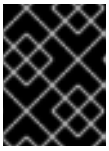


NOTE

If the certificate in the keystore is not signed by a public certificate authority (CA) then you must also create a trust store that contains either the signing certificate or the public key.

2. Add the keystore to the **$RHDG_HOME/server/conf** directory.

3. Add the keystore to a new security realm in your Data Grid Server configuration.



IMPORTANT

You should create dedicated keystores and security realms so that Data Grid Server endpoints do not use the same security realm as cluster transport.

```
<server xmlns="urn:infinispan:server:16.0">
  <security>
    <security-realms>
      <security-realm name="cluster-transport">
        <server-identities>
          <ssl>
            <!-- Adds a keystore that contains a certificate that provides SSL/TLS identity to
encrypt cluster transport. -->
            <keystore path="server.pfx"
                      relative-to="infinispan.server.config.path"
                      password="secret"
                      alias="server"/>
          </ssl>
        </server-identities>
      </security-realm>
    </security-realms>
  </security>
</server>
```

4. Configure cluster transport to use the security realm by specifying the name of the security realm with the **server:security-realm** attribute.

```
<infinispan>
  <cache-container>
    <transport server:security-realm="cluster-transport"/>
  </cache-container>
</infinispan>
```

Verification

When you start Data Grid Server, the following log message indicates that the cluster is using the security realm for cluster transport:

```
[org.infinispan.SERVER] ISPN080060: SSL Transport using realm <security_realm_name>
```

12.8.2. JGroups encryption protocols

To secure cluster traffic, you can configure Data Grid nodes to encrypt JGroups message payloads with secret keys.

Data Grid nodes can obtain secret keys from either:

- The coordinator node (asymmetric encryption).
- A shared keystore (symmetric encryption).

Retrieving secret keys from coordinator nodes

You configure asymmetric encryption by adding the **ASYM_ENCRYPT** protocol to a JGroups stack in your Data Grid configuration. This allows Data Grid clusters to generate and distribute secret keys.



IMPORTANT

When using asymmetric encryption, you should also provide keystores so that nodes can perform certificate authentication and securely exchange secret keys. This protects your cluster from man-in-the-middle (MitM) attacks.

Asymmetric encryption secures cluster traffic as follows:

1. The first node in the Data Grid cluster, the coordinator node, generates a secret key.
2. A joining node performs certificate authentication with the coordinator to mutually verify identity.
3. The joining node requests the secret key from the coordinator node. That request includes the public key for the joining node.
4. The coordinator node encrypts the secret key with the public key and returns it to the joining node.
5. The joining node decrypts and installs the secret key.
6. The node joins the cluster, encrypting and decrypting messages with the secret key.

Retrieving secret keys from shared keystores

You configure symmetric encryption by adding the **SYM_ENCRYPT** protocol to a JGroups stack in your Data Grid configuration. This allows Data Grid clusters to obtain secret keys from keystores that you provide.

1. Nodes install the secret key from a keystore on the Data Grid classpath at startup.
2. Node join clusters, encrypting and decrypting messages with the secret key.

Comparison of asymmetric and symmetric encryption

ASYM_ENCRYPT with certificate authentication provides an additional layer of encryption in comparison with **SYM_ENCRYPT**. You provide keystores that encrypt the requests to coordinator nodes for the secret key. Data Grid automatically generates that secret key and handles cluster traffic, while letting you specify when to generate secret keys. For example, you can configure clusters to generate new secret keys when nodes leave. This ensures that nodes cannot bypass certificate authentication and join with old keys.

SYM_ENCRYPT, on the other hand, is faster than **ASYM_ENCRYPT** because nodes do not need to exchange keys with the cluster coordinator. A potential drawback to **SYM_ENCRYPT** is that there is no configuration to automatically generate new secret keys when cluster membership changes. Users are responsible for generating and distributing the secret keys that nodes use to encrypt cluster traffic.

12.8.3. Securing cluster transport with asymmetric encryption

Configure Data Grid clusters to generate and distribute secret keys that encrypt JGroups messages.

Procedure

1. Create a keystore with certificate chains that enables Data Grid to verify node identity.
2. Place the keystore on the classpath for each node in the cluster.
   For Data Grid Server, you put the keystore in the $RHDG_HOME directory.
3. Add the **SSL_KEY_EXCHANGE** and **ASYM_ENCRYPT** protocols to a JGroups stack in your Data Grid configuration, as in the following example:

```
<infinispan>
  <jgroups>
    <!-- Creates a secure JGroups stack named "encrypt-tcp" that extends the default TCP
stack. -->
    <stack name="encrypt-tcp" extends="tcp">
      <!-- Adds a keystore that nodes use to perform certificate authentication. -->
      <!-- Uses the stack.combine and stack.position attributes to insert
SSL_KEY_EXCHANGE into the default TCP stack after VERIFY_SUSPECT2. -->
      <SSL_KEY_EXCHANGE keystore_name="mykeystore.jks"
                        keystore_password="changeit"
                        stack.combine="INSERT_AFTER"
                        stack.position="VERIFY_SUSPECT2"/>
      <!-- Configures ASYM_ENCRYPT -->
      <!-- Uses the stack.combine and stack.position attributes to insert ASYM_ENCRYPT into
the default TCP stack before pbcast.NAKACK2. -->
      <!-- The use_external_key_exchange = "true" attribute configures nodes to use the
`SSL_KEY_EXCHANGE` protocol for certificate authentication. -->
      <ASYM_ENCRYPT asym_keylength="2048"
                    asym_algorithm="RSA"
                    change_key_on_coord_leave = "false"
                    change_key_on_leave = "false"
                    use_external_key_exchange = "true"
                    stack.combine="INSERT_BEFORE"
                    stack.position="pbcast.NAKACK2"/>
    </stack>
  </jgroups>
  <cache-container name="default" statistics="true">
    <!-- Configures the cluster to use the JGroups stack. -->
```

```
    <transport cluster="${infinispan.cluster.name}"
               stack="encrypt-tcp"
               node-name="${infinispan.node.name:}"/>
  </cache-container>
</infinispan>
```

Verification

When you start your Data Grid cluster, the following log message indicates that the cluster is using the secure JGroups stack:

```
[org.infinispan.CLUSTER] ISPN000078: Starting JGroups channel cluster with stack
<encrypted_stack_name>
```

Data Grid nodes can join the cluster only if they use **ASYM_ENCRYPT** and can obtain the secret key from the coordinator node. Otherwise the following message is written to Data Grid logs:

```
[org.jgroups.protocols.ASYM_ENCRYPT] <hostname>: received message without encrypt header
from <hostname>; dropping it
```

Additional resources

- JGroups 5 Manual
- JGroups 5.5 Schema

12.8.4. Securing cluster transport with symmetric encryption

Configure Data Grid clusters to encrypt JGroups messages with secret keys from keystores that you provide.

Procedure

1. Create a keystore that contains a secret key.
2. Place the keystore on the classpath for each node in the cluster.
   For Data Grid Server, you put the keystore in the $RHDG_HOME directory.
3. Add the **SYM_ENCRYPT** protocol to a JGroups stack in your Data Grid configuration.

```
<infinispan>
  <jgroups>
    <!-- Creates a secure JGroups stack named "encrypt-tcp" that extends the default TCP stack. -->
    <stack name="encrypt-tcp" extends="tcp">
      <!-- Adds a keystore from which nodes obtain secret keys. -->
      <!-- Uses the stack.combine and stack.position attributes to insert SYM_ENCRYPT into the
default TCP stack after VERIFY_SUSPECT2. -->
      <SYM_ENCRYPT keystore_name="myKeystore.p12"
                   keystore_type="PKCS12"
                   store_password="changeit"
                   key_password="changeit"
                   alias="myKey"
                   stack.combine="INSERT_AFTER"
                   stack.position="VERIFY_SUSPECT2"/>
```

```
    </stack>
  </jgroups>
  <cache-container name="default" statistics="true">
    <!-- Configures the cluster to use the JGroups stack. -->
    <transport cluster="${infinispan.cluster.name}"
               stack="encrypt-tcp"
               node-name="${infinispan.node.name:}"/>
  </cache-container>
</infinispan>
```

Verification

When you start your Data Grid cluster, the following log message indicates that the cluster is using the secure JGroups stack:

```
[org.infinispan.CLUSTER] ISPN000078: Starting JGroups channel cluster with stack
<encrypted_stack_name>
```

Data Grid nodes can join the cluster only if they use **SYM_ENCRYPT** and can obtain the secret key from the shared keystore. Otherwise the following message is written to Data Grid logs:

```
[org.jgroups.protocols.SYM_ENCRYPT] <hostname>: received message without encrypt header from
<hostname>; dropping it
```

Additional resources

- JGroups 5 Manual
- JGroups 5.5 Schema

12.9. TCP AND UDP PORTS FOR CLUSTER TRAFFIC

Data Grid uses the following ports for cluster transport messages:

Default Port	Protocol	Description
7800	TCP/UDP	JGroups cluster bind port
46655	UDP	JGroups multicast
57800	TCP	Failure detection is provided by **FD_SOCK2**.

Cross-site replication

Data Grid uses the following ports for the JGroups RELAY2 protocol:

7900

For Data Grid clusters running on OpenShift.

7801

For other deployments.

12.10. VIRTUAL THREADS SUPPORT

Infinispan supports virtual threads, which can significantly improve application responsiveness and scalability under high concurrency. By default, they are enabled if you are running on JDK 21 or higher.



WARNING

On systems with JDK versions prior to 24 and low CPU counts (2 or less), Data Grid might experience thread pinning issues when using virtual threads. Thread pinning is a situation where virtual threads are unexpectedly bound to a limited number of OS threads, potentially leading to performance degradation or system freezes.

To work around this problem, virtual threads can be disabled as described in the procedure below, or the virtual thread scheduler parallelism may be increased with the Java option **-Djdk.virtualThreadScheduler.parallelism=<value>**. A common starting point for **<value>** on low-CPU systems is **4**.

12.10.1. Disable Virtual Threads

Disables the virtual threads pool.

1. Set the **-Dorg.infinispan.threads.virtual=false** option in the **JAVA_OPTS** environment variable before starting the Data Grid server.
   Alternatively, you can append this option directly to the command used to start the Data Grid server:

```
bin/server.sh -Dorg.infinispan.threads.virtual=false
```

2. Verify by checking the server logs. The following log entry should **not** be present:

```
Virtual threads support enabled
```

CHAPTER 13. CREATING REMOTE CACHES

When you create remote caches at runtime, Data Grid Server synchronizes your configuration across the cluster so that all nodes have a copy. For this reason you should always create remote caches dynamically with the following mechanisms:

- Data Grid Console
- Data Grid Command Line Interface (CLI)
- Hot Rod or HTTP clients

13.1. DEFAULT CACHE MANAGER

Data Grid Server provides a default Cache Manager that controls the lifecycle of remote caches. Starting Data Grid Server automatically instantiates the Cache Manager so you can create and delete remote caches and other resources like Protobuf schema.

After you start Data Grid Server and add user credentials, you can view details about the Cache Manager and get cluster information from Data Grid Console.

- Open **127.0.0.1:11222** in any browser.

You can also get information about the Cache Manager through the Command Line Interface (CLI) or REST API:

CLI

Run the **describe** command in the default container.

```
[//containers/default]> describe
```

REST

Open **127.0.0.1:11222/rest/v2/container/** in any browser.

Default Cache Manager configuration

XML

```
<infinispan>
  <!-- Creates a Cache Manager named "default" and enables metrics. -->
  <cache-container name="default"
                   statistics="true">
    <!-- Adds cluster transport that uses the default JGroups TCP stack. -->
    <transport cluster="${infinispan.cluster.name:cluster}"
               stack="${infinispan.cluster.stack:tcp}"
               node-name="${infinispan.node.name:}"/>
    <!-- Requires user permission to access caches and perform operations. -->
    <security>
      <authorization/>
    </security>
  </cache-container>
</infinispan>
```

JSON

```
{
  "infinispan" : {
    "jgroups" : {
      "transport" : "org.infinispan.remoting.transport.jgroups.JGroupsTransport"
    },
    "cache-container" : {
      "name" : "default",
      "statistics" : "true",
      "transport" : {
        "cluster" : "cluster",
        "node-name" : "",
        "stack" : "tcp"
      },
      "security" : {
        "authorization" : {}
      }
    }
  }
}
```

YAML

```
infinispan:
  jgroups:
    transport: "org.infinispan.remoting.transport.jgroups.JGroupsTransport"
  cacheContainer:
    name: "default"
    statistics: "true"
    transport:
      cluster: "cluster"
      nodeName: ""
      stack: "tcp"
    security:
      authorization: ~
```

13.2. CREATING CACHES WITH DATA GRID CONSOLE

Use Data Grid Console to create remote caches in an intuitive visual interface from any web browser.

Prerequisites

- Create a Data Grid user with **admin** permissions.
- Start at least one Data Grid Server instance.
- Have a Data Grid cache configuration.

Procedure

1. Open **127.0.0.1:11222/console/** in any browser.

2. Select **Create Cache** and follow the steps as Data Grid Console guides you through the process.

13.3. CREATING REMOTE CACHES WITH THE DATA GRID CLI

Use the Data Grid Command Line Interface (CLI) to add remote caches on Data Grid Server.

Prerequisites

- Create a Data Grid user with **admin** permissions.
- Start at least one Data Grid Server instance.
- Have a Data Grid cache configuration.

Procedure

1. Start the CLI.

```
bin/cli.sh
```

2. Run the **connect** command and enter your username and password when prompted.
3. Use the **create cache** command to create remote caches.
   For example, create a distributed cache named "distcache" as follows:

```
create cache distcache "<distributed-cache />"
```

You can use any configuration format supported by Data Grid. Create a cache using a JSON configuration as follows:

+

```
create cache json '{"distributed-cache":{"mode":"SYNC"}}'
```

It is possible to use a configuration stored in an external file. For example, create a cache named "mycache" from a file named **mycache.xml** as follows:

+

```
create cache --file=mycache.xml mycache
```

Verification

1. List all remote caches with the **ls** command.

```
ls caches
mycache
```

2. View cache configuration with the **describe** command.

```
describe caches/mycache
```

13.4. CREATING REMOTE CACHES FROM HOT ROD CLIENTS

Use the Data Grid Hot Rod API to create remote caches on Data Grid Server from Java, C++, .NET/C#, JS clients and more.

This procedure shows you how to use Hot Rod Java clients that create remote caches on first access. You can find code examples for other Hot Rod clients in the Data Grid Tutorials.

Prerequisites

- Create a Data Grid user with **admin** permissions.
- Start at least one Data Grid Server instance.
- Have a Data Grid cache configuration.

Procedure

- Invoke the **remoteCache()** method as part of your the **ConfigurationBuilder**.
- Set the **configuration** or **configuration_uri** properties in the **hotrod-client.properties** file on your classpath.

ConfigurationBuilder

```
File file = new File("path/to/infinispan.xml")
ConfigurationBuilder builder = new ConfigurationBuilder();
builder.remoteCache("another-cache")
      .configuration("<distributed-cache name=\"another-cache\"/>");
builder.remoteCache("my.other.cache")
      .configurationURI(file.toURI());
```

hotrod-client.properties

```
infinispan.client.hotrod.cache.another-cache.configuration=<distributed-cache name=\"another-cache\"/>
infinispan.client.hotrod.cache.[my.other.cache].configuration_uri=file:///path/to/infinispan.xml
```



IMPORTANT

If the name of your remote cache contains the **.** character, you must enclose it in square brackets when using **hotrod-client.properties** files.

Additional resources

- Hot Rod Client Configuration
- org.infinispan.client.hotrod.configuration.RemoteCacheConfigurationBuilder

13.5. CREATING REMOTE CACHES WITH THE REST API

Use the Data Grid REST API to create remote caches on Data Grid Server from any suitable HTTP client.

Prerequisites

- Create a Data Grid user with **admin** permissions.
- Start at least one Data Grid Server instance.
- Have a Data Grid cache configuration.

Procedure

- Invoke **POST** requests to **/rest/v2/caches/<cache_name>** with cache configuration in the payload.

Additional resources

- Creating and Managing Caches with the REST API

CHAPTER 14. RUNNING SCRIPTS AND TASKS ON DATA GRID SERVER

Add tasks and scripts to Data Grid Server deployments for remote execution from the Command Line Interface (CLI) and Hot Rod or REST clients. You can implement tasks as custom Java classes or define scripts in languages such as JavaScript.

14.1. ADDING TASKS TO DATA GRID SERVER DEPLOYMENTS

Add your custom server task classes to Data Grid Server.

Prerequisites

- Stop Data Grid Server if it is running.
  Data Grid Server does not support runtime deployment of custom classes.

Procedure

1. Add a **META-INF/services/org.infinispan.tasks.ServerTask** file that contains the fully qualified names of server tasks, for example:

   ```
   example.HelloTask
   ```

2. Package your server task implementation in a JAR file.
3. Copy the JAR file to the **$RHDG_HOME/server/lib** directory of your Data Grid Server installation.
4. Add your classes to the deserialization allow list in your Data Grid configuration. Alternatively set the allow list using system properties.

Reference

- Adding Java Classes to Deserialization Allow Lists
- Data Grid Configuration Schema

14.1.1. Data Grid Server tasks

Data Grid Server tasks are classes that extend the **org.infinispan.tasks.ServerTask** interface and generally include the following method calls:

setTaskContext()

Allows access to execution context information including task parameters, cache references on which tasks are executed, and so on. In most cases, implementations store this information locally and use it when tasks are actually executed. When using **SHARED** instantiation mode, the task should use a **ThreadLocal** to store the **TaskContext** for concurrent invocations.

getName()

Returns unique names for tasks. Clients invoke tasks with these names.

getExecutionMode()

Returns the execution mode for tasks.

- **TaskExecutionMode.ONE_NODE** only the node that handles the request executes the script. Although scripts can still invoke clustered operations. This is the default.
- **TaskExecutionMode.ALL_NODES** Data Grid uses clustered executors to run scripts across nodes. For example, server tasks that invoke stream processing need to be executed on a single node because stream processing is distributed to all nodes.

getInstantiationMode()

Returns the instantiation mode for tasks.

- **TaskInstantiationMode.SHARED** creates a single instance that is reused for every task execution on the same server. This is the default.
- **TaskInstantiationMode.ISOLATED** creates a new instance for every invocation.

call()

Computes a result. This method is defined in the **java.util.concurrent.Callable** interface and is invoked with server tasks.



IMPORTANT

Server task implementations must adhere to service loader pattern requirements. For example, implementations must have a zero-argument constructors.

The following **HelloTask** class implementation provides an example task that has one parameter. It also illustrates the use of a **ThreadLocal** to store the **TaskContext** for concurrent invocations.

```
package example;

import org.infinispan.tasks.ServerTask;
import org.infinispan.tasks.TaskContext;

public class HelloTask implements ServerTask<String> {

   private static final ThreadLocal<TaskContext> taskContext = new ThreadLocal<>();

   @Override
   public void setTaskContext(TaskContext ctx) {
      taskContext.set(ctx);
   }

   @Override
   public String call() throws Exception {
      TaskContext ctx = taskContext.get();
      String name = (String) ctx.getParameters().get().get("name");
      return "Hello " + name;
   }

   @Override
   public String getName() {
      return "hello-task";
   }

}
```

-

Reference

- org.infinispan.tasks.ServerTask
- java.util.concurrent.Callable.call()
- java.util.ServiceLoader

14.2. ADDING SCRIPTS TO DATA GRID SERVER DEPLOYMENTS

Use the command line interface to add scripts to Data Grid Server.

Prerequisites

Data Grid Server stores scripts in the **___script_cache** cache. If you enable cache authorization, users must have **CREATE** permissions to add to **___script_cache**.

Assign users the **deployer** role at minimum if you use default authorization settings.

Procedure

1. Define scripts as required.
   For example, create a file named **multiplication.js** that runs on a single Data Grid server, has two parameters, and uses JavaScript to multiply a given value:

```
// mode=local,language=javascript
multiplicand * multiplier
```

2. Create a CLI connection to Data Grid.
3. Use the **task** command to upload scripts, as in the following example:

```
task upload --file=multiplication.js multiplication
```

4. Verify that your scripts are available.

```
ls tasks
multiplication
```

14.2.1. Data Grid Server scripts

Data Grid Server scripting is based on the **javax.script** API and is compatible with any JVM-based ScriptEngine implementation.

Hello world

The following is a simple example that runs on a single Data Grid server, has one parameter, and uses JavaScript:

```
// mode=local,language=javascript,parameters=[greetee]
"Hello " + greetee
```

When you run the preceding script, you pass a value for the **greetee** parameter and Data Grid returns **"Hello ${value}"**.

14.2.1.1. Script metadata

Metadata provides additional information about scripts that Data Grid Server uses when running scripts.

Script metadata are **property=value** pairs that you add to comments in the first lines of scripts, such as the following example:

```
// name=test, language=javascript
// mode=local, parameters=[a,b,c]
```

- Use comment styles that match the scripting language (//, **;;**, **#**).
- Separate **property=value** pairs with commas.
- Separate values with single (') or double (") quote characters.

Table 14.1. Metadata Properties

Property	Description
mode	Defines the execution mode and has the following values: **local** only the node that handles the request executes the script. Although scripts can still invoke clustered operations. **distributed** Data Grid uses clustered executors to run scripts across nodes.
language	Specifies the ScriptEngine that executes the script.
extension	Specifies filename extensions as an alternative method to set the ScriptEngine.
role	Specifies roles that users must have to execute scripts.
parameters	Specifies an array of valid parameter names for this script. Invocations which specify parameters not included in this list cause exceptions.
datatype	Optionally sets the MediaType (MIME type) for storing data as well as parameter and return values. This property is useful for remote clients that support particular data formats only. Currently you can set only **text/plain; charset=utf-8** to use the String UTF-8 format for data.

14.2.1.2. Script bindings

Data Grid exposes internal objects as bindings for script execution.

Binding	Description
cache	Specifies the cache against which the script is run.
marshaller	Specifies the marshaller to use for serializing data to the cache.
cacheManager	Specifies the **cacheManager** for the cache.
scriptingManager	Specifies the instance of the script manager that runs the script. You can use this binding to run other scripts from a script.

14.2.1.3. Script parameters

Data Grid lets you pass named parameters as bindings for running scripts.

Parameters are **name,value** pairs, where **name** is a string and **value** is any value that the marshaller can interpret.

The following example script has two parameters, **multiplicand** and **multiplier**. The script takes the value of **multiplicand** and multiplies it with the value of **multiplier**.

```
// mode=local,language=javascript
multiplicand * multiplier
```

When you run the preceding script, Data Grid responds with the result of the expression evaluation.

14.2.2. Programmatically Creating Scripts

Add scripts with the Hot Rod **RemoteCache** interface as in the following example:

```
RemoteCache<String, String> scriptCache = cacheManager.getCache("___script_cache");
scriptCache.put("multiplication.js",
  "// mode=local,language=javascript\n" +
  "multiplicand * multiplier\n");
```

Reference

org.infinispan.client.hotrod.RemoteCache

14.3. RUNNING SCRIPTS AND TASKS

Use the command line interface to run tasks and scripts on Data Grid Server deployments. Alternatively you can execute scripts and tasks from Hot Rod clients.

Prerequisites

- Add scripts or tasks to Data Grid Server.

Procedure

1. Create a CLI connection to Data Grid.
2. Use the **task** command to run tasks and scripts, as in the following examples:
   - Execute a script named **multiplier.js** and specify two parameters:

     ```
     task exec multiplier.js -Pmultiplicand=10 -Pmultiplier=20
     200.0
     ```

   - Execute a task named **@@cache@names** to retrieve a list of all available caches:

     ```
     task exec @@cache@names
     ["___protobuf_metadata","mycache","___script_cache"]
     ```

Programmatic execution

- Call the **execute()** method to run scripts with the Hot Rod **RemoteCache** interface, as in the following examples:

Script execution

```
RemoteCache<String, Integer> cache = cacheManager.getCache();
// Create parameters for script execution.
Map<String, Object> params = new HashMap<>();
params.put("multiplicand", 10);
params.put("multiplier", 20);
// Run the script with the parameters.
Object result = cache.execute("multiplication.js", params);
```

Task execution

```
// Add configuration for a locally running server.
ConfigurationBuilder builder = new ConfigurationBuilder();
builder.addServer().host("127.0.0.1").port(11222);

// Connect to the server.
RemoteCacheManager cacheManager = new RemoteCacheManager(builder.build());

// Retrieve the remote cache.
RemoteCache<String, String> cache = cacheManager.getCache();

// Create task parameters.
Map<String, String> parameters = new HashMap<>();
parameters.put("name", "developer");

// Run the server task.
String greet = cache.execute("hello-task", parameters);
System.out.println(greet);
```

Additional resources

- org.infinispan.client.hotrod.RemoteCache

CHAPTER 15. CONFIGURING DATA GRID SERVER LOGGING

Data Grid Server uses Apache Log4j 2 to provide configurable logging mechanisms that capture details about the environment and record cache operations for troubleshooting purposes and root cause analysis.

15.1. DATA GRID SERVER LOG FILES

Data Grid writes server logs to the following files in the **$RHDG_HOME/server/log** directory:

server.log

Messages in human readable format, including boot logs that relate to the server startup. Data Grid creates this file when you start the server.

server.log.json

Messages in JSON format that let you parse and analyze Data Grid logs. Data Grid creates this file when you enable the **JSON-FILE** appender.

15.1.1. Configuring Data Grid Server logs

Data Grid uses Apache Log4j technology to write server log messages. You can configure server logs in the **log4j2.xml** file.

Procedure

1. Open **$RHDG_HOME/server/conf/log4j2.xml** with any text editor.
2. Change server logging as appropriate.
3. Save and close **log4j2.xml**.

Additional resources

- Apache Log4j manual

15.1.2. Log levels

Log levels indicate the nature and severity of messages.

Log level	Description
TRACE	Fine-grained debug messages, capturing the flow of individual requests through the application.
DEBUG	Messages for general debugging, not related to an individual request.
INFO	Messages about the overall progress of applications, including lifecycle events.

Log level	Description
WARN	Events that can lead to error or degrade performance.
ERROR	Error conditions that might prevent operations or activities from being successful but do not prevent applications from running.
FATAL	Events that could cause critical service failure and application shutdown.

In addition to the levels of individual messages presented above, the configuration allows two more values: **ALL** to include all messages, and **OFF** to exclude all messages.

15.1.3. Data Grid logging categories

Data Grid provides categories for **INFO**, **WARN**, **ERROR**, **FATAL** level messages that organize logs by functional area.

org.infinispan.CLUSTER

Messages specific to Data Grid clustering that include state transfer operations, rebalancing events, partitioning, and so on.

org.infinispan.CONFIG

Messages specific to Data Grid configuration.

org.infinispan.CONTAINER

Messages specific to the data container that include expiration and eviction operations, cache listener notifications, transactions, and so on.

org.infinispan.PERSISTENCE

Messages specific to cache loaders and stores.

org.infinispan.SECURITY

Messages specific to Data Grid security.

org.infinispan.SERVER

Messages specific to Data Grid servers.

org.infinispan.XSITE

Messages specific to cross-site replication operations.

15.1.4. Log appenders

Log appenders define how Data Grid Server records log messages.

CONSOLE

Write log messages to the host standard out (**stdout**) or standard error (**stderr**) stream. Uses the **org.apache.logging.log4j.core.appender.ConsoleAppender** class by default.

FILE

Write log messages to a file.
Uses the **org.apache.logging.log4j.core.appender.RollingFileAppender** class by default.

JSON-FILE

Write log messages to a file in JSON format.
Uses the **org.apache.logging.log4j.core.appender.RollingFileAppender** class by default.

15.1.5. Log pattern formatters

The **CONSOLE** and **FILE** appenders use a **PatternLayout** to format the log messages according to a **pattern**.

An example is the default pattern in the FILE appender:
%d{yyyy-MM-dd HH:mm:ss,SSS} %-5p (%t) [%c{1}] %m%throwable%n

- **%d{yyyy-MM-dd HH:mm:ss,SSS}** adds the current time and date.
- **%-5p** specifies the log level, aligned to the right.
- **%t** adds the name of the current thread.
- **%c{1}** adds the short name of the logging category.
- **%m** adds the log message.
- **%throwable** adds the exception stack trace.
- **%n** adds a new line.

Patterns are fully described in the **PatternLayout** documentation .

15.1.6. Enabling the JSON log handler

Data Grid Server provides a log handler to write messages in JSON format.

Prerequisites

- Stop Data Grid Server if it is running.
  You cannot dynamically enable log handlers.

Procedure

1. Open **$RHDG_HOME/server/conf/log4j2.xml** with any text editor.
2. Uncomment the **JSON-FILE** appender and comment out the **FILE** appender:

```
<!--<AppenderRef ref="FILE"/>-->
<AppenderRef ref="JSON-FILE"/>
```

3. Optionally configure the JSON appender and JSON layout as required.
4. Save and close **log4j2.xml**.

When you start Data Grid, it writes each log message as a JSON map in the following file:
$RHDG_HOME/server/log/server.log.json

Additional resources

- RollingFileAppender
- JSONLayout

15.2. ACCESS LOGS

Access logs record all inbound client requests for Hot Rod and REST endpoints to files in the **$RHDG_HOME/server/log** directory.

org.infinispan.HOTROD_ACCESS_LOG

Logging category that writes Hot Rod access messages to a **hotrod-access.log** file.

org.infinispan.REST_ACCESS_LOG

Logging category that writes REST access messages to a **rest-access.log** file.

15.2.1. Enabling access logs

To record Hot Rod, REST and Memcached endpoint access messages, you need to enable the logging categories in **log4j2.xml**.

Procedure

1. Open **$RHDG_HOME/server/conf/log4j2.xml** with any text editor.
2. Change the level for the **org.infinispan.HOTROD_ACCESS_LOG**, **org.infinispan.REST_ACCESS_LOG** and **org.infinispan.MEMCACHED_ACCESS_LOG** logging categories to **TRACE**.
3. Save and close **log4j2.xml**.

```
<Logger name="org.infinispan.HOTROD_ACCESS_LOG" additivity="false" level="TRACE">
  <AppenderRef ref="HR-ACCESS-FILE"/>
</Logger>
```

15.2.2. Access log properties

The default format for access logs is as follows:

```
%X{address} %X{user} [%d{dd/MMM/yyyy:HH:mm:ss Z}] &amp;quot;%X{method} %m
%X{protocol}&amp;quot; %X{status} %X{requestSize} %X{responseSize} %X{duration}%n
```

The preceding format creates log entries such as the following:

127.0.0.1 - [DD/MM/YYYY:HH:MM:SS +0000] "PUT /rest/v2/caches/default/key HTTP/1.1" 404 5 77 10

Logging properties use the **%X{name}** notation and let you modify the format of access logs. The following are the default logging properties:

Property	Description
address	Either the **X-Forwarded-For** header or the client IP address.
user	Principal name, if using authentication.
method	The protocol-specific method used. **PUT**, **GET**, and so on.
protocol	Protocol used. **HTTP/1.1**, **HTTP/2**, **HOTROD/2.9**, **MCTXT**, **MCBIN** and so on.
status	An HTTP status code for the REST endpoint. **OK** or an exception for the Hot Rod endpoint.
requestSize	Size, in bytes, of the request.
responseSize	Size, in bytes, of the response.
duration	Number of milliseconds that the server took to handle the request.

TIP

Use the header name prefixed with **h:** to log headers that were included in requests; for example, **%X{h:User-Agent}**.

15.3. AUDIT LOGS

Audit logs let you track changes to your Data Grid Server deployment so you know when changes occur and which users make them. Enable and configure audit logging to record server configuration events and administrative operations.

org.infinispan.AUDIT

Logging category that writes security audit messages to an **audit.log** file in the **$RHDG_HOME/server/log** directory.

15.3.1. Enabling audit logging

To record security audit messages, you need to enable the logging category in **log4j2.xml**.

Procedure

1. Open **$RHDG_HOME/server/conf/log4j2.xml** with any text editor.
2. Change the level for the **org.infinispan.AUDIT** logging category to **INFO**.

3. Save and close **log4j2.xml**.

```
<!-- Set to INFO to enable audit logging -->
<Logger name="org.infinispan.AUDIT" additivity="false" level="INFO">
  <AppenderRef ref="AUDIT-FILE"/>
</Logger>
```

15.3.2. Configuring audit logging appenders

Apache Log4j provides different appenders that you can use to send audit messages to a destination other than the default log file. For instance, if you want to send audit logs to a syslog daemon, JDBC database, or Apache Kafka server, you can configure an appender in **log4j2.xml**.

Procedure

1. Open **$RHDG_HOME/server/conf/log4j2.xml** with any text editor.
2. Comment or remove the default **AUDIT-FILE** rolling file appender.

```
<!--RollingFile name="AUDIT-FILE"
  ...
</RollingFile-->
```

3. Add the desired logging appender for audit messages.
   For example, you could add a logging appender for a Kafka server as follows:

```
<Kafka name="AUDIT-KAFKA" topic="audit">
  <PatternLayout pattern="%date %message"/>
  <Property name="bootstrap.servers">localhost:9092</Property>
</Kafka>
```

4. Save and close **log4j2.xml**.

Additional resources

- Log4j Appenders

15.3.3. Using custom audit logging implementations

You can create custom implementations of the **org.infinispan.security.AuditLogger** API if configuring Log4j appenders does not meet your needs.

Prerequisites

- Implement **org.infinispan.security.AuditLogger** as required and package it in a JAR file.

Procedure

1. Add your JAR to the **server/lib** directory in your Data Grid Server installation.
2. Specify the fully qualified class name of your custom audit logger as the value for the **audit-logger** attribute on the **authorization** element in your cache container security configuration.

For example, the following configuration defines **my.package.CustomAuditLogger** as the class for logging audit messages:

```
<infinispan>
  <cache-container>
    <security>
      <authorization audit-logger="my.package.CustomAuditLogger"/>
    </security>
  </cache-container>
</infinispan>
```

Additional resources

- **org.infinispan.security.AuditLogger**

CHAPTER 16. PERFORMING ROLLING UPGRADES FOR DATA GRID SERVER CLUSTERS

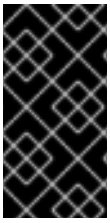
Perform rolling upgrades of your Data Grid clusters to change between versions without downtime or data loss and migrate data over the Hot Rod protocol.

16.1. SETTING UP TARGET DATA GRID CLUSTERS

Create a cluster that uses the Data Grid version to which you plan to upgrade and then connect the source cluster to the target cluster using a remote cache store.

Prerequisites

- Install Data Grid Server nodes with the desired version for your target cluster.



IMPORTANT

Ensure the network properties for the target cluster do not overlap with those for the source cluster. You should specify unique names for the target and source clusters in the JGroups transport configuration. Depending on your environment you can also use different network interfaces and port offsets to separate the target and source clusters.

Procedure

1. Create a remote cache store configuration, in JSON format, that allows the target cluster to connect to the source cluster.
   Remote cache stores on the target cluster use the Hot Rod protocol to retrieve data from the source cluster.

```
{
  "remote-store": {
    "cache": "myCache",
    "shared": true,
    "security": {
      "authentication": {
        "digest": {
          "username": "username",
          "password": "changeme",
          "realm": "default"
        }
      }
    },
    "remote-server": [
      {
        "host": "127.0.0.1",
        "port": 12222
      }
    ]
  }
}
```

2. Use the Data Grid Command Line Interface (CLI) or REST API to add the remote cache store configuration to the target cluster so it can connect to the source cluster.

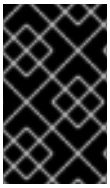
- CLI: Use the **migrate cluster connect** command on the target cluster.

```
[//containers/default]> migrate cluster connect -c myCache --file=remote-store.json
```

- REST API: Invoke a POST request that includes the remote store configuration in the payload with the **rolling-upgrade/source-connection** method.

```
POST /rest/v2/caches/myCache/rolling-upgrade/source-connection
```

3. Repeat the preceding step for each cache that you want to migrate.
4. Switch clients over to the target cluster, so it starts handling all requests.
   a. Update client configuration with the location of the target cluster.
   b. Restart clients.



IMPORTANT

If you need to migrate Indexed caches you must first migrate the internal **___protobuf_metadata** cache so that the .proto schemas defined on the source cluster will also be present on the target cluster.

Additional resources

- Remote cache store configuration schema

16.2. SYNCHRONIZING DATA TO TARGET CLUSTERS

When you set up a target Data Grid cluster and connect it to a source cluster, the target cluster can handle client requests using a remote cache store and load data on demand. To completely migrate data to the target cluster, so you can decommission the source cluster, you can synchronize data. This operation reads data from the source cluster and writes it to the target cluster. Data migrates to all nodes in the target cluster in parallel, with each node receiving a subset of the data. You must perform the synchronization for each cache that you want to migrate to the target cluster.

Prerequisites

- Set up a target cluster with the appropriate Data Grid version.

Procedure

1. Start synchronizing each cache that you want to migrate to the target cluster with the Data Grid Command Line Interface (CLI) or REST API.
   - CLI: Use the **migrate cluster synchronize** command.

```
migrate cluster synchronize -c myCache
```

   - REST API: Use the **?action=sync-data** parameter with a POST request.

```
POST /rest/v2/caches/myCache?action=sync-data
```

When the operation completes, Data Grid responds with the total number of entries copied to the target cluster.

2. Disconnect each node in the target cluster from the source cluster.

   - CLI: Use the **migrate cluster disconnect** command.

     ```
     migrate cluster disconnect -c myCache
     ```

   - REST API: Invoke a DELETE request.

     ```
     DELETE /rest/v2/caches/myCache/rolling-upgrade/source-connection
     ```

Next steps

After you synchronize all data from the source cluster, the rolling upgrade process is complete. You can now decommission the source cluster.

CHAPTER 17. TROUBLESHOOTING DATA GRID SERVER DEPLOYMENTS

Gather diagnostic information about Data Grid Server deployments and perform troubleshooting steps to resolve issues.

17.1. GETTING DIAGNOSTIC REPORTS FROM DATA GRID SERVER

Data Grid Server provides aggregated reports in **tar.gz** archives that contain diagnostic information about server instances and host systems. The report provides details about CPU, memory, open files, network sockets and routing, threads, in addition to configuration and log files.

Procedure

1. Create a CLI connection to Data Grid Server.
2. Use the **server report** command to download a **tar.gz** archive:

```
server report
Downloaded report 'infinispan-<hostname>-<timestamp>-report.tar.gz'
```

   The command responds with the name of the report, as in the following example:

```
Downloaded report 'infinispan-<hostname>-<timestamp>-report.tar.gz'
```

3. Move the **tar.gz** file to a suitable location on your filesystem.
4. Extract the **tar.gz** file with any archiving tool.

17.1.1. Understanding the report

The report generated with the CLI contains different categories of files containing information about the environment and the application. We split the files into subcategories and briefly describe how to read each one.

17.1.1.1. Environment

These files contain metrics of the environment the Data Grid server is running on. In addition to understanding how the application behaves, external factors might cause issues, such as misconfiguration in the network, many processes running in the same host, etc. The following files shed light on these factors for investigation:

Host:

- **cpuinfo**: Provides information about the host's CPU.
- **df**: Provides information about the host's filesystem.
- **lsof**: Describes the open files in the host.
- **meminfo**: Provides information about the host's memory.
- **os-release**: Describes the current OS the server is running on.

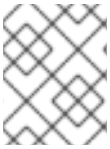
- **uname**: Additional system information.

Network:

- **ip-address**: Describes the network interfaces in the host and the address and capabilities of each one.
- **ip-maddress**: Describes the multicast address on each network interface in the host.
- **ip-mroute**: Describes the multicast routing table in the host.
- **ip-route**: Describes the the routing table in the host.
- **ss-tcp**: List the TCP sockets in the host.
- **ss-udp**: List the UDP sockets in the host.

17.1.1.2. Data Grid server

Additionally, the report collects information from the running server. The report generates a thread dump utilizing the **jstack** command. The server logs and configurations are also included in the files.



NOTE

The report tries to collect all this information. However, some files might be missing if the host cannot run a specific command.

17.2. CHANGING DATA GRID SERVER LOGGING CONFIGURATION AT RUNTIME

Modify the logging configuration for Data Grid Server at runtime to temporarily adjust logging to troubleshoot issues and perform root cause analysis.

Modifying the logging configuration through the CLI is a runtime-only operation, which means that changes:

- Are not saved to the **log4j2.xml** file. Restarting server nodes or the entire cluster resets the logging configuration to the default properties in the **log4j2.xml** file.
- Apply only to the nodes in the cluster when you invoke the CLI. Nodes that join the cluster after you change the logging configuration use the default properties.

Procedure

1. Create a CLI connection to Data Grid Server.
2. Use the **logging** to make the required adjustments.
   - List all appenders defined on the server:

     ```
     logging list-appenders
     ```

     The command provides a JSON response such as the following:

```
{
  "STDOUT" : {
    "name" : "STDOUT"
  },
  "JSON-FILE" : {
    "name" : "JSON-FILE"
  },
  "HR-ACCESS-FILE" : {
    "name" : "HR-ACCESS-FILE"
  },
  "FILE" : {
    "name" : "FILE"
  },
  "REST-ACCESS-FILE" : {
    "name" : "REST-ACCESS-FILE"
  }
}
```

- List all logger configurations defined on the server:

```
logging list-loggers
```

The command provides a JSON response such as the following:

```
[ {
  "name" : "",
  "level" : "INFO",
  "appenders" : [ "STDOUT", "FILE" ]
}, {
  "name" : "org.infinispan.HOTROD_ACCESS_LOG",
  "level" : "INFO",
  "appenders" : [ "HR-ACCESS-FILE" ]
}, {
  "name" : "com.arjuna",
  "level" : "WARN",
  "appenders" : [ ]
}, {
  "name" : "org.infinispan.REST_ACCESS_LOG",
  "level" : "INFO",
  "appenders" : [ "REST-ACCESS-FILE" ]
} ]
```

- Add and modify logger configurations with the **set** subcommand
  For example, the following command sets the logging level for the **org.infinispan** package to **DEBUG**:

```
logging set --level=DEBUG org.infinispan
```

- Remove existing logger configurations with the **remove** subcommand.
  For example, the following command removes the **org.infinispan** logger configuration, which means the root configuration is used instead:

```
logging remove org.infinispan
```

17.2.1. Access logging

The command also allows for enabling access logs during runtime to analyze issues. You can enable the access logging for a specific endpoint and utilize the CLI command to parse those files to generate server-side metrics. This technique is useful in identifying whether latency is an issue from the connection between client and server or intra-cluster.

The workflow for this analysis involves a few steps:

1. Enable access logging with the CLI for the endpoint you are analyzing.

   ```
   logging set --level=TRACE org.infinispan.RESP_ACCESS_LOG
   ```

   In this example, enabling the access log for the RESP endpoint.

2. Run the tests to reproduce the issue or workload to measure.

3. Disable access logging with the CLI.

   ```
   logging set --level=INFO org.infinispan.RESP_ACCESS_LOG
   ```

4. Download the access log files.
   The **server report** command is also capable of downloading the log files.

With the access log files available, you can utilize the CLI command to analyze the server's behavior more deeply. The CLI command provides information about long-running commands, segmentation per client, and operations.

Categories
Data Grid provides an access log for each endpoint available on the server. You can edit the logging level to enable each endpoint individually. The loggers available for each endpoint are:

- **org.infinispan.HOTROD_ACCESS_LOG**: Enables the access log for the Hot Rod endpoint.
- **org.infinispan.REST_ACCESS_LOG**: Enables the access log for the REST endpoint.
- **org.infinispan.MEMCACHED_ACCESS_LOG**: Enables the access log for the Memcached endpoint.
- **org.infinispan.RESP_ACCESS_LOG**: Enables the access log for the RESP endpoint.

Observe that each logger can be activated individually and independently of the others.

17.3. GATHERING RESOURCE STATISTICS FROM THE CLI

You can inspect server-collected statistics for some Data Grid Server resources with the **stats** command.

Use the **stats** command either from the context of a resource that provides statistics (containers, caches) or with a path to such a resource:

```
stats
```

```
{
  "statistics_enabled" : true,
```

```
  "number_of_entries" : 0,
  "hit_ratio" : 0.0,
  "read_write_ratio" : 0.0,
  "time_since_start" : 0,
  "time_since_reset" : 49,
  "current_number_of_entries" : 0,
  "current_number_of_entries_in_memory" : 0,
  "off_heap_memory_used" : 0,
  "data_memory_used" : 0,
  "stores" : 0,
  "retrievals" : 0,
  "hits" : 0,
  "misses" : 0,
  "remove_hits" : 0,
  "remove_misses" : 0,
  "evictions" : 0,
  "average_read_time" : 0,
  "average_read_time_nanos" : 0,
  "average_write_time" : 0,
  "average_write_time_nanos" : 0,
  "average_remove_time" : 0,
  "average_remove_time_nanos" : 0,
  "required_minimum_number_of_nodes" : -1
}
```

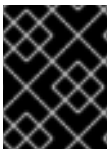
```
stats /containers/default/caches/mycache
```

```
{
  "time_since_start" : -1,
  "time_since_reset" : -1,
  "current_number_of_entries" : -1,
  "current_number_of_entries_in_memory" : -1,
  "off_heap_memory_used" : -1,
  "data_memory_used" : -1,
  "stores" : -1,
  "retrievals" : -1,
  "hits" : -1,
  "misses" : -1,
  "remove_hits" : -1,
  "remove_misses" : -1,
  "evictions" : -1,
  "average_read_time" : -1,
  "average_read_time_nanos" : -1,
  "average_write_time" : -1,
  "average_write_time_nanos" : -1,
  "average_remove_time" : -1,
  "average_remove_time_nanos" : -1,
  "required_minimum_number_of_nodes" : -1
}
```

17.4. JVM SETTINGS FOR DATA GRID

You can define Java Virtual Machine (JVM) settings for Data Grid either by editing the **server.conf** configuration file, or by setting the **JAVA_OPTS** and **JAVA_OPTIONS** environment variables. Set the

JAVA_OPTIONS variable if you just want to append JVM settings to those that are automatically set by the **server.conf** file. Set the **JAVA_OPTS** variable if you want to completely override the JVM settings.



IMPORTANT

If you are running Data Grid in a container do not set **Xmx** or **Xms** because the values are automatically calculated from the container settings to be 70% of the container size.

Editing the configuration file

You can edit the required values in the **server.conf** configuration file. For example, to set the options to pass to the JVM, edit the following lines:

```
if [ "$JAVA_OPTS" = "" ]; then
  JAVA_OPTS="..."
else
  echo "JAVA_OPTS already set in environment; overriding default settings with values:
$JAVA_OPTS"
fi
```

You can uncomment the existing example settings as well. For example, to configure Java Platform Debugger Architecture (JPDA) settings for remote socket debugging, update the file as follows:

```
# Sample JPDA settings for remote socket debugging
JAVA_OPTS="$JAVA_OPTS -
agentlib:jdwp=transport=dt_socket,address=8787,server=y,suspend=n"
```

Additionally, you can add more settings to **JAVA_OPTS** like this:

```
JAVA_OPTS="$JAVA_OPTS <key_1>=<value_1>, ..., <key_N>=<value_N> "
```

Setting an environment variable

You can override the settings in **server.conf** configuration file by setting the **JAVA_OPTS** environment variable. For example:

Linux

```
export JAVA_OPTS="-Xmx1024M"
```

Windows

```
set JAVA_OPTS="-Xmx1024M"
```

17.4.1. Garbage Collection Logging

Data Grid automatically enables garbage collection logs to the **$ISPN_HOME/server/log** directory. Disable the logs by setting the **GC_LOG** environment variable to **false** before launching the server.

Linux

```
export GC_LOG="false"
```

Windows

```
set "GC_LOG=false"
```

17.4.2. Heap Dump on OutOfMemoryError

Data Grid enables automatic heap dumps on **OutOfMemoryError** to the **$ISPN_HOME/server/log** directory. Heap dumps are useful for troubleshooting memory issues. You can use tools such as **jhat** (included in the JDK), VisualVM or Eclipse Memory Analyzer to inspect heap dumps. Disable heap dumps by setting the **HEAP_DUMP** environment variable to **false** before launching the server.

Linux

```
export HEAP_DUMP="false"
```

Windows

```
set "HEAP_DUMP=false"
```



WARNING

Heap dumps are snapshots of the in-memory data you have stored in Data Grid caches and may therefore contain sensitive data. Consider sanitizing the heap dumps before sharing them with third parties by using a tool such as the Heap Dump Tool.

17.5. VIRTUAL THREADS SUPPORT

Infinispan supports virtual threads, which can significantly improve application responsiveness and scalability under high concurrency. By default, they are enabled if you are running on JDK 21 or higher.



WARNING

On systems with JDK versions prior to 24 and low CPU counts (2 or less), Data Grid might experience thread pinning issues when using virtual threads. Thread pinning is a situation where virtual threads are unexpectedly bound to a limited number of OS threads, potentially leading to performance degradation or system freezes.

To work around this problem, virtual threads can be disabled as described in the procedure below, or the virtual thread scheduler parallelism may be increased with the Java option **-Djdk.virtualThreadScheduler.parallelism=<value>**. A common starting point for **<value>** on low-CPU systems is **4**.

17.5.1. Disable Virtual Threads

Disables the virtual threads pool.

1. Set the **-Dorg.infinispan.threads.virtual=false** option in the **JAVA_OPTS** environment variable before starting the Data Grid server.
   Alternatively, you can append this option directly to the command used to start the Data Grid server:

```
bin/server.sh -Dorg.infinispan.threads.virtual=false
```

2. Verify by checking the server logs. The following log entry should **not** be present:

```
Virtual threads support enabled
```

17.6. ACCESSING CLUSTER HEALTH VIA REST

Get Data Grid cluster health via the REST API.

Procedure

- Invoke a **GET** request to retrieve cluster health.

```
GET /rest/v2/container/health
```

Data Grid responds with a **JSON** document such as the following:

```
{
   "cluster_health":{
      "cluster_name":"ISPN",
      "health_status":"HEALTHY",
      "number_of_nodes":2,
      "node_names":[
         "NodeA-36229",
         "NodeB-28703"
      ]
   },
   "cache_health":[
      {
         "status":"HEALTHY",
         "cache_name":"___protobuf_metadata"
      },
      {
         "status":"HEALTHY",
         "cache_name":"cache2"
      },
      {
         "status":"HEALTHY",
         "cache_name":"mycache"
      },
      {
         "status":"HEALTHY",
         "cache_name":"cache1"
      }
```

```
    ]
}
```

TIP

Get Cache Manager status as follows:

```
GET /rest/v2/container/health/status
```

Reference

See the *REST v2 (version 2) API* documentation for more information.

17.7. ACCESSING CLUSTER HEALTH VIA JMX

Retrieve Data Grid cluster health statistics via JMX.

Procedure

1. Connect to Data Grid server using any JMX capable tool such as JConsole and navigate to the following object:

   ```
   org.infinispan:type=CacheManager,name="default",component=CacheContainerHealth
   ```

2. Select available MBeans to retrieve cluster health statistics.

17.8. BENCHMARK THE DATA GRID SERVER

Data Grid provides a built-in benchmark utility accessible through the CLI to validate server configuration changes. This benchmark will run operations for writes and reads, submitting requests as fast as possible. Since this might be a contrived scenario that does not represent a real production environment, the numbers should be read carefully. Nevertheless, this approach is simple enough to verify for configuration changes.

Check the CLI command documentation for more information.

CHAPTER 18. REFERENCE

18.1. DATA GRID SERVER 8.6.0 README

Information about Data Grid Server 16.0.6.Final-redhat-00001 distribution.

18.1.1. Requirements

Data Grid Server requires JDK 11 or later.

18.1.2. Starting servers

Use the **server** script to run Data Grid Server instances.

Unix / Linux

```
$RHDG_HOME/bin/server.sh
```

Windows

```
$RHDG_HOME\bin\server.bat
```

TIP

Include the **--help** or **-h** option to view command arguments.

18.1.3. Stopping servers

Use the **shutdown** command with the CLI to perform a graceful shutdown.

Alternatively, enter Ctrl-C from the terminal to interrupt the server process or kill it via the TERM signal.

18.1.4. Configuration

Server configuration extends Data Grid configuration with the following server-specific elements:

cache-container
: Defines cache containers for managing cache lifecycles.

endpoints
: Enables and configures endpoint connectors for client protocols.

security
: Configures endpoint security realms.

socket-bindings
: Maps endpoint connectors to interfaces and ports.

The default configuration file is **$RHDG_HOME/server/conf/infinispan.xml**.

infinispan.xml

Provides configuration to run Data Grid Server using default cache container with statistics and authorization enabled. Demonstrates how to set up authentication and authorization using security realms.

Data Grid provides other ready-to-use configuration files that are primarily for development and testing purposes.

$RHDG_HOME/server/conf/ provides the following configuration files:

infinispan-dev-mode.xml

Configures Data Grid Server specifically for cross-site replication with IP multicast discovery. The configuration provides **BASIC** authentication to connect to the Hot Rod and REST endpoints. The configuration is designed for development mode and should not be used in production environments.

infinispan-local.xml

Configures Data Grid Server without clustering capabilities.

infinispan-xsite.xml

Configures cross-site replication on a single host and uses IP multicast for discovery.

infinispan-memcached.xml

Configures Data Grid Server to behave like a default Memcached server, listening on port 11221 and without authentication.

infinispan-resp.xml

Configures Data Grid Server to behave like a default Redis server, listening on port 6379 and without authentication.

log4j2.xml

Configures Data Grid Server logging.

Use different configuration files with the **-c** argument, as in the following example that starts a server without clustering capabilities:

Unix / Linux

```
$RHDG_HOME/bin/server.sh -c infinispan-local.xml
```

Windows

```
$RHDG_HOME\bin\server.bat -c infinispan-local.xml
```

18.1.5. Bind address

Data Grid Server binds to the loopback IP address **localhost** on your network by default.

Use the **-b** argument to set a different IP address, as in the following example that binds to all network interfaces:

Unix / Linux

```
$RHDG_HOME/bin/server.sh -b 0.0.0.0
```

Windows

```
$RHDG_HOME\bin\server.bat -b 0.0.0.0
```

18.1.6. Bind port

Data Grid Server listens on port **11222** by default.

Use the **-p** argument to set an alternative port:

Unix / Linux

```
$RHDG_HOME/bin/server.sh -p 30000
```

Windows

```
$RHDG_HOME\bin\server.bat -p 30000
```

18.1.7. Clustering address

Data Grid Server configuration defines cluster transport so multiple instances on the same network discover each other and automatically form clusters.

Use the **-k** argument to change the IP address for cluster traffic:

Unix / Linux

```
$RHDG_HOME/bin/server.sh -k 192.168.1.100
```

Windows

```
$RHDG_HOME\bin\server.bat -k 192.168.1.100
```

18.1.8. Cluster stacks

JGroups stacks configure the protocols for cluster transport. Data Grid Server uses the **tcp** stack by default.

Use alternative cluster stacks with the **-j** argument, as in the following example that uses UDP for cluster transport:

Unix / Linux

```
$RHDG_HOME/bin/server.sh -j udp
```

Windows

```
$RHDG_HOME\bin\server.bat -j udp
```

18.1.9. Authentication

Data Grid Server requires authentication.

Create a username and password with the CLI as follows:

Unix / Linux

```
$RHDG_HOME/bin/cli.sh user create username -p "qwer1234!"
```

Windows

```
$RHDG_HOME\bin\cli.bat user create username -p "qwer1234!"
```

18.1.10. Server home directory

Data Grid Server uses **infinispan.server.home.path** to locate the contents of the server distribution on the host filesystem.

The server home directory, referred to as **$RHDG_HOME**, contains the following folders:

```
├─── bin
├─── boot
├─── docs
├─── lib
├─── server
└─── static
```

Folder	Description
/bin	Contains scripts to start servers and CLI.
/boot	Contains **JAR** files to boot servers.
/docs	Provides configuration examples, schemas, component licenses, and other resources.
/lib	Contains **JAR** files that servers require internally. Do not place custom **JAR** files in this folder.
/server	Provides a root folder for Data Grid Server instances.
/static	Contains static resources for Data Grid Console.

18.1.11. Server root directory

Data Grid Server uses **infinispan.server.root.path** to locate configuration files and data for Data Grid Server instances.

You can create multiple server root folders in the same directory or in different directories and then specify the locations with the **-s** or **--server-root** argument, as in the following example:

Unix / Linux

```
-
```

```
$RHDG_HOME/bin/server.sh -s server2
```

Windows

```
$RHDG_HOME\bin\server.bat -s server2
```

Each server root directory contains the following folders:

```
├─── server
│    ├─── conf
│    ├─── data
│    ├─── lib
│    └─── log
```

Folder	Description	System property override
/server/conf	Contains server configuration files.	**infinispan.server.config.path**
/server/data	Contains data files organized by container name.	**infinispan.server.data.path**
/server/lib	Contains server extension files. This directory is scanned recursively and used as a classpath.	**infinispan.server.lib.path** Separate multiple paths with the following delimiters: **:** on Unix / Linux **;** on Windows
/server/log	Contains server log files.	**infinispan.server.log.path**

18.1.12. Logging

Configure Data Grid Server logging with the **log4j2.xml** file in the **server/conf** folder.

Use the **--logging-config=<path_to_logfile>** argument to use custom paths, as follows:

Unix / Linux

```
$RHDG_HOME/bin/server.sh --logging-config=/path/to/log4j2.xml
```

TIP

To ensure custom paths take effect, do not use the ~ shortcut.

Windows

```
$RHDG_HOME\bin\server.bat --logging-config=path\to\log4j2.xml
```